esprit
Se former autrement

2025-2026

PROJET DE FIN D'ÉTUDES

DIPLÔME NATIONAL D'INGÉNIEUR

SPÉCIALITÉ : INFORMATIQUE

Conception et développement d'une plateforme web et mobile pour la digitalisation de la gestion des tickets

Réalisé par: SyrineTrabelsi

Encadré par:

Encadrant ESPRIT: M. Anis Belhadjhassin

Encadrant Entreprise: M. Nizar ktari

TUNISYS
تونسيس

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I validate the submission of the student's report:

- Nom & Prénom /Name & Surname : Syrine Trabelsi

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname : Mr.Nizar Ktari

Cachet & Signature / Stamp & Signature

TUNISYS تونسيس
124, Av. de la Liberté - 1002 Tunis
Tél: 71 791 699
Fax: 71 788 532 - 71 786 188

Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname : Mr.Anis Belhadjhassin

Signature / Signature

Ce formulaire doit être rempli, signé et <u>scanné</u>/This form must be completed, signed and <u>scanned</u>.

Ce formulaire doit être introduit après la page de garde/ This form must be inserted after the cover p

Remerciements

“

Au terme de ce travail, j'adresse mes remerciements les plus sincères à :

M. Nizar Ktari

Manager de l'entreprise ***Tunisys*** *et mon responsable de stage, pour son soutien technique, ses critiques constructives ainsi que l'aide précieuse qu'il m'a accordée tout au long de ce projet.*

M. Anis Belhajhassin

Pour son encadrement, sa disponibilité, son écoute ainsi que ses précieux conseils, qui ont grandement contribué à la réussite de ce projet.

Je tiens également à exprimer ma gratitude à toute l'équipe de ***Tunisys*** *pour son accueil chaleureux, ses encouragements et l'atmosphère agréable qui a régné durant toute la période du stage.*

Enfin, je souhaite remercier les membres du jury qui ont accepté d'évaluer ce travail, en espérant qu'ils y trouveront la rigueur, la clarté et la motivation attendues.

”

Dédicace

“

*À **l'âme de mon cher père,***

qui restera toujours présent dans mon cœur. Que Dieu lui accorde Sa miséricorde. Son souvenir, ses valeurs et ses encouragements continueront à m'accompagner tout au long de ma vie.

*À **ma chère mère,***

pour son amour inconditionnel, ses sacrifices, son soutien permanent et ses prières qui m'ont toujours donné la force d'avancer et de croire en moi.

*À **mon frère et ma sœur,***

pour leur soutien moral, leurs encouragements et leur présence dans les moments les plus importants de mon parcours.

*À **mes chères amies Donia et Sabrine,***

pour leur aide, leur gentillesse, leurs conseils précieux et tous les moments de motivation et de soutien partagés durant cette période.

”

-Syrine Trabelsi

Résumé

Le présent travail a été réalisé dans le cadre du projet de fin d'études pour l'obtention du diplôme d'ingénieur en informatique. Ce projet porte sur le développement de **TSS**, une plateforme intelligente dédiée à la gestion et au suivi des services techniques ainsi qu'à la gestion des tickets d'incidents.

L'objectif principal de cette application est d'optimiser le processus de traitement des demandes techniques en facilitant la communication entre les différents acteurs, notamment les coordinateurs, les techniciens et le client. La plateforme permet la création, le suivi et la gestion des tickets, tout en assurant une meilleure organisation des interventions et une amélioration de la qualité des services.

Le système intègre plusieurs fonctionnalités importantes, telles que la gestion des utilisateurs selon leurs rôles, la gestion des tickets et des équipements, le suivi des interventions techniques, la génération de statistiques et de tableaux de bord, ainsi qu'un système de notifications en temps réel. Une interface web moderne et responsive a également été développée afin d'offrir une expérience utilisateur fluide et intuitive.

Mots clés : plateforme web, gestion des tickets, maintenance, suivi des interventions, notifications, tableau de bord, application intelligente.

Abstract

This work was carried out as part of the final-year project for obtaining the Computer Engineering degree. The project focuses on the development of **TSS**, an intelligent platform dedicated to technical service management and incident ticket handling.

The main objective of this application is to optimize the processing of technical requests by improving communication between different actors, including coordinators, technicians, and helpdesk agents. The platform allows the creation, tracking, and management of tickets while ensuring better organization of technical interventions and improving service quality.

The system integrates several important features such as user management based on roles, ticket and equipment management, monitoring of technical interventions, generation of statistics and dashboards, and a real-time notification system. A modern and responsive web interface was also developed to provide a smooth and intuitive user experience.

Keywords : web platform, ticket management, maintenance, intervention tracking, notifications, dashboard, intelligent application.

Table des matières

I Cadre générale du projet . . . 4
- I.1 Introduction . . . 4
- I.2 Présentation de l'organisme d'accueil . . . 4
  - I.2.1 Fiche d'identité de Tunisys . . . 5
  - I.2.2 l'organigramme . . . 5
  - I.2.3 Domaines d'activité de TUNISYS . . . 5
- I.3 Contexte de Projet . . . 6
  - I.3.1 Problématique . . . 6
  - I.3.2 Étude de l'existant . . . 7
  - I.3.3 Solution proposée . . . 9
- I.4 Étude des méthodologies de développement . . . 10
  - I.4.1 Choix de la méthodologie . . . 10
  - I.4.2 Méthodologie adoptée . . . 11
- I.5 Conclusion . . . 13

II Analyse et spécification des besoins . . . 14
- II.1 Introduction . . . 14
- II.2 Modélisation du contexte . . . 14
  - II.2.1 Identification des acteurs . . . 14
  - II.2.2 spécification des besions . . . 15
  - II.2.3 Diagramme de cas d'utilisation général . . . 18
- II.3 Analyse Globale . . . 21
  - II.3.1 Diagramme de classes d'analyse . . . 21
  - II.3.2 Architecture globale . . . 22
    - II.3.2.1 Architecture logique de la plateforme . . . 24
    - II.3.2.2 Architecture physique de la plateforme . . . 25
- II.4 Pilotage du projet avec SCRUM . . . 26
  - II.4.1 Backlog du produit . . . 26
  - II.4.2 Planification des Sprints . . . 29
- II.5 Environnement de travail . . . 29
  - II.5.1 Environnement matériel de développement . . . 29
  - II.5.2 Environnement logiciel de développement . . . 30
    - II.5.2.1 Langages de programmation . . . 30
    - II.5.2.2 Frameworks et bibliothèques . . . 31
    - II.5.2.3 Outils DevOps et déploiement . . . 32
- II.6 Conclusion . . . 32

III Release 0 Mise en place des fonctionnalités fondamentales 33
III.1 Introduction 33
III.2 Backlog du Sprint 1 33
III.3 Diagramme de cas d'utilisation du Sprint 1 35
III.4 Services Web 36
III.5 Les cas d'utilisation du Sprint 1 37
III.5.1 Cas d'utilisation : « Se connecter avec OTP » 37
III.5.1.1 Description textuelle 37
III.5.1.2 Diagramme de séquence système 38
III.5.1.3 Diagramme de séquence objet 39
III.6 Réalisation du Sprint 1 39
III.7 Backlog du Sprint 2 42
III.8 Diagramme de cas d'utilisation du Sprint 2 44
III.9 Services Web du Sprint 2 46
III.10 Les cas d'utilisation du Sprint 2 47
III.10.1 Cas d'utilisation : « Ajouter un ticket » 47
III.10.1.1 Description textuelle 47
III.10.1.2 Diagramme de séquence système 48
III.10.1.3 Diagramme de séquence objet 49
III.11 Réalisation du Sprint 2 50
III.12 Conclusion 52

IV Release 1:Gestion des espaces clients et pilotage des interventions . 53
IV.1 Introduction 53
IV.2 Backlog du Sprint 3 53
IV.3 Diagramme de cas d'utilisation du Sprint 3 55
IV.4 Services Web du Sprint 3 56
IV.4.1 Cas d'utilisation : « Traitement d'une demande et la convertir en ticket d'intervention » 57
IV.4.1.1 Description textuelle 58
IV.4.1.2 Diagramme de séquence système 58
IV.4.1.3 Diagramme de séquence objet 59
IV.4.2 Cas d'utilisation : « Identifier un incident via DocBot » 60
IV.4.2.1 Classification des incidents 61
IV.4.2.2 Présentation d'Ollama et de LLaMA 3 61
IV.4.2.3 Avantages dans le contexte de l'organisme : 61
IV.4.2.4 Architecture logique de DocBot 62
IV.4.2.5 Processus de traitement 62
IV.4.2.6 Description textuelle 63
IV.4.2.7 Diagramme de séquence système 64
IV.4.2.8 Diagramme de séquence objet 65
IV.4.2.9 Diagramme d'activités 66
IV.5 Réalisation du Sprint 3 66
IV.6 Conclusion 71

V Release 2 — Sprint 4 : Gestion Automatique des Rapports 72

V.1 Introduction . . . 72
V.2 Backlog du Sprint 4 . . . 73
V.3 Diagramme de cas d'utilisation du Sprint 4 . . . 74
V.4 Services Web . . . 75
V.5 Les cas d'utilisation du Sprint 4 . . . 75
V.5.1 Cas d'utilisation : « Détecter une récurrence et générer un rapport automatique » . . . 75
V.5.1.1 Description textuelle . . . 76
V.5.1.2 Diagramme de séquence système . . . 76
V.5.1.3 Diagramme de séquence objet . . . 77
V.5.2 Cas d'utilisation : « Consulter le tableau de bord des rapports et du `nbr_chance` » . . . 78
V.5.2.1 Description textuelle . . . 78
V.5.2.2 Diagramme de séquence système . . . 79
V.5.2.3 Diagramme de séquence objet . . . 80
V.6 Diagramme d'activité . . . 80
V.7 Réalisation du Sprint 4 . . . 81
V.8 Conclusion . . . 85

VI Release 3 — Sprint 5 : Gestion des Logs . . . 86
VI.1 Introduction . . . 86
VI.2 Backlog du Sprint 5 . . . 86
VI.3 Diagramme de cas d'utilisation du Sprint 5 . . . 88
VI.4 Services Web . . . 89
VI.5 Les cas d'utilisation du Sprint 5 . . . 90
VI.5.1 Cas d'utilisation : « Enregistrer les logs de connexion » . . . 90
VI.5.1.1 Description textuelle . . . 90
VI.5.1.2 Diagramme de séquence système . . . 91
VI.5.1.3 Diagramme de séquence objet . . . 92
VI.5.2 Cas d'utilisation : « Valider une modification par le manager » . . . 93
VI.5.2.1 Description textuelle . . . 93
VI.5.2.2 Diagramme de séquence système . . . 94
VI.5.2.3 Diagramme de séquence objet . . . 95
VI.5.3 Cas d'utilisation : « Valider une suppression par le manager » . . . 95
VI.5.3.1 Description textuelle . . . 96
VI.5.3.2 Diagramme de séquence système . . . 97
VI.5.3.3 Diagramme de séquence objet . . . 98
VI.6 Diagramme d'activité . . . 99
VI.7 Réalisation du Sprint 5 . . . 99
VI.8 Conclusion . . . 101

VII Déploiement de la Plateforme TSS . . . 102
VII.1 Introduction . . . 102
VII.2 Déploiement initial — Approche traditionnelle . . . 102
VII.2.1 Architecture initiale . . . 102
VII.2.2 Le serveur IIS (Internet Information Services) . . . 103

VII.2.2.1 Présentation d'IIS 103
VII.2.3 Problèmes et limites identifiés 103
VII.3 Solution proposée : Pipeline CI/CD avec Jenkins et Docker 104
VII.3.1 Choix technologiques 104
VII.3.2 Architecture cible avec Docker 105
VII.4 Mise en œuvre de la pipeline Jenkins 106
VII.4.1 Vue d'ensemble de la pipeline 106
VII.4.2 Description des étapes 107
VII.5 Comparaison avant / après 107
VII.6 Réalisation 108
VII.7 Conclusion 111

VIII Conclusion et perspectives 112

Table des figures

I.1 Logo Entreprise 4
I.2 organigramme de l'entreprise 5
I.3 Cycle de la méthodologie Scrum. 12
II.1 Diagramme de cas d'utilisation global. 21
II.2 Diagramme de classe d'analyse Global. 22
II.3 Architecture logique de la plateforme TSS 24
II.4 Architecture physique de la plateforme TSS. 25
III.1 Diagramme de cas d'utilisation du Sprint 1 35
III.2 Diagramme de séquence système du cas d'utilisation « Se connecter avec OTP » 38
III.3 Diagramme de séquence objet du cas d'utilisation « Se connecter avec OTP » 39
III.4 Interface de connexion 40
III.5 Interface de vérification OTP 40
III.6 Liste des utilisateurs 41
III.7 Ajout d'un client 41
III.8 Gestion des agences 42
III.9 Consultation des équipements 42
III.10 Diagramme de cas d'utilisation du Sprint 2 45
III.11 Diagramme de séquence système « Ajouter un ticket » 49
III.12 Diagramme de séquence objet « Ajouter un ticket » 50
III.13 Liste des tickets téléphoniques 50
III.14 Ajout d'un ticket téléphonique 51
III.15 Tickets reportés 51
III.16 Tableau de bord 52
IV.1 Diagramme de cas d'utilisation du Sprint 3 56
IV.2 Diagramme de séquence système du « Traitement d'une demande par la Coordinatrice » 59
IV.3 Diagramme de séquence objet « Traitement d'une demande par la Coordinatricen » 60
IV.4 Architecture logique du module DocBot basé sur Ollama 62
IV.5 Diagramme de séquence système - DocBot 64
IV.6 Diagramme de séquence objet - DocBot 65
IV.7 Diagramme d'activités - Identification d'un incident via DocBot 66
IV.8 Formulaire de soumission d'une demande client 67
IV.9 email d'ouverture ticket client 67
IV.10 Suivi des demandes — Espace client 68
IV.11 détails des demandes — Espace client 68

IV.12 Gestion des demandes — Coordinatrice 69
IV.13 Validation d'une demande et conversion en ticket d'intervention 69
IV.14 Gestion des demandes non traitées — Manager 70
IV.15 Gestion des demandes non traitées — Manager 70
IV.16 Gestion des demandes non traitées — Manager 70
IV.17 Gestion des demandes Modifiées — Manager 71
IV.18 Interface du chatbot IA DocBot 71
V.1 Diagramme de cas d'utilisation du Sprint 4 74
V.2 Diagramme de séquence système — Détecter une récurrence et générer un rapport 77
V.3 Diagramme de séquence objet — Détecter une récurrence et générer un rapport 78
V.4 Diagramme de séquence système — Consulter le tableau de bord des rapports 79
V.5 Diagramme de séquence objet — Consulter le tableau de bord des rapports 80
V.6 Diagramme d'activité — Sprint 4 : Détection de récurrence et alertes . . 81
V.7 Création d'un ticket curatif et détection automatique de récurrence . . . 82
V.8 Rapport de récurrence — Détail automatique 82
V.9 E-mail de notification envoyé au manager avec rapport PDF 83
V.10 Tableau de bord des rapports de récurrence — Manager 83
V.11 Suivi du `nbr_chance` des techniciens — Manager 84
V.12 Notification en temps réel 84
V.13 Notification en temps réel 85
VI.1 Diagramme de cas d'utilisation du Sprint 5 88
VI.2 Diagramme de séquence système — Enregistrer un log de connexion . . 91
VI.3 Diagramme de séquence objet — Enregistrer un log de connexion 92
VI.4 Diagramme de séquence système — Valider une modification 94
VI.5 Diagramme de séquence objet — Valider une modification 95
VI.6 Diagramme de séquence système — Valider une suppression 97
VI.7 Diagramme de séquence objet — Valider une suppression 98
VI.8 Diagramme d'activité — Sprint 5 : Logs et validation manager 99
VI.9 Interface de consultation des logs de connexion — Manager 100
VI.10 Interface de consultation des logs de modification 100
VI.11 Tableau de bord des logs de suppression — Manager 101
VI.12 E-mail de notification envoyé au manager avec liens d'action rapide . . . 101
VII.1 Architecture de déploiement initiale — Machine virtuelle Windows . . . 103
VII.2 Architecture de déploiement cible — Conteneurs Docker 105
VII.3 Vue d'ensemble de la pipeline Jenkins — Étapes du build 106
VII.4 Interface Jenkins — Pipeline exécutée avec succès 109
VII.5 Logs d'exécution détaillés d'un build Jenkins 109
VII.6 Dépôt Docker Hub — Images versionées de la plateforme TSS 110
VII.7 Docker Desktop — Les quatre conteneurs TSS en cours d'exécution . . . 110
VII.8 Plateforme TSS accessible après déploiement automatique 111

Liste des tableaux

I.1 Fiche d'identité Tunisys 5
I.2 Comparaison des solutions existantes de GMAO et ITSM 8
I.3 Étude comparative des méthodologies de développement 11
II.1 Besoins fonctionnels par acteur 17
II.2 Comparaison des architectures Monolithique, N-Tiers et Microservices . . . 23
II.3 Backlog du produit 28
II.4 Planification des sprints 29
II.5 Outils utilisés pour la modélisation, le développement et la gestion de données 30
II.6 Langages de programmation utilisés dans la plateforme TSS 31
II.7 Frameworks et bibliothèques utilisés dans la plateforme TSS 31
II.8 Outils DevOps et de déploiement de la plateforme TSS 32
III.1 Backlog du Sprint 2 35
III.2 Liste des services web du sprint 1. 37
III.3 Cas d'utilisation : Se connecter avec OTP 38
III.4 Backlog du Sprint 2 44
III.5 Liste des services web du Sprint 2. 47
III.6 Cas d'utilisation : Ajouter un ticket 48
IV.1 Backlog du Sprint 3 55
IV.2 Liste des services web du Sprint 3. 57
IV.3 Cas d'utilisation : Compléter ou rejet d'une demande client 58
IV.4 Cas d'utilisation : Identifier un incident via DocBot 63
V.1 Backlog du Sprint 4 74
V.2 Liste des services web du Sprint 4. 75
V.3 Cas d'utilisation : Détecter une récurrence et générer un rapport 76
V.4 Cas d'utilisation : Consulter le tableau de bord des rapports 79
VI.1 Backlog du Sprint 5 88
VI.2 Liste des services web du Sprint 5. 89
VI.3 Cas d'utilisation : Enregistrer les logs de connexion 90
VI.4 Cas d'utilisation : Valider une modification par le manager 93
VI.5 Cas d'utilisation : Valider une suppression par le manager 96
VII.1 Problèmes identifiés dans le déploiement initial 104
VII.2 Description des quatre conteneurs de la plateforme TSS 106
VII.3 Description des étapes de la pipeline Jenkins 107
VII.4 Comparaison du déploiement avant et après la mise en place de la pipeline CI/CD 108

Introduction générale

À l'ère de la transformation numérique, les entreprises évoluent dans un environnement où la rapidité, l'efficacité et la circulation instantanée de l'information sont devenues des éléments essentiels à leur réussite. La digitalisation des services ne représente plus uniquement un avantage concurrentiel, mais une nécessité permettant d'améliorer l'organisation interne, d'optimiser la gestion des ressources et d'assurer une meilleure qualité de service aux clients.

Dans ce contexte, les applications web occupent aujourd'hui une place centrale dans le fonctionnement des entreprises modernes. Elles facilitent l'automatisation des tâches, réduisent les erreurs humaines et offrent un accès rapide et structuré aux informations. Grâce à ces solutions numériques, les entreprises peuvent assurer une meilleure communication entre les différents acteurs, garantir une plus grande traçabilité des opérations et améliorer considérablement le suivi des activités quotidiennes.

Par ailleurs, dans le domaine de la maintenance et du support technique, la gestion des incidents et des interventions représente un enjeu majeur. Une mauvaise organisation des tickets ou un manque de suivi peut engendrer des pertes de temps, des difficultés de communication et une diminution de la satisfaction des clients. Il devient alors indispensable de mettre en place des systèmes capables de centraliser les informations, de suivre l'historique des interventions et d'assurer une coordination efficace entre les équipes techniques et les clients.

C'est dans cette perspective que s'inscrit notre Projet de Fin d'Études, réalisé au sein de **TUNISYS**. Ce projet consiste à concevoir et à développer une application web dédiée à la gestion des tickets de maintenance, au suivi des interventions techniques ainsi qu'à la génération de rapports et de notifications automatiques. L'objectif principal est d'améliorer la traçabilité des opérations, de faciliter le traitement des incidents et de renforcer la qualité du suivi entre l'entreprise et ses clients.

La réalisation de ce projet nous a permis de mettre en pratique les connaissances acquises durant notre formation académique tout en découvrant les réalités du monde professionnel. Elle nous a également offert l'opportunité de travailler sur des problématiques concrètes liées au développement web, à la gestion des données et à l'organisation des processus métiers.

Ce présent rapport est structuré comme suit :

- Dans le premier chapitre, intitulé « Cadre général du projet », nous présentons l'organisme d'accueil, le contexte du projet ainsi que la méthodologie de travail adoptée.
- Dans le deuxième chapitre, intitulé « Analyse des besoins », nous décrivons les besoins fonctionnels et non fonctionnels ainsi que les différents cas d'utilisation du système.
- Dans le troisième chapitre, intitulé « Release 0 », nous présentons les fonctionnalités développées dans la première version de notre solution, notamment le Sprint 1 consacré à la gestion des utilisateurs et des clients, ainsi que le Sprint 2 dédié à la gestion des tickets et des équipements.
- Dans le quatrième chapitre, intitulé « Release 1 », nous décrivons les fonctionnalités relatives à la gestion des demandes clients.
- Dans le cinquième chapitre, intitulé « Release 2 », nous détaillons les fonctionnalités liées à la gestion des logs, des rapports et des alertes.
- Dans le sixième chapitre, intitulé « Release 3 », nous détaillons les fonctionnalités liées à la gestion des logs.
- Dans le dernier chapitre, intitulé « Déploiement de la plateforme TSS », nous détaillons le pipeline CI/CD intégré au processus de déploiement de la plateforme.

Finalement, nous clôturons ce rapport par une conclusion générale qui résume le travail effectué et présente quelques perspectives pouvant être perçues comme des améliorations de notre contribution.

Chapitre I

Cadre générale du projet

I.1 Introduction

Dans ce chapitre, nous commençons par la présentation du cadre de stage ainsi que l'organisme d'accueil. Puis, nous mettons le projet dans son contexte général. en identifiant les problématiques et en fixant ses objectifs. Finalement, nous présentons la méthodologie de travail adoptée dans ce projet.

I.2 Présentation de l'organisme d'accueil

Fondée en 1981, **Tunisys** [1] est parmi les pionniers du marché de l'IT. Créative et rigoureuse, passionnée par le monde des technologies et des affaires, l'entreprise transforme les ressources informatiques en véritables alliés stratégiques pour les entreprises.
Intégrateur de solutions IT depuis 1981, Tunisys se positionne l'implantation de solutions personnalisées, performantes et efficaces. Que ce soit pour une petite entreprise ou une grande organisation, et quel que soit le secteur d'activité, l'expertise de Tunisys est mise à disposition pour aider à atteindre les objectifs.
L'équipe d'experts se consacre à l'élaboration de solutions sur mesure, adaptées aux besoins spécifiques. En travaillant en étroite collaboration avec les clients, l'entreprise comprend les défis uniques et développe des stratégies qui optimisent les opérations, augmentent la productivité et renforcent la compétitivité sur le marché. La figure I.1 présente le logo de Tunisys.



Fig. I.1 : Logo Entreprise

I.2.1 Fiche d'identité de Tunisys

Le tableau 1.1 présente la fiche d'identité de Tunisys.

Nom de la société	**Tunisys**
Adresse	124, Avenue de la Liberté 1002- Belvédère Tunis
Téléphone	+216 +216 71 791 699
Adresse-Mail	tunisys.commercial@tunisys.com.tn
Site Web	https ://www.tunisys.com.tn/
Logo	TUNISYS تونسيس

TAB. I.1 : Fiche d'identité Tunisys
[1]

I.2.2 l'organigramme

Dans cette partie, nous allons présenter l'organigramme de l'entreprise et décrire le rôle de chaque direction. Veuillez noter que cet organigramme est à jour au 17 mars 2023.

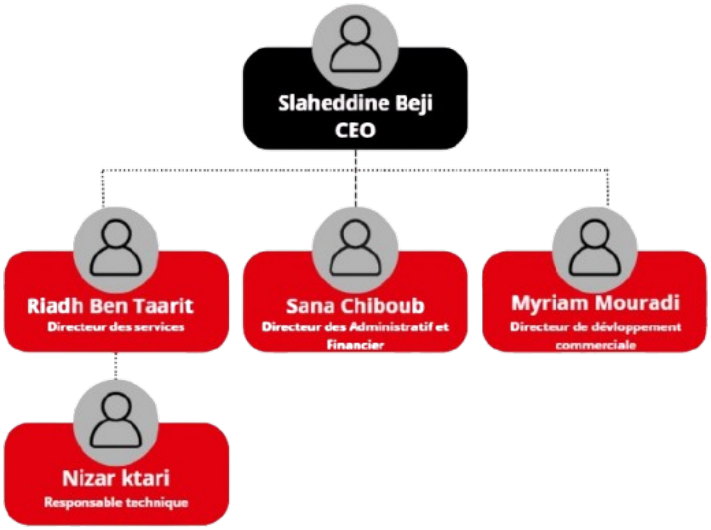


FIG. I.2 : organigramme de l'entreprise

I.2.3 Domaines d'activité de TUNISYS

TUNISYS est une entreprise spécialisée dans les solutions technologiques et les services informatiques destinés principalement aux secteurs bancaire, financier et administratif.

Ses domaines d'activité couvrent plusieurs axes stratégiques :

- **Maintenance des équipements bancaires :**
  - Maintenance corrective et préventive des distributeurs automatiques de billets (DAB).
  - Gestion des interventions techniques sur les équipements électroniques et informatiques.
  - Assistance technique et support sur site pour les clients.
- **Services et solutions informatiques :**
  - Développement d'applications web et de plateformes de gestion internes.
  - Intégration de solutions logicielles adaptées aux besoins des entreprises.
  - Gestion des infrastructures informatiques et réseaux.
- **Gestion du service après-vente (SAV) :**
  - Suivi et traitement des tickets d'incidents techniques.
  - Coordination entre les techniciens, les coordinateurs et les clients.
  - Optimisation des processus de maintenance et d'intervention.
- **Transformation digitale :**
  - Digitalisation des processus métiers.
  - Mise en place de solutions innovantes pour améliorer la productivité et la qualité des services.
  - Automatisation des tâches de gestion et de suivi.
- **Accompagnement et support client :**
  - Assistance technique continue pour les entreprises partenaires.
  - Formation et accompagnement des utilisateurs sur les solutions déployées.
  - Garantie d'un service rapide et efficace afin d'assurer la satisfaction client.

I.3 Contexte de Projet

Dans cette section, nous décrivons le problème puis examinons La solution proposée, et enfin nous préciserons la méthode de travail adoptée.

I.3.1 Problématique

Dans les entreprises spécialisées dans la maintenance technique et le support informatique, la gestion des tickets constitue un élément central pour assurer la continuité des services et la satisfaction des clients. Chaque incident nécessite un suivi rigoureux ainsi qu'une bonne coordination entre les différents intervenants afin de garantir un traitement rapide et efficace.

Cependant, de nombreuses organisations continuent d'utiliser des méthodes traditionnelles telles que les appels téléphoniques, les e-mails ou les fichiers Excel. Ces

approches rendent le processus de gestion peu centralisé, difficile à suivre et souvent sujet à des retards ou à un manque de visibilité.

Cette absence de système unifié entraîne des difficultés dans le suivi des tickets, la communication entre les équipes et le contrôle global des interventions. Elle limite également la capacité à disposer d'indicateurs fiables pour évaluer et améliorer la performance du service.

Ainsi, ces insuffisances montrent la nécessité de mettre en place une solution moderne, centralisée et automatisée permettant d'optimiser la gestion des tickets, d'améliorer la communication et d'assurer un suivi clair et en temps réel des interventions.

La problématique peut donc être résumée comme suit :

Comment concevoir une plateforme centralisée et automatisée permettant d'optimiser la gestion des tickets techniques et d'assurer un suivi efficace, fluide et en temps réel des interventions ?

I.3.2 Étude de l'existant

Dans le domaine de la maintenance et du support technique, plusieurs solutions de Gestion de la Maintenance Assistée par Ordinateur (GMAO)[1] et Information Technology Service Management (ITSM)[2] permettent de gérer les tickets, les interventions et les équipements. Ces outils visent à améliorer la qualité du service, la traçabilité des opérations ainsi que la productivité des équipes.

GLPI

GLPI[2] est une solution open source dédiée à la gestion des services informatiques(ITSM). Elle permet la gestion des tickets, le suivi des incidents, l'inventaire des équipements ainsi que la gestion des utilisateurs.

IBM Maximo

IBM Maximo[3] est une solution de GMAO destinée principalement aux grandes entreprises. Elle offre des fonctionnalités avancées telles que la gestion des actifs, la planification des interventions, la maintenance préventive ainsi que le suivi des performances des équipements.

[1]GMAO : est un ensemble de méthodes et de logiciels permettant d'organiser, de planifier, de suivre et d'optimiser les opérations de maintenance au sein d'une organisation.

[2]ITSM : désigne l'ensemble des processus et des outils permettant de gérer les services informatiques d'une organisation.

ServiceNow

ServiceNow[4] est une plateforme cloud ITSM permettant la gestion des incidents, des demandes et des workflows automatisés. Elle est largement utilisée pour la gestion des services IT avec un suivi en temps réel et une forte automatisation des processus.

Comparaison des solutions existantes

Critères	**GLPI**	**IBM Maximo**	**ServiceNow**
Gestion des tickets	Oui	Oui	Oui
Gestion des équipements	Oui	Oui	Oui
Suivi des interventions	Oui	Oui	Oui
Affectation des techniciens	Oui	Oui	Oui
Tableaux de bord et reporting	Partiel	Avancé	Très avancé
Suivi en temps réel	Limité	Oui	Oui
Automatisation des workflows	Faible	Moyen	Très avancé
Gestion des SLA	Non	Oui	Oui
Gestion des zones d'intervention	Non	Partiel	Oui
Notifications automatiques	Limité	Oui	Oui
Personnalisation métier	Moyenne	Faible	Très élevée
Modèle économique	Gratuit (Open Source)	Payant	Payant (SaaS)

Tab. I.2 : Comparaison des solutions existantes de GMAO et ITSM

Conclusion de l'existant

L'étude des solutions existantes montre que les outils de GMAO et ITSM offrent des fonctionnalités avancées pour la gestion des tickets et des interventions. Cependant, ces solutions présentent certaines limites en termes de flexibilité, de personnalisation et d'adaptation aux besoins spécifiques des entreprises. Cela justifie le développement d'une solution sur mesure répondant précisément aux exigences du projet.

I.3.3 Solution proposée

Afin de répondre aux problématiques identifiées dans la partie précédente, notre projet s'inscrit dans le cadre de l'**évolution et de l'amélioration de la solution TSS (Tunisys Service Solution) déjà existante**, dédiée à la gestion des tickets techniques et au suivi des interventions.

L'objectif de cette évolution est d'enrichir la solution existante par l'intégration de **nouvelles fonctionnalités destinées aux clients**, afin de faciliter et d'améliorer la relation entre TUNISYS et ses clients. Un espace client dédié ainsi qu'un **tableau de bord client** sont ainsi intégrés à la plateforme.

Grâce à cet espace, le client peut désormais **soumettre directement ses demandes d'intervention et créer des tickets depuis la plateforme**, sans avoir recours aux moyens de communication traditionnels tels que les appels téléphoniques ou les échanges par e-mail. Il peut également consulter l'état de ses demandes, suivre leur évolution, accéder à l'historique des tickets et disposer d'une meilleure visibilité sur les interventions réalisées.

Le **tableau de bord client** permet de centraliser ces informations et de présenter une vue synthétique de l'activité liée aux demandes du client. Cette évolution contribue ainsi à améliorer la **transparence, la communication et la réactivité** dans le traitement des demandes, tout en facilitant le suivi des interventions.

Enfin, la solution conserve son approche hybride combinant les concepts de la **GMAO** et de l'**ITSM**. Elle assure ainsi la gestion des équipements et des interventions techniques, tout en intégrant la gestion des tickets, des workflows et du support utilisateur.

Principales fonctionnalités de la solution

- **Gestion des tickets :** création, modification, affectation et suivi des tickets en temps réel, avec un historique complet des actions réalisées.
- **Suivi client :** accès à un espace personnel permettant de consulter l'état d'avancement des tickets, les réponses associées et de soumettre directement de nouvelles demandes d'intervention.
- **Espace manager :** supervision complète de la plateforme, incluant la validation des demandes de création, de modification et de suppression des ressources, le suivi des journaux de connexion (logs), ainsi que la consultation des statistiques et des indicateurs de performance.
- **Gestion par la coordinatrice :** centralisation des demandes, affectation des techniciens, suivi du traitement des tickets et coordination des interventions.
- **Système de notifications :** envoi d'alertes en temps réel pour informer les utilisateurs des mises à jour importantes et des événements nécessitant une action.
- **Tableaux de bord :** visualisation des indicateurs de performance, des statistiques d'activité et de l'évolution des tickets afin de faciliter la prise de décision.

Apports de la solution

La solution TSS permet de :

- Centraliser l'ensemble des informations liées aux tickets sur une seule plateforme ;
- Améliorer la communication entre les différents acteurs
- Réduire les erreurs et pertes d'information
- Accélérer le traitement des incidents
- Assurer une meilleure transparence et traçabilité des opérations
- Combiner les avantages des approches GMAO et ITSM dans une solution unifiée.

Ainsi, la solution proposée constitue une réponse efficace aux limites des systèmes existants et permet d'optimiser la gestion globale des interventions techniques au sein de l'entreprise.

I.4 Étude des méthodologies de développement

Le choix d'une méthodologie de développement constitue une étape essentielle dans la réussite d'un projet logiciel. Une méthode adaptée permet d'améliorer l'organisation du travail, de faciliter la collaboration entre les différents acteurs et d'assurer une réalisation efficace des fonctionnalités attendues.

I.4.1 Choix de la méthodologie

Il existe plusieurs méthodologies de développement logiciel, chacune présentant des avantages et des limites. Afin de sélectionner la plus adaptée à notre projet, nous avons réalisé une étude comparative des principales méthodes, présentée dans le tableau I.3.

Méthode	Forces	Faiblesses
RUP	Bonne traçabilité. Architecture bien structurée.	Coût de mise en œuvre élevé. Complexité lors des premières phases.
XP	Développement progressif. Tests continus. Coût relativement réduit.	Forte implication de l'équipe. Difficile à appliquer sur de grands projets.
2TUP	Adaptée à des projets de différentes tailles. Prise en compte des risques. Développement itératif.	Principalement centrée sur le développement. Peu de modèles documentaires.
Scrum	Forte implication du client. Travail collaboratif. Adaptation rapide aux changements. Livraison incrémentale.	Gestion plus complexe avec de grandes équipes. Dépend fortement de l'implication des membres.

Tab. I.3 : Étude comparative des méthodologies de développement

I.4.2 Méthodologie adoptée

À l'issue de cette étude comparative, nous avons retenu la méthodologie agile **Scrum**, qui répond le mieux aux besoins de notre projet.

Scrum repose sur un développement itératif et incrémental organisé en cycles courts appelés *sprints*. Cette approche favorise la collaboration entre les membres de l'équipe et les parties prenantes tout en permettant de s'adapter rapidement aux évolutions des besoins.

Elle offre également la possibilité de livrer régulièrement des versions fonctionnelles de l'application afin de recueillir les retours des utilisateurs et d'améliorer progressivement le produit.

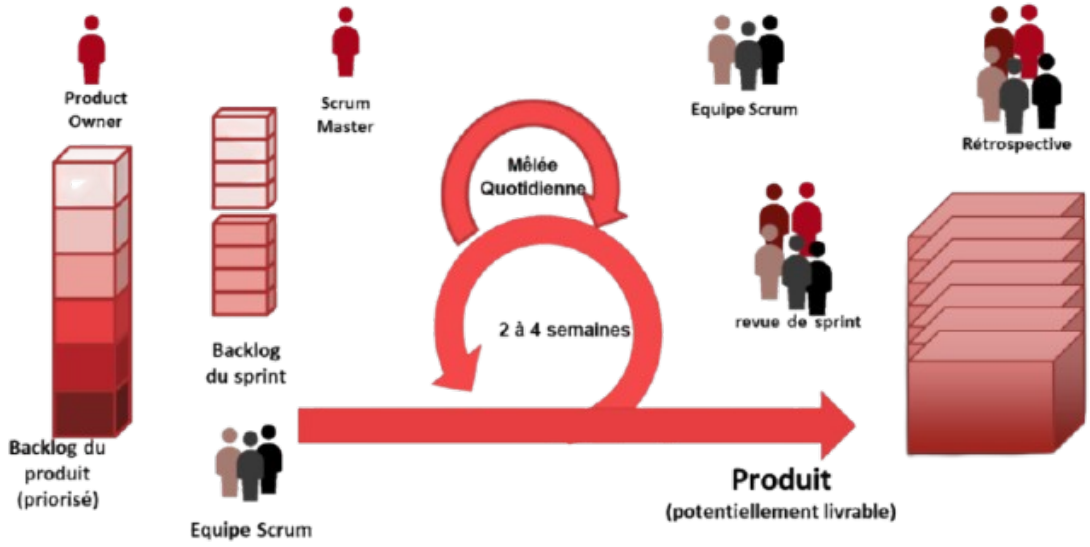


FIGURE I.3 : Cycle de la méthodologie Scrum.

Organisation de l'équipe Scrum

La méthodologie Scrum s'appuie sur trois rôles principaux :

Product Owner Le Product Owner représente les besoins des utilisateurs et définit la vision du produit. Il est chargé de gérer et de prioriser le backlog afin de maximiser la valeur du projet.

Ses principales responsabilités sont :

- Définir et prioriser les fonctionnalités ;
- Planifier les différentes releases ;
- Ajuster les priorités en fonction des besoins ;
- Valider les fonctionnalités développées.

Scrum Master Le Scrum Master veille au respect des principes Scrum et facilite la collaboration entre les membres de l'équipe. Il accompagne également l'équipe dans la résolution des obstacles pouvant ralentir le développement.

Équipe de développement L'équipe de développement est responsable de la conception, de l'implémentation et des tests des différentes fonctionnalités du système afin d'atteindre les objectifs fixés pour chaque sprint.

Répartition des rôles

Dans le cadre de notre projet, les rôles Scrum sont répartis comme suit :

- **Product Owner :** Nizar Ktari
- **Scrum Master :** Nizar Ktari , Riadh ben Taarit
- **Équipe de développement :** Syrine Trabelsi, Donia Gharbi et Sabrine Benomar.

I.5 Conclusion

Dans ce chapitre, nous avons présenté l'organisme d'accueil ainsi que le contexte général du projet. Nous avons ensuite exposé la problématique, étudié les solutions existantes et comparé plusieurs méthodologies de développement afin de justifier le choix de Scrum. Le chapitre suivant sera consacré à la phase préparatoire du projet, communément appelée *Sprint 0*.

Chapitre II

Analyse et spécification des besoins

II.1 Introduction

Dans le premier chapitre, nous avons choisi d'adopter la méthodologie Scrum pour concevoir notre futur système. Dans ce chapitre, nous présentons les phases de planification et d'architecture. Nous déterminons les principales fonctionnalités a fin de générer le backlog produit et la première planification des sprints, et définissons les outils utilisés pour mettre en œuvre le projet.

II.2 Modélisation du contexte

Dans le cycle de vie du développement logiciel, la spécification des besoins fonctionnels et non fonctionnels représente la première étape de ce cycle. Elle permet de spécifier les attentes du plateforme à concevoir, tout en tenant en compte la compréhension claire des besoins de l'organisation.

II.2.1 Identification des acteurs

La plateforme TSS met en interaction plusieurs acteurs qui participent à la gestion et au suivi des demandes d'assistance technique. Chaque acteur dispose d'un rôle spécifique afin d'assurer le bon fonctionnement du système.

- **Client :** Utilisateur de la plateforme disposant d'un accès en consultation. Il peut suivre l'état d'avancement des tickets qui concernent son organisme, consulter les informations associées ainsi qu'accéder à son tableau de bord.
- **Super Client :** Représentant principal de l'organisme client. Il est habilité à soumettre des demandes d'intervention, à consulter l'ensemble des tickets de son organisme et à gérer les comptes des utilisateurs rattachés à celui-ci.
- **Coordinatrice :** Responsable de la gestion opérationnelle des tickets, elle assure leur création, leur affectation aux techniciens, le suivi de leur traitement ainsi que la coordination des interventions.

- **Technicien :** Acteur chargé de l'exécution des interventions techniques. Il prend en charge les tickets qui lui sont attribués, réalise les actions nécessaires à leur résolution et met à jour leur état tout au long du cycle de traitement.
- **Manager :** Responsable du pilotage et du suivi des activités de la plateforme. Il consulte les tableaux de bord, analyse les indicateurs de performance et supervise le bon déroulement des opérations afin d'assurer une prise de décision efficace.

II.2.2 spécification des besions

Notre projet a pour but de faire l'analyse, la conception et la réalisation d'une plateforme pour les investisseurs immobilier. La solution que nous développons doit répondre aux exigences et aux besoins que nous avons déterminés tout au long de la phase de recherche.

Besoins fonctionnels

Les exigences fonctionnelles représentent l'ensemble des tâches et des fonctionnalités que le système doit assurer afin de répondre aux besoins des utilisateurs. Elles seront présentées, dans le tableau suivant, en fonction de chaque acteur de la plateforme. II.1

Acteurs	**Fonctionnalités**
Client	- Gérer son profil - Consulter son tableau de bord - Suivre l'état d'avancement des tickets - Consulter les détails des tickets le concernant - Consulter l'historique des tickets - Recevoir des notifications relatives aux tickets
Super Client	- Dispose de toutes les fonctionnalités du Client - Créer des demandes d'intervention - Gérer les utilisateurs de son organisation - Consulter la documentation des services proposés

Suite du tableau à la page suivante

Acteurs	Fonctionnalités
Technicien	- S'authentifier à l'application mobile - Consulter les tickets qui lui sont affectés - Accepter ou refuser la prise en charge d'un ticket - Mettre à jour le statut des tickets - Ajouter des commentaires et comptes d'intervention - Consulter l'historique de ses interventions - Clôturer les tickets résolus - Recevoir des notifications concernant les tickets affectés
Coordinatrice	- S'authentifier à la plateforme - Créer et gérer les tickets - Affecter les tickets aux techniciens - Réaffecter les tickets si nécessaire - Gérer la disponibilité des techniciens - Suivre l'état d'avancement des tickets - Completer demandes cleints - Gérer les clients, agences et équipements - Consulter les statistiques opérationnelles - Exporter les informations au format Excel - Analyser des documents via OCR et IA (Gemini) - Recevoir des notifications

Suite du tableau à la page suivante

Acteurs	Fonctionnalités
Manager	- Consulter l'ensemble des tickets du système - Consulter les tableaux de bord décisionnels - Suivre les indicateurs de performance (KPI) - Analyser les statistiques d'activité - Évaluer les performances des techniciens - Consulter les rapports d'activité et de traçabilité - Consulter les journaux de connexion (logs) - Consulter les journaux des tickets - Consulter les journaux des réclamations - Superviser les opérations de maintenance - Générer des rapports de suivi

Tab. II.1 : Besoins fonctionnels par acteur

Besoins non fonctionnels

Les besoins non fonctionnels décrivent les contraintes techniques, les critères de qualité ainsi que les exigences de performance auxquelles la plateforme doit répondre afin de garantir son bon fonctionnement.

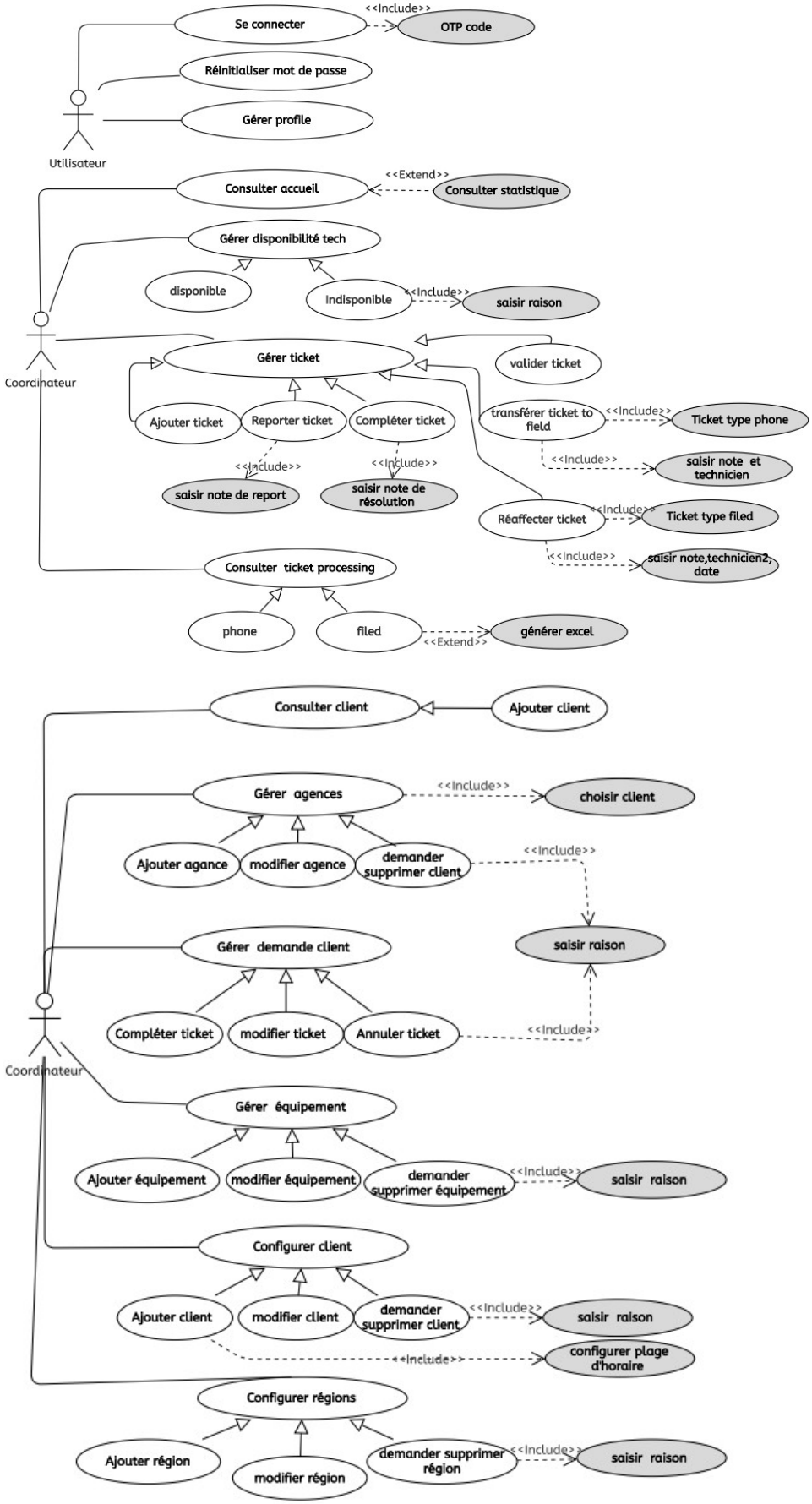
- **Sécurité :**
  - Authentifier les utilisateurs à l'aide d'un identifiant et d'un mot de passe.
  - Renforcer la sécurité des connexions par une double authentification basée sur un code OTP envoyé par mail .
  - Sécuriser les échanges entre le frontend et le backend grâce à des jetons JWT.
  - Contrôler les droits d'accès aux ressources en fonction du rôle de chaque utilisateur.
- **Performance :**
  - Assurer une communication en temps réel entre les utilisateurs grâce à Socket.IO[1].
  - Optimiser les temps de réponse par la mise en cache des données les plus consultées.
  - Garantir une exécution fluide des principales fonctionnalités de la plateforme.

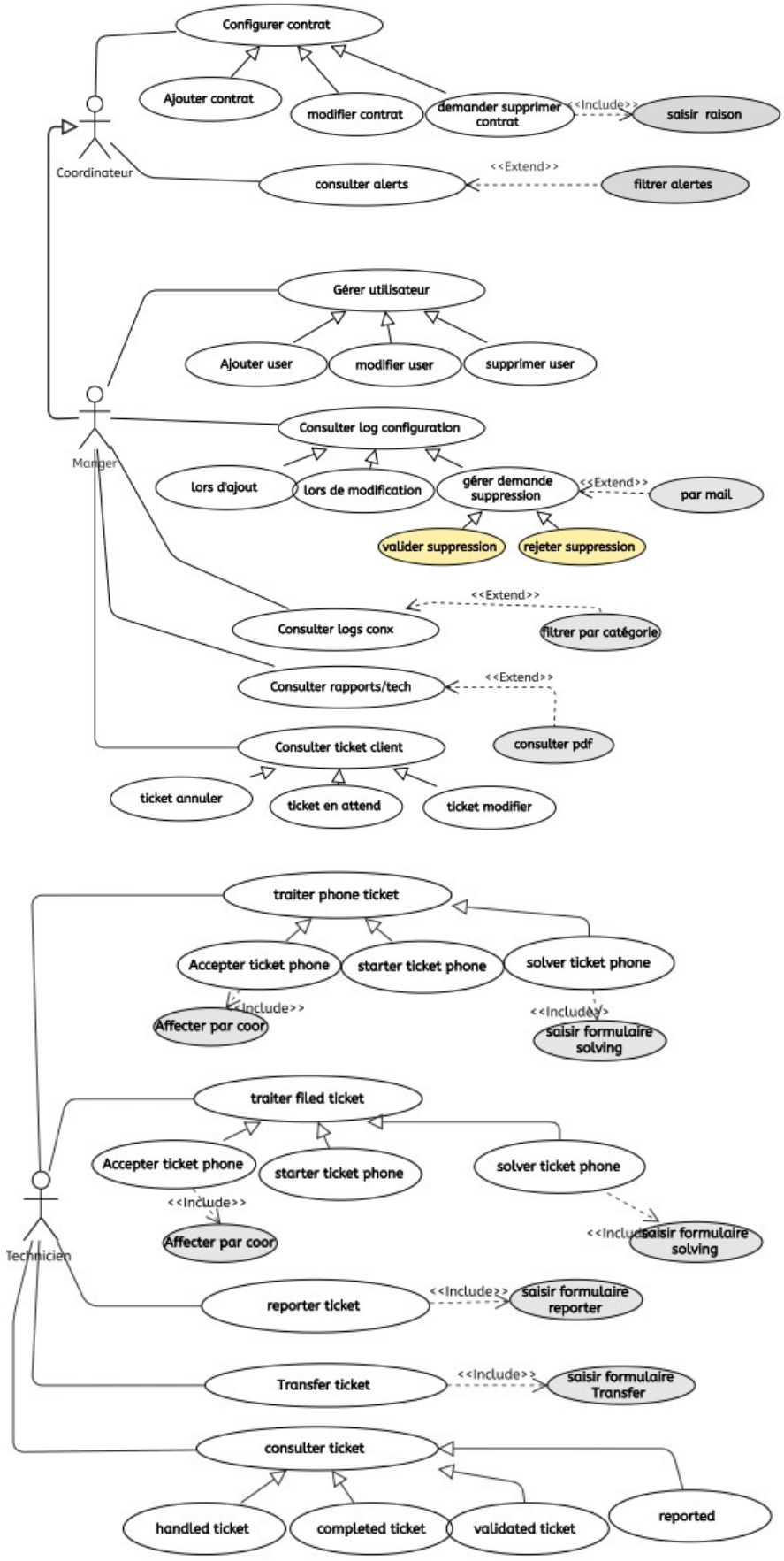
[1]Socket.IO est une bibliothèque JavaScript qui permet des échanges de données en temps réel entre le client et le serveur grâce au protocole WebSocket.

- **Disponibilité et Maintenabilité :**
  - Garantir le redémarrage automatique des services en cas de défaillance.
  - Structurer le code source selon une architecture MVC favorisant la modularité.
  - Conteneuriser l'application avec Docker afin de faciliter son déploiement et sa maintenance.
- **Ergonomie :**
  - Proposer une interface web responsive et intuitive.
  - Assurer une cohérence visuelle entre les différentes pages et composants de l'application.
  - Faciliter la navigation et l'utilisation des fonctionnalités offertes par la plateforme.
- **Interopérabilité :**
  - Permettre la communication entre le frontend React et le backend Node.js à travers une API REST.
  - Intégrer un microservice OCR développé avec Python Flask et Gemini AI via des échanges HTTP.
  - Faciliter l'interconnexion avec des services externes si nécessaire.
- **Scalabilité :**
  - Supporter l'augmentation du nombre d'utilisateurs, de tickets et de données grâce à une architecture en trois tiers séparant les couches présentation, métier et données.
  - Permettre l'évolution indépendante des différents services grâce à une architecture modulaire et des composants indépendants.
  - Assurer une gestion flexible et évolutive des données grâce à MongoDB, qui permet une adaptation facile du modèle de données et le stockage de volumes importants d'informations.

II.2.3 Diagramme de cas d'utilisation général

Dans cette section, nous décrivons les exigences du système de manière formelle en utilisant le diagramme de cas d'utilisation global. La figure II.1 suivante montre toutes les fonctions fournies par l'application. Tous les cas d'utilisation nécessitent une authentification comme condition préalable.


Se connecter
<<Include>>
OTP code
Réinitialiser mot de passe
Gérer profile
Utilisateur
Consulter accueil
<<Extend>>
Consulter statistique
Gérer disponibilité tech
disponible
Indisponible
<<Include>>
saisir raison
Coordinateur
Gérer ticket
valider ticket
Ajouter ticket
Reporter ticket
Compléter ticket
transférer ticket to field
<<Include>>
Ticket type phone
<<Include>>
saisir note et technicien
<<Include>>
saisir note de report
<<Include>>
saisir note de résolution
Réaffecter ticket
<<Include>>
Ticket type filed
<<Include>>
saisir note,technicien2, date
Consulter ticket processing
phone
filed
<<Extend>>
générer excel
Consulter client
Ajouter client
Gérer agences
<<Include>>
choisir client
Ajouter agance
modifier agence
demander supprimer client
<<Include>>
saisir raison
Gérer demande client
Compléter ticket
modifier ticket
Annuler ticket
<<Include>>
Coordinateur
Gérer équipement
Ajouter équipement
modifier équipement
demander supprimer équipement
<<Include>>
saisir raison
Configurer client
Ajouter client
modifier client
demander supprimer client
<<Include>>
saisir raison
<<Include>>
configurer plage d'horaire
Configurer régions
Ajouter région
modifier région
demander supprimer région
<<Include>>
saisir raison


Configurer contrat
Ajouter contrat
modifier contrat
demander supprimer contrat
<<Include>>
saisir raison
Coordinateur
consulter alerts
<<Extend>>
filtrer alertes
Gérer utilisateur
Ajouter user
modifier user
supprimer user
Manager
Consulter log configuration
lors d'ajout
lors de modification
gérer demande suppression
<<Extend>>
par mail
valider suppression
rejeter suppression
<<Extend>>
Consulter logs conx
filtrer par catégorie
Consulter rapports/tech
<<Extend>>
Consulter ticket client
consulter pdf
ticket annuler
ticket en attend
ticket modifier
traiter phone ticket
Accepter ticket phone
starter ticket phone
solver ticket phone
<<Include>>
Affecter par coor
<<Include>>
saisir formulaire solving
traiter filed ticket
Accepter ticket phone
starter ticket phone
solver ticket phone
<<Include>>
Affecter par coor
Technicien
saisir formulaire solving
reporter ticket
<<Include>>
saisir formulaire reporter
Transfer ticket
<<Include>>
saisir formulaire Transfer
consulter ticket
handled ticket
completed ticket
validated ticket
reported

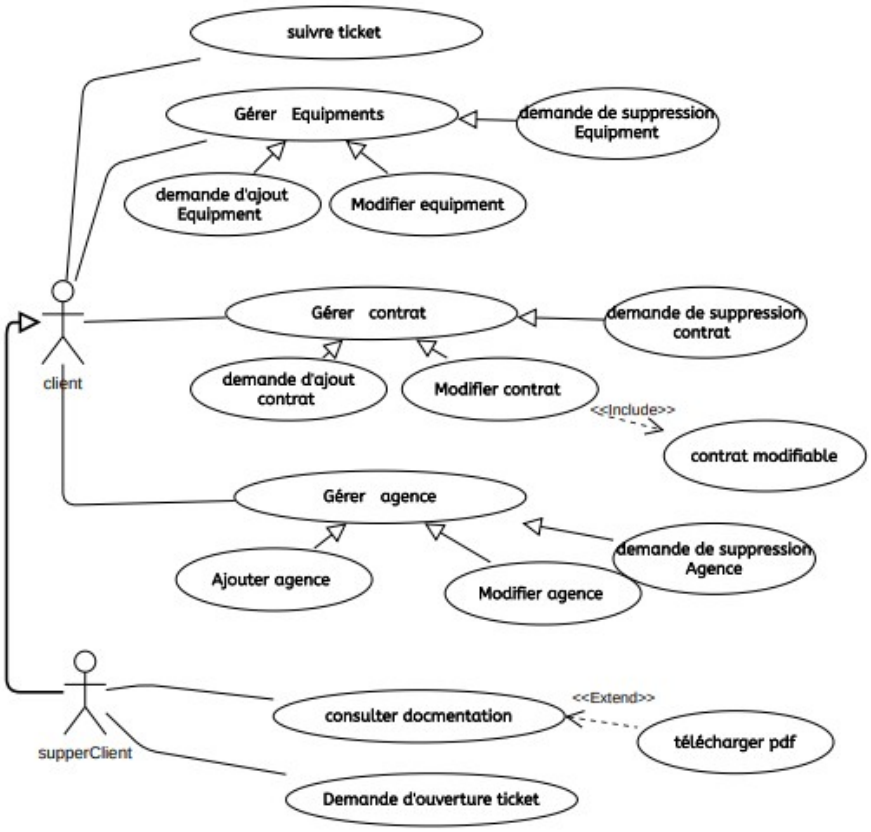


FIGURE II.1 : Diagramme de cas d'utilisation global.

II.3 Analyse Globale

Nous présentons, dans ce qui suit, une analyse globale comportant le diagramme de classes ainsi que l'architecture de la solution.

II.3.1 Diagramme de classes d'analyse

Bien que l'application repose sur une base de données **NoSQL** [5], dont le modèle de stockage est fondé sur des **collections** et des **documents** plutôt que sur des tables relationnelles, il demeure pertinent d'élaborer un diagramme de classes d'analyse afin de **représenter** les principales entités du système. Cette modélisation conceptuelle ne vise pas à reproduire fidèlement la structure physique de la base de données, mais **à offrir une vision abstraite des différents objets** métier manipulés par l'application. Elle permet ainsi d'identifier les principales collections, leurs attributs essentiels ainsi que les relations fonctionnelles qui les unissent, facilitant ainsi la compréhension de l'architecture des données et des interactions entre les différents composants du système.

II.2 suivant.

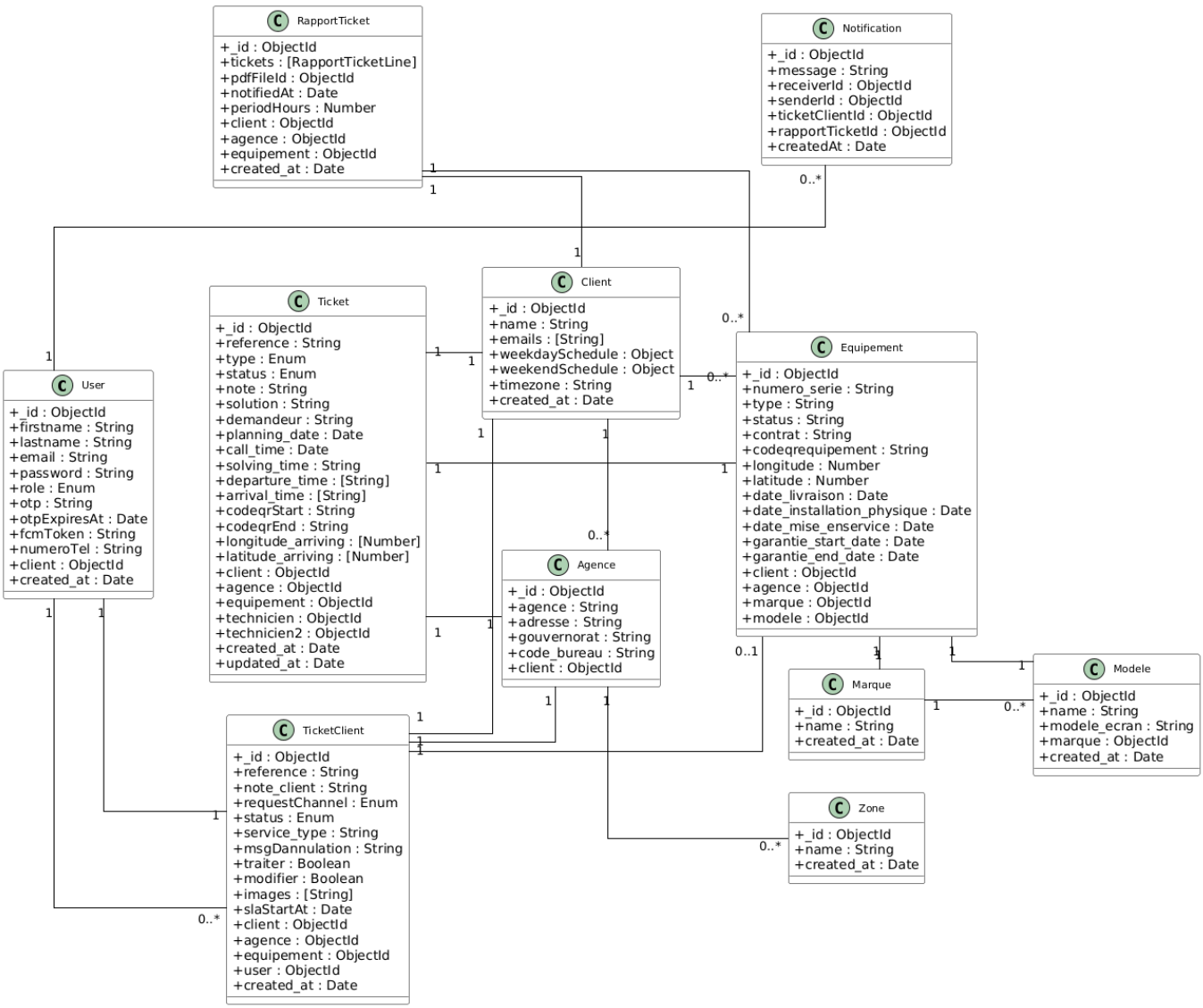


FIGURE II.2 : Diagramme de classe d'analyse Global.

II.3.2 Architecture globale

La définition de l'architecture logicielle constitue une étape essentielle dans la conception de notre plateforme TSS. En effet, le choix de l'architecture influence directement les performances, la maintenabilité, l'évolutivité ainsi que la capacité du système à satisfaire les exigences fonctionnelles et non fonctionnelles identifiées lors de l'analyse des besoins.

Dans cette perspective, une étude comparative a été menée entre trois architectures largement adoptées dans le développement des applications modernes : l'architecture monolithique, l'architecture client–serveur à trois niveaux (N-Tiers) et l'architecture orientée microservices. Cette comparaison vise à mettre en évidence les caractéristiques, les avantages et les limites de chacune de ces approches afin de retenir celle qui répond le mieux aux exigences de notre plateforme.

Architecture	Caractéristiques
Monolithique	• L'application est développée et déployée sous la forme d'un unique bloc logiciel. • Les différents modules sont fortement couplés. • Le déploiement est relativement simple. • Toute évolution ou correction nécessite le redéploiement complet de l'application. • La maintenance et l'évolutivité deviennent complexes lorsque l'application grandit.
N-Tiers	• Séparation claire des responsabilités entre les couches de présentation, métier et données. • Déploiement indépendant du frontend et du backend. • Base de données centralisée partagée par les différentes couches. • Facilite les opérations de maintenance, de test et d'évolution. • Offre un bon compromis entre modularité, simplicité de développement et performances.
Microservices	• Décomposition de l'application en services indépendants et faiblement couplés. • Chaque service dispose généralement de sa propre base de données. • Permet un déploiement, une mise à l'échelle et une évolution indépendants de chaque service. • Les échanges entre services s'effectuent via des API REST ou des mécanismes de communication asynchrones. • Offre une grande flexibilité mais implique une orchestration et une supervision plus complexes.

Tab. II.2 : Comparaison des architectures Monolithique, N-Tiers et Microservices

Au terme de cette étude comparative, l'architecture **client–serveur à trois niveaux (N-Tiers)** a été retenue pour la conception de la plateforme TSS. Ce choix s'explique par sa capacité à assurer une séparation claire des responsabilités entre les différentes couches de l'application, tout en garantissant une bonne maintenabilité, une évolutivité satisfaisante et une mise en œuvre relativement simple. Contrairement à l'architecture monolithique, elle favorise une meilleure modularité, tandis qu'elle évite la complexité de déploiement et d'orchestration inhérente à une architecture basée sur les microservices. Elle constitue ainsi une solution adaptée aux besoins techniques et fonctionnels du projet.

II.3.2.1 Architecture logique de la plateforme

L'architecture logicielle joue un rôle essentiel dans le développement d'un système fiable, efficace et capable d'évoluer en fonction des besoins. Dans le cadre de ce projet, nous avons opté pour une architecture logique organisée en plusieurs couches afin de garantir une répartition claire des responsabilités entre les différents composants de l'application. Cette organisation distingue notamment la couche de présentation, la logique métier, la gestion de la sécurité ainsi que la couche dédiée à la persistance des données.

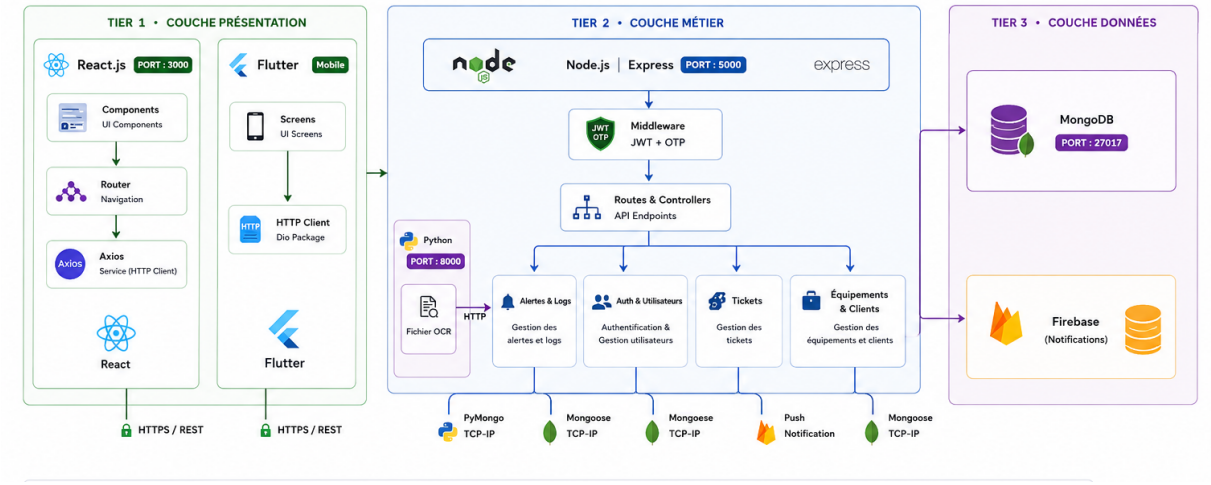


Figure II.3 : Architecture logique de la plateforme TSS

- **La couche présentation (Front-End) :** Cette couche gère l'ensemble des interactions avec les utilisateurs. Elle se décline en deux interfaces complémentaires :
  - **React.js (application web) :** Interface destinée à la coordinatrice, au manager et au clients . Elle est structurée autour d'un routeur, de composants réutilisables et d'un service Axios assurant la communication avec l'API via des requêtes HTTP.
  - **Flutter (application mobile) :** Interface mobile dédiée aux techniciens de terrain, permettant l'acceptation des tickets, le démarrage et la résolution des interventions, ainsi que le scan de QR code.
- **La couche métier (Back-End) :** Cette couche assure le traitement des données, la logique métier et l'exposition des services via une API REST. Elle est composée de deux modules indépendants :
  - **API principale (Node.js / Express) :** Gère l'authentification, la gestion des utilisateurs, des clients, des équipements et des tickets. Elle est structurée en routes, contrôleurs et modèles Mongoose.
- **La couche sécurité :** Assurée par un système de double authentification combinant un token **JWT** pour la gestion des sessions et un code **OTP** envoyé par email pour la vérification en deux étapes. Un middleware d'authentification protège toutes les routes sensibles de l'API.

- **La couche données :** Repose sur **MongoDB** pour le stockage et la persistance de l'ensemble des données de l'application (utilisateurs, clients, équipements, tickets, journaux d'activité). **Firebase** est utilisé en complément pour la gestion des notifications push en temps réel envoyées aux techniciens et HelpDesks.

II.3.2.2 Architecture physique de la plateforme

La plateforme TSS est déployée sur un serveur physique hébergeant une machine virtuelle Windows Server. Cette machine virtuelle centralise les différents composants de l'application.

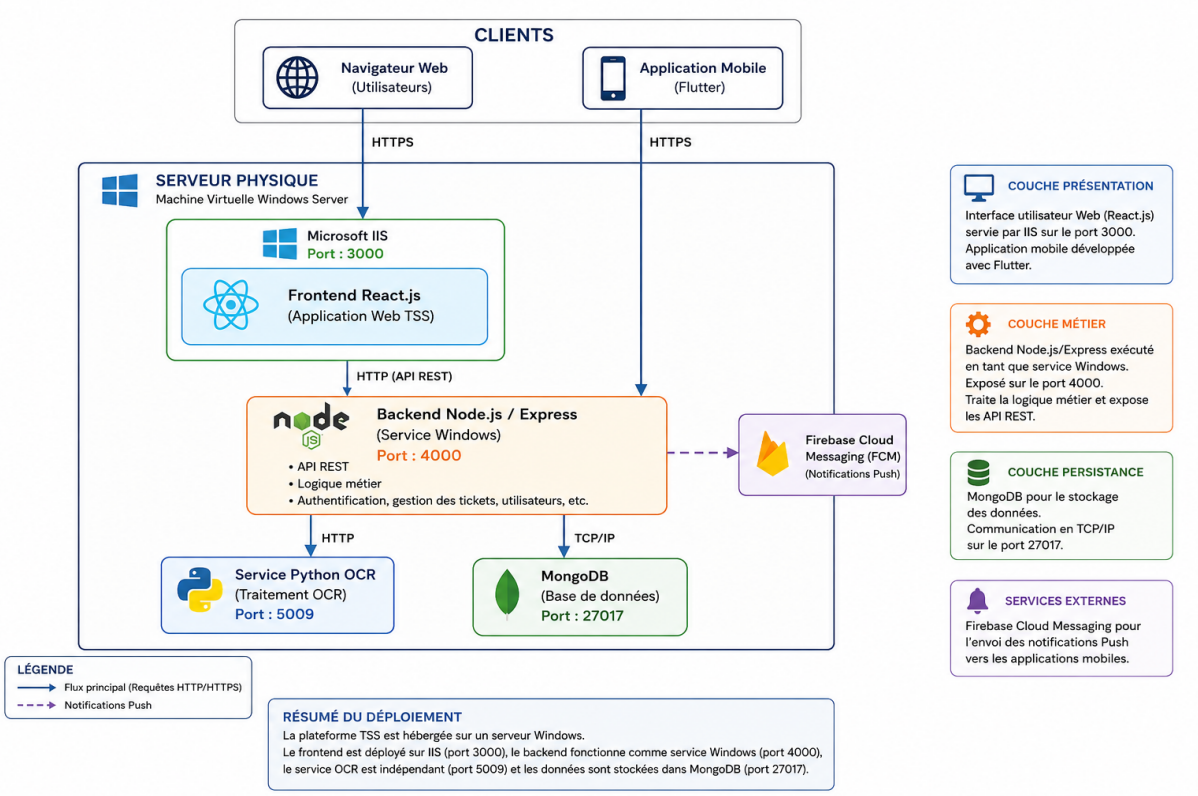


Figure II.4 : Architecture physique de la plateforme TSS.

L'architecture physique est composée des éléments suivants :

- **Frontend React.js :** l'application Web est déployée sous **IIS** et est accessible sur le **port 3000**. Les utilisateurs accèdent à cette interface via le protocole HTTPS.
- **Backend Node.js/Express :** le serveur applicatif est exécuté en tant que **service Windows**. Il expose les API REST sur le **port 4000** et assure le traitement de la logique métier.
- **Service Python OCR :** ce service indépendant est invoqué par le backend pour effectuer les traitements OCR des documents transmis.
- **MongoDB :** la base de données est hébergée sur la même machine virtuelle et communique avec le backend via TCP/IP sur le **port 27017**.

- **Firebase Cloud Messaging (FCM) :** utilisé pour l'envoi des notifications Push vers les applications mobiles.

II.4 Pilotage du projet avec SCRUM

Notre projet est piloté par Scrum, étant la méthode la plus sollicitée parmi les méthodes Agiles existantes.

II.4.1 Backlog du produit

Le tableau II.4.1 présente le backlog du produit TSS. Les fonctionnalités sont organisées par module et décrites sous forme de User Stories.

ID	Module	ID US	User Story	Complexité
M1	Authentification	M1.1	En tant qu'utilisateur, je souhaite me connecter à la plateforme.	Moyenne
		M1.2	En tant qu'utilisateur, je souhaite réinitialiser mon mot de passe.	Moyenne
		M1.3	En tant qu'utilisateur, je souhaite me déconnecter du système.	Faible
		M1.4	En tant qu'administrateur, je souhaite gérer les accès utilisateurs.	Moyenne
		M1.5	En tant qu'utilisateur, je souhaite modifier son mot de passe.	Faible
		M1.6	En tant qu'utilisateur, je souhaite consulter son profil.	Faible
		M1.7	En tant qu'utilisateur, je souhaite modifier ses informations personnelles.	Moyenne
M2	Gestion des utilisateurs	M2.1	En tant qu'administrateur, je souhaite ajouter un utilisateur.	Moyenne
		M2.2	En tant qu'administrateur, je souhaite modifier un utilisateur.	Moyenne
		M2.3	En tant qu'administrateur, je souhaite supprimer un utilisateur.	Moyenne
		M2.4	En tant qu'administrateur, je souhaite attribuer un rôle à un utilisateur.	Élevée
		M2.5	En tant qu'administrateur, je souhaite consulter la liste des utilisateurs.	Faible

Suite à la page suivante

ID	Module	ID US	User Story	Complexité
M3	Gestion des clients	M3.1	En tant que coordinateur, je souhaite ajouter un client.	Moyenne
		M3.2	En tant que coordinateur, je souhaite modifier les informations d'un client.	Moyenne
		M3.3	En tant que coordinateur, je souhaite consulter la liste des clients.	Faible
		M3.4	En tant que coordinateur, je souhaite rechercher un client.	Faible
M4	Gestion des tickets	M4.1	En tant que coordinateur, je souhaite créer un ticket.	Élevée
		M4.2	En tant que coordinateur, je souhaite affecter un ticket à un technicien.	Élevée
		M4.3	En tant que technicien, je souhaite consulter ses tickets affectés.	Moyenne
		M4.4	En tant que technicien, je souhaite mettre à jour le statut d'un ticket.	Élevée
		M4.5	En tant que coordinateur, je souhaite transférer un ticket à un autre technicien.	Élevée
		M4.7	En tant que manager, je souhaite suivre l'avancement des tickets.	Moyenne
		M4.8	En tant qu'utilisateur, je souhaite consulter l'historique d'un ticket.	Moyenne
M5	Gestion des équipements	M5.1	En tant que coordinateur, je souhaite ajouter un équipement.	Moyenne
		M5.2	En tant que coordinateur, je souhaite modifier un équipement.	Moyenne
		M5.3	En tant que coordinateur, je souhaite consulter les équipements.	Faible
		M5.4	En tant que coordinateur, je souhaite associer un équipement à un client.	Élevée
M6	Gestion des rapports	M6.1	En tant que manager, je souhaite consulter les statistiques des tickets.	Élevée
		M6.2	En tant que manager, je souhaite générer des rapports d'activité.	Élevée

Suite à la page suivante

ID	Module	ID US	User Story	Complexité
		M6.3	En tant que manager, je souhaite exporter les rapports au format PDF.	Moyenne
		M6.4	En tant que manager, je souhaite analyser les performances des techniciens.	Élevée
M7	Gestion des demandes clients	M7.1	En tant que client, je souhaite soumettre une demande d'assistance.	Élevée
		M7.2	En tant que client, je souhaite suivre l'état de sa demande.	Moyenne
		M7.3	En tant que client, je souhaite consulter l'historique de ses demandes.	Moyenne
		M7.4	En tant que coordinateur, je souhaite traiter les demandes des clients.	Élevée
M8	Gestion des logs	M8.1	En tant qu'administrateur, je souhaite consulter les journaux d'activité.	Moyenne
		M8.2	En tant qu'administrateur, je souhaite filtrer les logs par utilisateur ou date.	Moyenne
		M8.3	En tant qu'administrateur, je souhaite assurer la traçabilité des actions réalisées.	Élevée
M9	Gestion des alertes et notifications	M9.1	En tant qu'utilisateur, je souhaite recevoir des notifications lors d'un changement de statut d'un ticket.	Élevée
		M9.2	En tant que coordinateur, je souhaite être notifié lorsqu'un ticket est signalé.	Élevée
		M9.3	En tant que technicien, je souhaite recevoir une notification lorsqu'un ticket lui est affecté.	Moyenne
		M9.4	En tant que client, je souhaite être informé de l'évolution de sa demande.	Moyenne
		M9.5	En tant que manager, je souhaite recevoir des alertes sur les tickets critiques.	Élevée

Tab. II.3 : Backlog du produit

II.4.2 Planification des Sprints

Après l'élaboration du backlog produit et l'estimation de la charge de travail, le projet a été réparti en quatre sprints de quatre semaines chacun. Chaque sprint regroupe un ensemble cohérent de fonctionnalités permettant de livrer progressivement les différents modules de la plateforme.

Sprint	Modules développés
Sprint 1	• Authentification (M1) • Gestion des utilisateurs (M2) • Gestion des clients (M3) • Gestion des équipements (M5)
Sprint 2	• Gestion des tickets (M4) • Gestion tableaux de bord (M6)
Sprint 3	• Gestion des demandes clients (M7) • Gestion des alertes et notifications (M9)
Sprint 4	• Gestion des rapports et tableaux de bord (M6) • Gestion des logs et traçabilité (M8)

Tab. II.4 : Planification des sprints

II.5 Environnement de travail

II.5.1 Environnement matériel de développement

Le développement de l'application a été réalisé dans un environnement matériel adapté pour répondre aux exigences du projet. L'ordinateur utilisé dispose des caractéristiques suivantes :

- **Modèle :** Ordinateur portable GIGABYTE G5 MF5
- **Système d'exploitation :** Windows 11 (64 bits)
- **Processeur :** Intel(R) Core(TM) i7-13620H CPU @ 2400MHZ

- **Mémoire RAM :** 32 Go
- **Stockage :** 1 To SSD

II.5.2 Environnement logiciel de développement

Pour mener à bien la conception et le développement de la plateforme TSS, plusieurs outils ont été utilisés, allant de la modélisation UML à la documentation des API, en passant par le développement intégré et la gestion des bases de données.

Outil	Description	Logo
visual-paradigm	Logiciel en ligne utilisé pour la création des diagrammes UML (cas d'utilisation, classes, séquences) et des schémas d'architecture de la plateforme TSS.	
Visual Studio Code	Éditeur de code principal utilisé pour le développement du frontend React.js et du backend Node.js/Express. Il offre une prise en charge complète de JavaScript et de nombreuses extensions facilitant le développement.	
Postman	Outil utilisé pour tester, documenter et valider les endpoints de l'API REST de la plateforme TSS (tickets, équipements, utilisateurs, authentification).	
MongoDB Compass	Interface graphique officielle de MongoDB permettant de visualiser, explorer et manipuler les données stockées dans la base de données de la plateforme.	mongoDB Compass
GitHub	Plateforme de gestion de versions utilisée pour le suivi du code source, la collaboration entre développeurs et l'intégration avec la pipeline CI/CD Jenkins.	GitHub

Tab. II.5 : Outils utilisés pour la modélisation, le développement et la gestion de données

II.5.2.1 Langages de programmation

Le choix des langages de programmation est basé sur les exigences techniques de la plateforme TSS, notamment la performance du backend, la réactivité du frontend web et la portabilité de l'application mobile.

Langage	Description	Logo
JavaScript	Langage principal utilisé pour le développement du frontend (React.js) et du backend (Node.js/Express). Il permet une cohérence technologique entre les deux couches grâce à l'écosystème npm.	
Dart	Langage développé par Google, utilisé avec le framework Flutter pour développer l'application mobile des techniciens. Il offre des performances natives sur Android et iOS à partir d'une seule base de code.	
Python	Utilisé pour développer le module d'intelligence artificielle (OCR) de la plateforme. Sa richesse en bibliothèques de traitement d'image en fait un choix idéal pour ce type de fonctionnalité.	

Tab. II.6 : Langages de programmation utilisés dans la plateforme TSS

II.5.2.2 Frameworks et bibliothèques

Les frameworks et bibliothèques retenus ont été sélectionnés pour leur maturité, leur performance et leur adéquation aux besoins fonctionnels de la plateforme TSS.

Framework	Description	Logo
React.js	Bibliothèque JavaScript développée par Meta, utilisée pour construire l'interface web de la plateforme TSS. Son architecture basée sur les composants réutilisables garantit une interface moderne et réactive.	
Node.js / Express	Framework backend JavaScript permettant de construire une API REST performante et asynchrone. Il gère l'ensemble de la logique métier : authentification, tickets, équipements, clients et alertes.	
Flutter	Framework développé par Google permettant de créer l'application mobile des techniciens de terrain à partir d'une seule base de code, déployable sur Android et iOS.	

Tab. II.7 : Frameworks et bibliothèques utilisés dans la plateforme TSS

II.5.2.3 Outils DevOps et déploiement

La mise en production de la plateforme TSS repose sur une chaîne DevOps automatisée garantissant la fiabilité et la répétabilité des déploiements.

Outil	Description	Logo
Docker	Outil de conteneurisation utilisé pour empaqueter l'application et ses dépendances dans des conteneurs isolés, garantissant un environnement identique entre le développement et la production.	
Jenkins	Serveur d'intégration et de déploiement continu (CI/CD) utilisé pour automatiser les pipelines de build, de test et de déploiement de la plateforme TSS sur le serveur de production.	
Firebase	Plateforme cloud de Google utilisée pour l'envoi des notifications push en temps réel aux techniciens et aux HelpDesks lors de l'assignation ou de la mise à jour d'un ticket d'intervention.	

TAB. II.8 : Outils DevOps et de déploiement de la plateforme TSS

II.6 Conclusion

Nous nous sommes concentrés dans ce chapitre à faire l'étude globale du projet afin de bien planifier le projet. Un grand effort a été fourni durant ce Sprint Afin de choisir les bons outils à utiliser.

Chapitre III

Release 0 Mise en place des fonctionnalités fondamentales

III.1 Introduction

Ce chapitre est dédié à la première phase d'implémentation de la plateforme TSS, correspondant à la Release 1 du projet. Elle regroupe les deux premiers sprints qui constituent le socle fonctionnel du système.

Le Sprint 1 couvre l'authentification ainsi que la gestion des utilisateurs, des clients, des agences et des équipements.

Le Sprint 2 porte sur la gestion des tickets d'intervention et du tableau de bord.

L'objectif de cette release est de mettre en place les fonctionnalités essentielles permettant à la plateforme d'être opérationnelle dans un contexte réel d'utilisation.

III.2 Backlog du Sprint 1

Après la définition des objectifs du Sprint 1, les User Stories sélectionnées ont été décomposées en tâches techniques afin de faciliter leur implémentation et leur suivi.

Le tableau III.1 présente le backlog détaillé du Sprint 1.

User Story	**Tâches**	**Estimation**
En tant qu'utilisateur, je veux me connecter à la plateforme.	Développer l'API d'authentification.	1
	Développer l'interface de connexion.	1
	Implémenter la validation des champs.	1
	Implémenter l'envoi du code OTP par email.	1
	Développer la vérification du code OTP.	1
	Tester la fonctionnalité.	1

User Story	Tâches	Estimation
En tant qu'utilisateur, je veux réinitialiser mon mot de passe.	Développer l'API de réinitialisation du mot de passe.	1
	Implémenter l'envoi du lien de réinitialisation par email.	1
	Développer l'interface de changement du mot de passe.	1
	Tester la fonctionnalité.	1
En tant qu'utilisateur, je veux consulter et modifier mon profil.	Développer l'API de consultation du profil.	1
	Développer l'API de modification du profil.	1
	Réaliser l'interface de gestion du profil.	1
	Tester la fonctionnalité.	1
En tant qu'utilisateur, je veux me déconnecter de la plateforme.	Implémenter la déconnexion sécurisée.	1
	Tester la fonctionnalité.	1
En tant que manager, je veux gérer les utilisateurs.	Développer l'API d'ajout d'utilisateur.	1
	Développer l'API de modification des rôles.	1
	Développer l'API de suppression d'utilisateur.	1
	Réaliser l'interface de gestion des utilisateurs.	1
	Tester les fonctionnalités.	1
En tant que coordinatrice, je veux gérer les clients.	Développer l'API d'ajout de client.	1
	Développer l'API de modification des clients.	1
	Développer l'API de consultation et recherche.	1
	Réaliser les interfaces de gestion des clients.	1
	Tester les fonctionnalités.	1
En tant que coordinatrice, je veux gérer les équipements.	Développer l'API d'ajout d'équipement.	1
	Développer l'API d'ajout d'un fournisseur.	1
	Développer l'API de modification des équipements.	1
	Développer l'API de suppression des équipements.	1
	Réaliser les interfaces de gestion des équipements.	1

User Story	Tâches	Estimation
	Tester les fonctionnalités.	1
En tant que coordinatrice, je veux gérer les agences.	Développer l'API d'ajout d'agence.	1
	Développer l'API de modification des agences.	1
	Développer la consultation et la recherche des agences.	1
	Tester les fonctionnalités.	1

Tab. III.1 : Backlog du Sprint 2

III.3 Diagramme de cas d'utilisation du Sprint 1

Dans cette section, nous présentons le diagramme de cas d'utilisation correspondant aux fonctionnalités développées lors du premier sprint, tel qu'illustré dans la figure III.1.

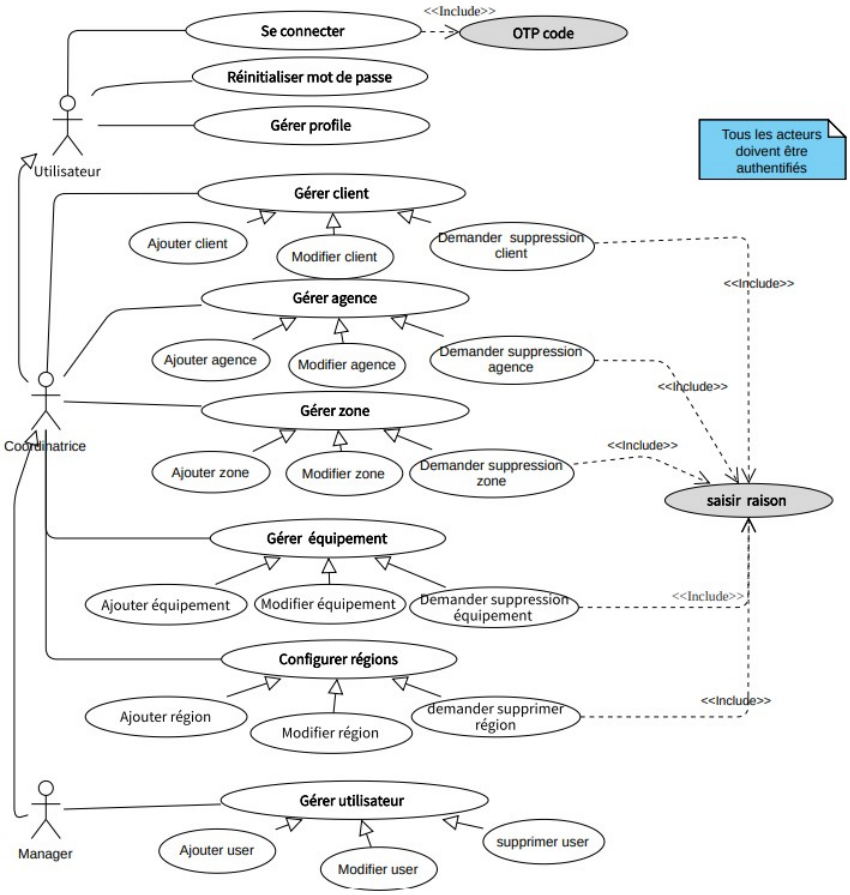


Figure III.1 : Diagramme de cas d'utilisation du Sprint 1

III.4 Services Web

Les services Web fournissent des services via Internet. Ils constituent donc une interface qui permet à deux machines (ou applications) de communiquer. Dans cette section nous allons présenter la liste des services web composant l'API de notre application. Les services web présentés ci-dessous permettent d'assurer la performance de notre plateforme dans le tableau III.2.

Méthode	URL	Description
Authentification		
Post	/api/users/login	Se connecter avec email et mot de passe.
Post	/api/users/loginotp	Initier la connexion via code OTP.
Post	/api/users/verify-otp	Valider le code OTP reçu par email.
Post	/api/email/send-reset-password-email	Envoyer un email de réinitialisation de mot de passe.
Post	/api/email/reset-password	Réinitialiser le mot de passe via le token.
Get	/api/users/profile/ :id	Consulter le profil d'un utilisateur.
Patch	/api/users/ :id	Modifier les informations du profil.
Gestion des utilisateurs		
Post	/api/users/add	Ajouter un nouvel utilisateur avec son rôle.
Patch	/api/users/ :id	Modifier le rôle ou les informations d'un utilisateur.
Delete	/api/users/ :id	Supprimer un utilisateur du système.
Get	/api/users/list	Récupérer la liste de tous les utilisateurs.
Gestion des clients		
Post	/api/clients/add	Ajouter un nouveau client.
Put	/api/clients/ :id	Modifier les informations d'un client.
Delete	/api/clients/ :id	Supprimer un client et ses agences.
Get	/api/clients/list	Récupérer la liste de tous les clients.

Gestion des équipements		
Post	/api/marques/add	Ajouter un fournisseur (marque).
Post	/api/equipements/add	Ajouter un nouvel équipement.
Patch	/api/equipements/ :id	Modifier les informations d'un équipement.
Delete	/api/equipements/ :id	Supprimer un équipement du système.
Get	/api/equipements/list	Récupérer la liste de tous les équipements.
Get	/api/equipements/ :id	Récupérer un équipement par son identifiant.

Tab. III.2 : Liste des services web du sprint 1.

III.5 Les cas d'utilisation du Sprint 1

III.5.1 Cas d'utilisation : « Se connecter avec OTP »

Dans cette partie, nous proposons une description textuelle accompagnée du diagramme de séquence système relatif au cas d'utilisation « Se connecter avec OTP ».

III.5.1.1 Description textuelle

Acteurs	Coordinatrice, Manager, Technicien, Client, Super Client
Objectif	Permettre à l'utilisateur de s'authentifier sur la plateforme de manière sécurisée grâce à une double vérification par code OTP.
Pré-condition	L'utilisateur doit disposer d'un compte actif dans le système et d'une adresse email valide.
Scénario principal	1- L'utilisateur accède à la page de connexion et saisit son adresse email et son mot de passe. 2- Le système vérifie les identifiants et génère un code OTP à usage unique. 3- Le code OTP est envoyé à l'adresse email de l'utilisateur (valide 10 minutes). 4- L'utilisateur saisit le code OTP reçu dans le champ de vérification. 5- Le système valide le code et génère un token JWT. 6- L'utilisateur est redirigé vers son tableau de bord selon son rôle.

Scénario alternatif	**Identifiants incorrects :** 1- Le système affiche un message d'erreur « Email ou mot de passe incorrect ». 2- L'utilisateur peut réessayer ou cliquer sur « Mot de passe oublié ». **Code OTP expiré :** 1- Si l'utilisateur saisit le code après 10 minutes, le système l'informe de l'expiration. 2- L'utilisateur doit relancer le processus de connexion pour recevoir un nouveau code.

Tab. III.3 : Cas d'utilisation : Se connecter avec OTP

III.5.1.2 Diagramme de séquence système

La figure III.2 illustre le diagramme de séquence système associé à ce cas d'utilisation, mettant en évidence les interactions entre l'utilisateur et le système lors du processus d'authentification.

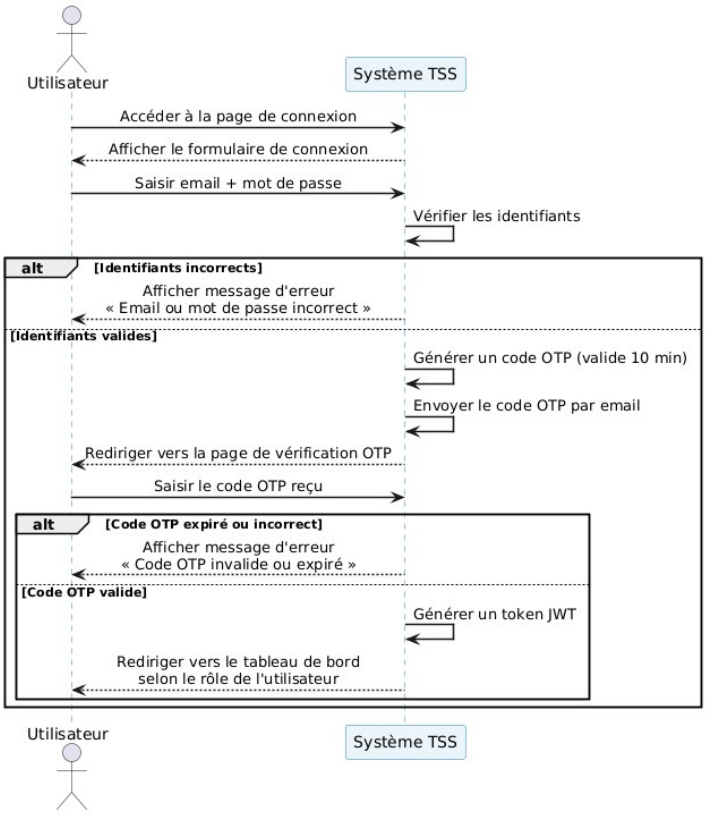


Figure III.2 : Diagramme de séquence système du cas d'utilisation « Se connecter avec OTP »

III.5.1.3 Diagramme de séquence objet

La figure III.3 illustre le diagramme de séquence objet associé au cas d'utilisation « Se connecter avec OTP », détaillant les interactions entre les composants internes du système (contrôleur, service, base de données).

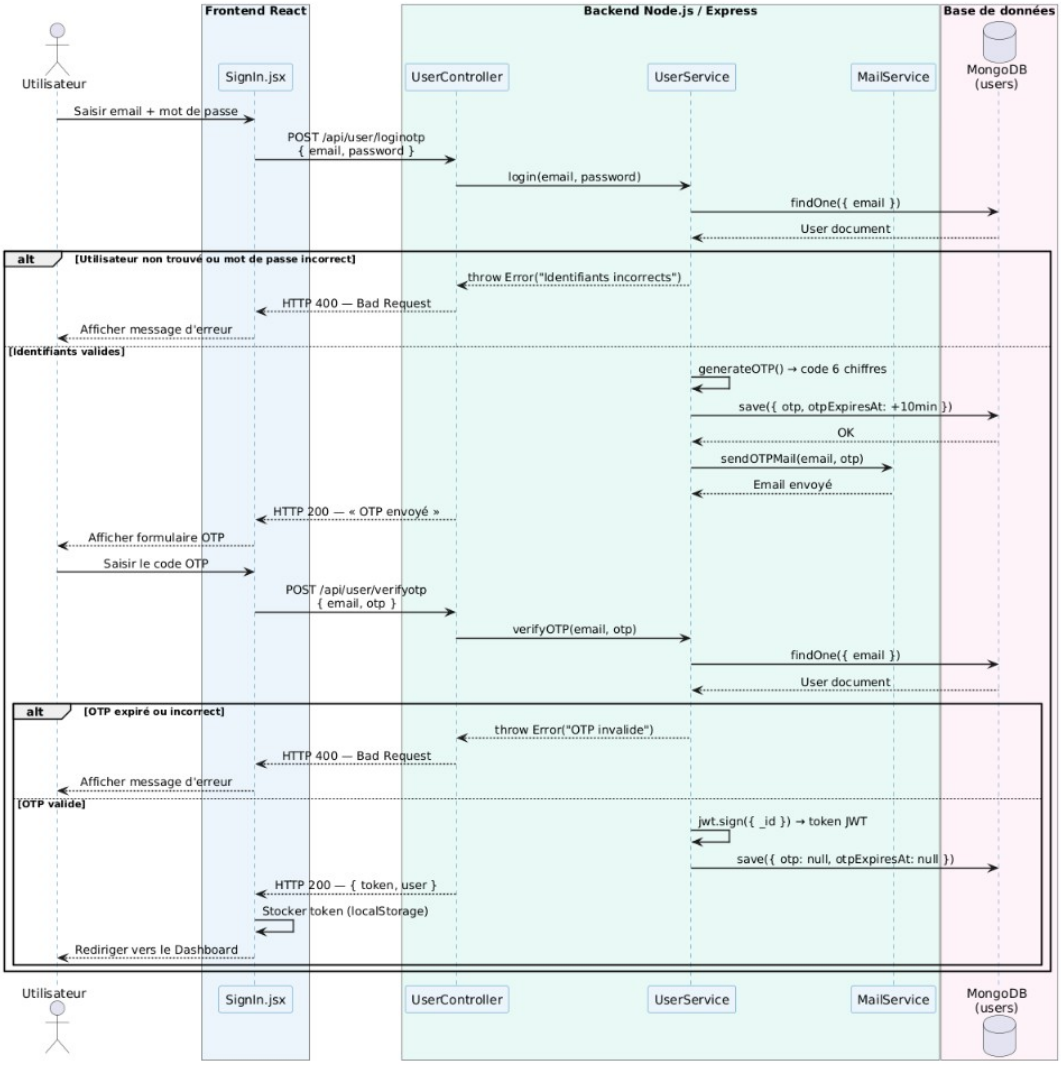


Figure III.3 : Diagramme de séquence objet du cas d'utilisation « Se connecter avec OTP »

III.6 Réalisation du Sprint 1

- **Interface de connexion :**

La figure III.4 illustre la page de connexion de la plateforme TSS. L'utilisateur y renseigne son adresse email et son mot de passe avant de soumettre le formulaire. Une fois les identifiants validés, un code OTP est automatiquement envoyé à son adresse email.

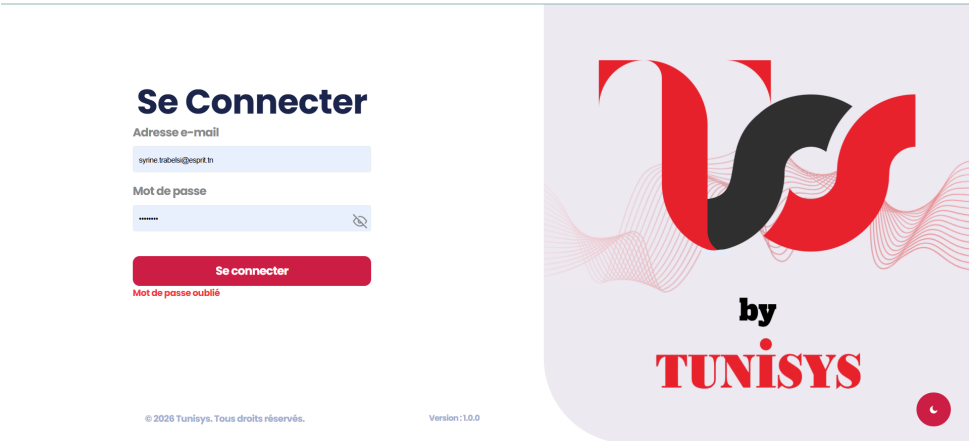


Figure III.4 : Interface de connexion

- **Interface de vérification OTP :**

La figure III.5 présente l'interface de saisie du code OTP. Après avoir reçu le code par email, l'utilisateur le saisit dans ce formulaire pour finaliser son authentification. Le code est valable pendant 10 minutes.

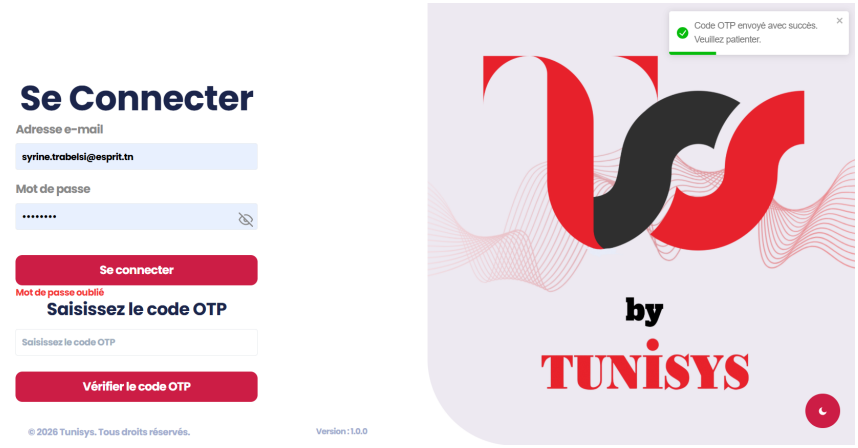


Figure III.5 : Interface de vérification OTP

- **Liste des utilisateurs :**

La figure III.6 illustre l'interface de consultation de la liste des utilisateurs enregistrés dans le système. Le manager peut y visualiser le nom, le prénom, l'email et le rôle de chaque utilisateur, et effectuer des actions de modification ou de suppression.

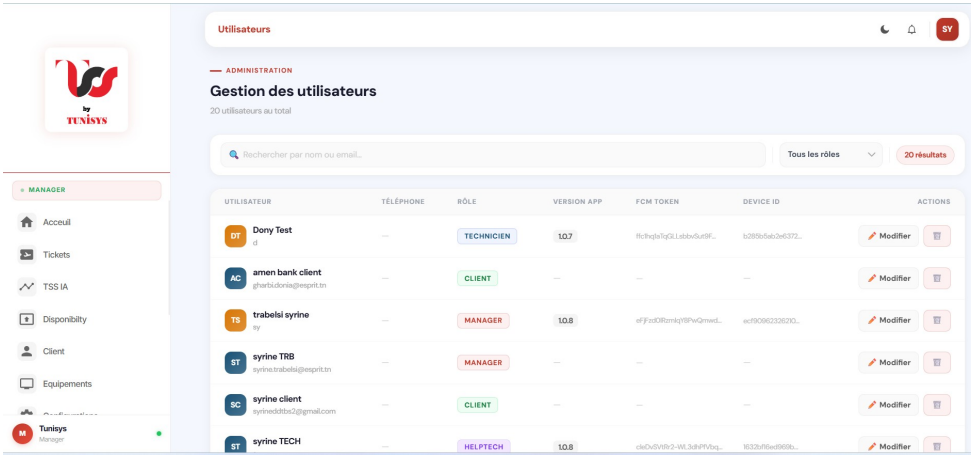


Figure III.6 : Liste des utilisateurs

- **Ajout d'un nouveau client :**

La figure III.7 présente le formulaire permettant à la coordinatrice d'ajouter un nouveau client, en renseignant le nom, les emails de contact et les horaires de disponibilité.

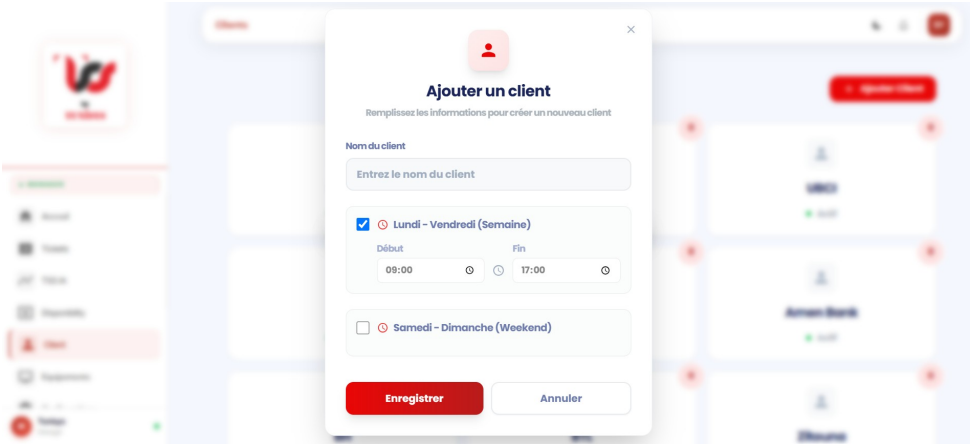


Figure III.7 : Ajout d'un client

- **Gestion des agences :**

La figure III.8 illustre l'interface de gestion des agences associées à un client. La coordinatrice peut y ajouter, modifier ou supprimer une agence, et préciser sa localisation géographique (zone, gouvernorat).

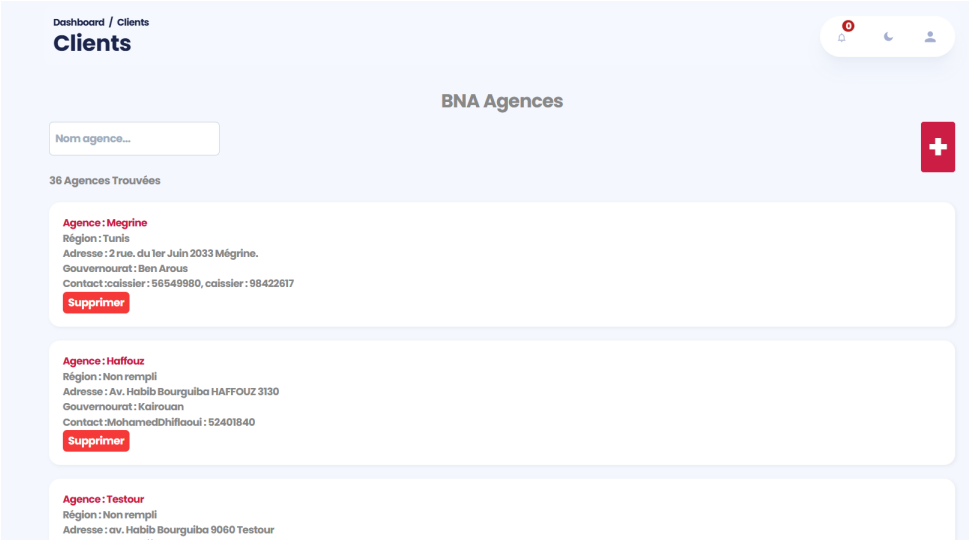


Figure III.8 : Gestion des agences

- **Consultation des équipements :**

La figure III.9 illustre l'interface de consultation de la liste des équipements enregistrés dans la plateforme. La coordinatrice peut y visualiser l'ensemble des équipements avec leurs informations principales (numéro de série, type, modèle, client et agence associés, statut), et effectuer des actions de modification ou de suppression.

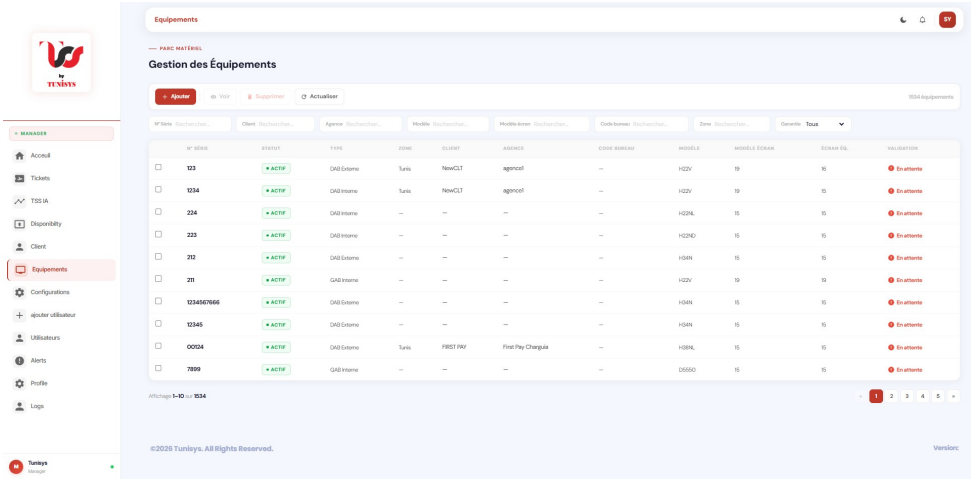


Figure III.9 : Consultation des équipements

III.7 Backlog du Sprint 2

Après la définition des objectifs du Sprint 2, les User Stories sélectionnées ont été décomposées en tâches techniques afin de faciliter leur implémentation et leur suivi.

Le tableau III.4 présente le backlog détaillé du Sprint 2.

User Story	Tâches	Estimation
En tant que coordinatrice, je veux créer un ticket d'intervention téléphonique.	Développer l'API de création d'un ticket téléphonique.	1
	Implémenter la génération automatique de la référence.	1
	Développer l'assignation du ticket à un HelpDesk.	1
	Implémenter la notification en temps réel.	1
	Réaliser l'interface de création du ticket.	1
	Tester la fonctionnalité.	1
En tant que coordinatrice, je veux créer un ticket d'intervention terrain.	Développer l'API de création d'un ticket terrain.	1
	Implémenter la génération automatique de la référence.	1
	Développer l'assignation du ticket à un technicien.	1
	Implémenter la notification en temps réel.	1
	Réaliser l'interface de création du ticket terrain.	1
	Tester la fonctionnalité.	1
En tant que technicien/HelpDesk, je veux traiter un ticket assigné.	Développer l'API d'acceptation d'un ticket.	1
	Développer l'API de démarrage et de résolution.	1
	Implémenter la mise à jour du statut en temps réel.	1
	Réaliser l'interface de traitement du ticket.	1
	Tester la fonctionnalité.	1
En tant que technicien/HelpDesk, je veux reporter un ticket non résolu.	Développer l'API de report de ticket.	1
	Implémenter la saisie de la raison et de la date de report.	1
	Réaliser l'interface de report de ticket.	1
	Tester la fonctionnalité.	1
En tant que coordinatrice, je veux consulter et rechercher les tickets.	Développer l'API de récupération des tickets.	1
	Implémenter le filtrage par statut, type et client.	1

User Story	Tâches	Estimation
	Réaliser l'interface de consultation des tickets.	1
	Tester la fonctionnalité.	1
En tant que manager, je veux consulter le tableau de bord.	Développer les APIs de statistiques et de comptage.	1
	Implémenter les graphiques de suivi des tickets.	1
	Réaliser l'interface du tableau de bord.	1
	Tester la fonctionnalité.	1

Tab. III.4 : Backlog du Sprint 2

III.8 Diagramme de cas d'utilisation du Sprint 2

Dans cette section, nous présentons le diagramme de cas d'utilisation correspondant aux fonctionnalités développées lors du deuxième sprint, tel qu'illustré dans la figure III.10.

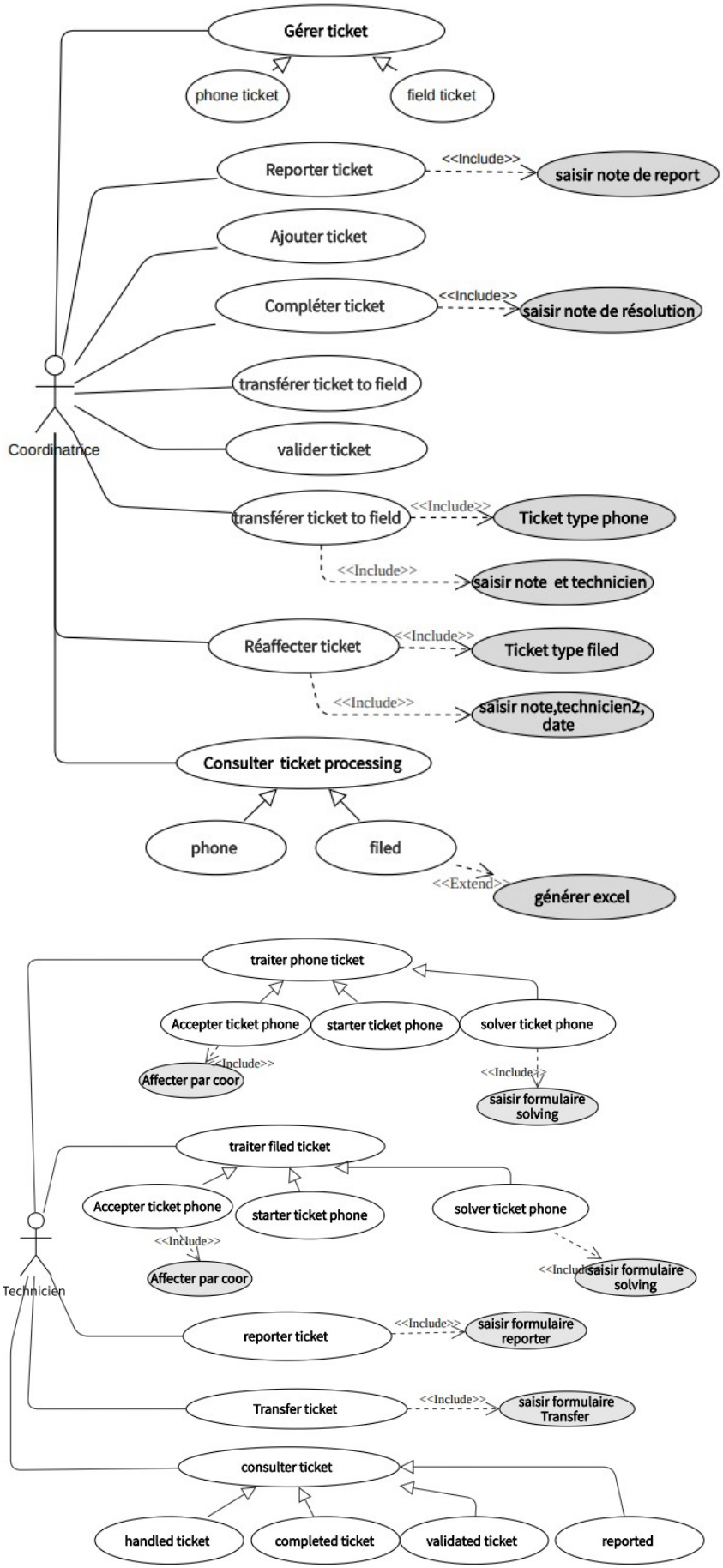


Figure III.10 : Diagramme de cas d'utilisation du Sprint 2

III.9 Services Web du Sprint 2

Les services Web fournissent des services via Internet. Ils constituent une interface permettant à deux applications de communiquer. Dans cette section, nous présentons les principaux services web du Sprint 2, dédiés à la gestion des tickets d'intervention. Les services web présentés ci-dessous sont résumés dans le tableau III.5.

Méthode	URL	Description
Gestion des tickets téléphoniques		
Post	`/api/tickets/phone`	Créer un ticket d'intervention téléphonique.
Get	`/api/tickets/phone`	Récupérer la liste des tickets téléphoniques.
Get	`/api/tickets/number/phone`	Obtenir le nombre de tickets téléphoniques.
Get	`/api/tickets/number/phone`	Récupérer les tickets téléphoniques créés durant la semaine.
Gestion des tickets terrain		
Post	`/api/tickets/field`	Créer un ticket d'intervention terrain.
Get	`/api/tickets/field`	Récupérer la liste des tickets terrain.
Get	`/api/tickets/number/field`	Obtenir le nombre de tickets terrain.
Get	`/api/tickets/fieldLastWeek`	Récupérer les tickets terrain créés durant la semaine.
Traitement des tickets		
Get	`/api/tickets/:id`	Consulter un ticket par son identifiant.
Put	`/api/tickets/accepted/:id`	Accepter un ticket assigné.
Put	`/api/tickets/started/:id`	Démarrer le traitement d'un ticket.
Put	`/api/tickets/solved/:id`	Marquer un ticket comme résolu.

Traitement des tickets		
Put	`/api/tickets/reported/:id`	Reporter un ticket non résolu.
Put	`/api/tickets/approved/:id`	Approuver la résolution d'un ticket.
Put	`/api/tickets/transfer/:id`	Transférer un ticket vers un autre technicien.
Delete	`/api/tickets/:id`	Supprimer un ticket du système.

Tableau de bord		
Get	`/api/tickets/number`	Obtenir le nombre total des tickets.
Get	`/api/tickets/nbreported`	Obtenir le nombre de tickets reportés.
Get	/api/tickets/nbclient	Obtenir le nombre de clients actifs.
Get	`/api/tickets/nbtechnicien`	Obtenir le nombre de techniciens actifs.
Get	`/api/tickets/solvedLastWeek`	Récupérer les tickets résolus durant la semaine.

Tab. III.5 : Liste des services web du Sprint 2.

III.10 Les cas d'utilisation du Sprint 2

III.10.1 Cas d'utilisation : « Ajouter un ticket »

Dans cette partie, nous proposons une description textuelle accompagnée du diagramme de séquence système relatif au cas d'utilisation « Ajouter un ticket ».

III.10.1.1 Description textuelle

Acteurs	Coordinatrice
Objectif	Permettre à la coordinatrice de créer un ticket d'intervention (téléphonique ou terrain) et de l'assigner automatiquement à un technicien ou un HelpDesk disponible.

Pré-condition	La coordinatrice est authentifiée dans le système. Un équipement, un client et une agence associée doivent exister dans la base de données. Au moins un technicien ou HelpDesk doit être disponible.
Scénario principal	1- La coordinatrice accède au module de gestion des tickets. 2- Elle sélectionne le type de ticket (téléphonique ou terrain). 3- Le système affiche le formulaire de création. 4- La coordinatrice sélectionne le client, l'agence et l'équipement concerné. 5- Elle saisit le type de service, la note d'intervention et la date d'appel. 6- Elle sélectionne le technicien ou HelpDesk à assigner. 7- Elle soumet le formulaire. 8- Le système génère automatiquement une référence unique pour le ticket. 9- Le système enregistre le ticket avec le statut « ASSIGNED ». 10- Le système envoie une notification en temps réel au technicien assigné. 11- Le système affiche un message de confirmation.
Scénario alternatif	**Champs obligatoires manquants :** 1- Le système détecte des champs vides (équipement, type, note). 2- Le système met en évidence les champs manquants. 3- La coordinatrice complète les informations et soumet à nouveau. **Aucun technicien disponible :** 1- Le système détecte qu'aucun technicien ou HelpDesk n'est disponible. 2- Le système informe la coordinatrice et bloque la création. 3- La coordinatrice reporte la création du ticket.

Tab. III.6 : Cas d'utilisation : Ajouter un ticket

III.10.1.2 Diagramme de séquence système

La figure III.11 illustre le diagramme de séquence système associé à ce cas d'utilisation, mettant en évidence les interactions entre la coordinatrice et le système lors du processus de création d'un ticket d'intervention.

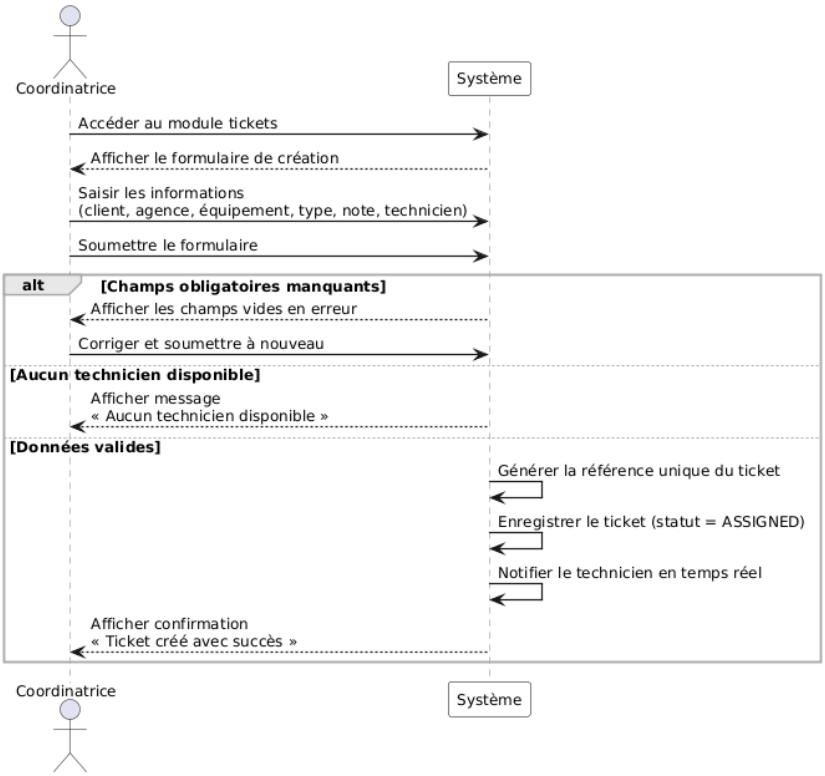


Figure III.11 : Diagramme de séquence système « Ajouter un ticket »

III.10.1.3 Diagramme de séquence objet

La figure III.12 illustre le diagramme de séquence objet associé au cas d'utilisation « Ajouter un ticket », détaillant les interactions entre les composants internes du système (contrôleur, modèle Mongoose, base de données et service de notification).

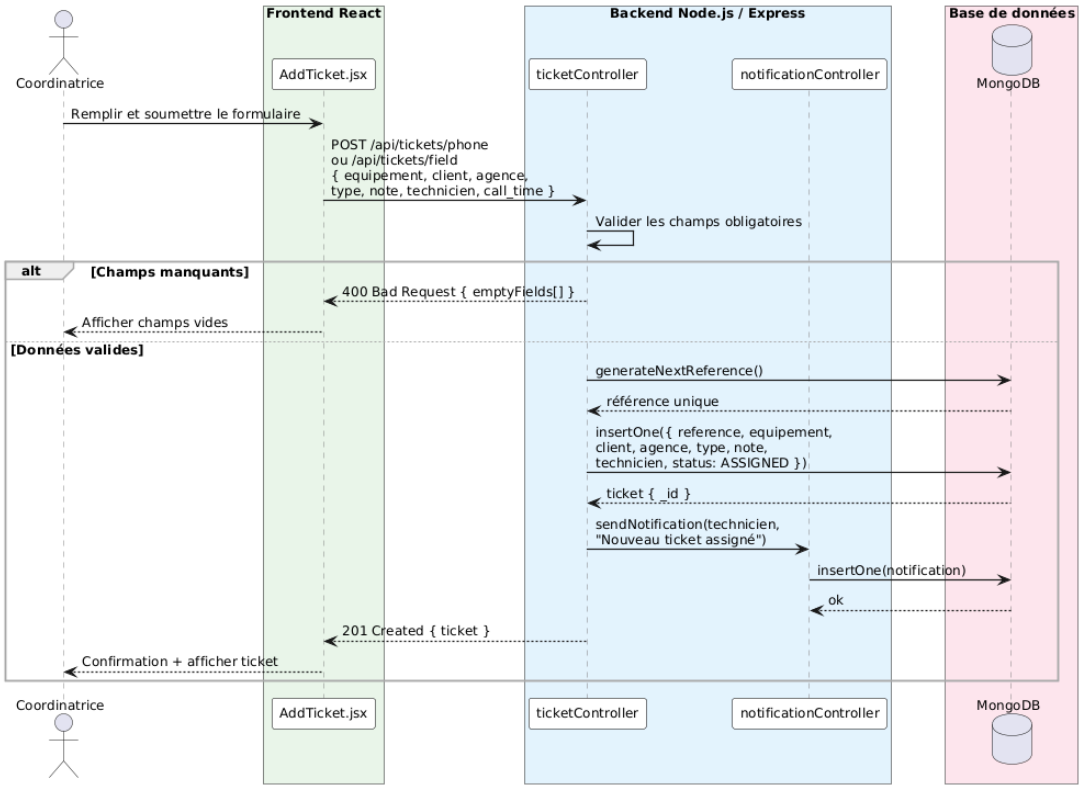


Figure III.12 : Diagramme de séquence objet « Ajouter un ticket »

III.11 Réalisation du Sprint 2

- **Liste des tickets téléphoniques :**

La figure III.13 illustre l'interface de consultation des tickets d'intervention téléphoniques. La coordinatrice peut y visualiser l'ensemble des tickets avec leur référence, le client concerné, le statut et le HelpDesk assigné. Elle dispose également d'options de filtrage et de recherche pour retrouver rapidement un ticket.

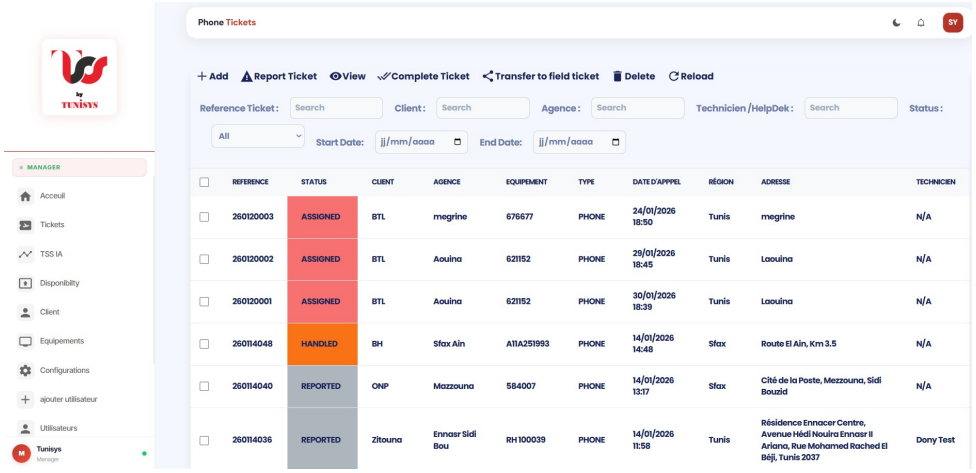


Figure III.13 : Liste des tickets téléphoniques

- **Ajout d'un ticket téléphonique :**

La figure III.14 présente le formulaire de création d'un ticket téléphonique. La coordinatrice sélectionne le client, l'agence et l'équipement concerné, saisit la note d'intervention et la date d'appel, puis assigne le ticket au HelpDesk disponible. Le système génère automatiquement une référence unique dès la validation du formulaire.

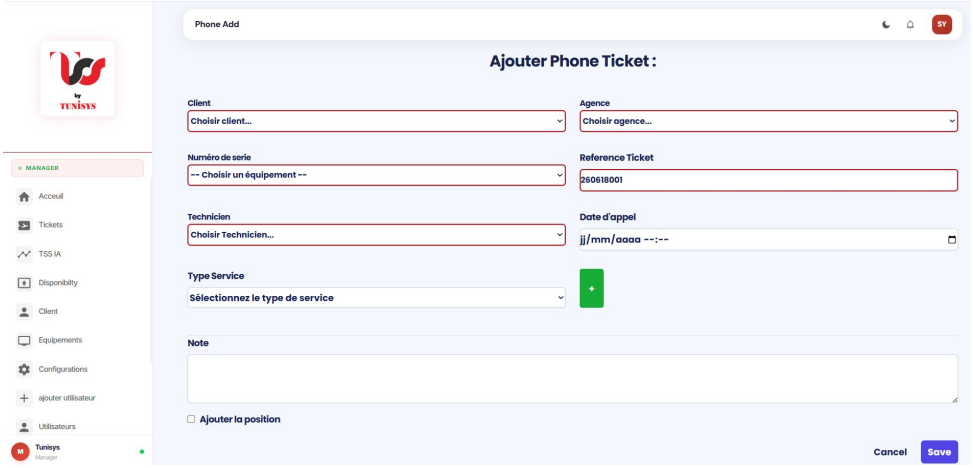


Figure III.14 : Ajout d'un ticket téléphonique

- **Tickets reportés :**

La figure III.15 illustre l'interface de consultation des tickets reportés. La coordinatrice peut y visualiser les tickets dont l'intervention n'a pas pu être réalisée à temps, accompagnés de la raison du report, de la note du technicien et de l'horodatage. Cette vue lui permet de planifier les relances et de suivre les tickets en attente.

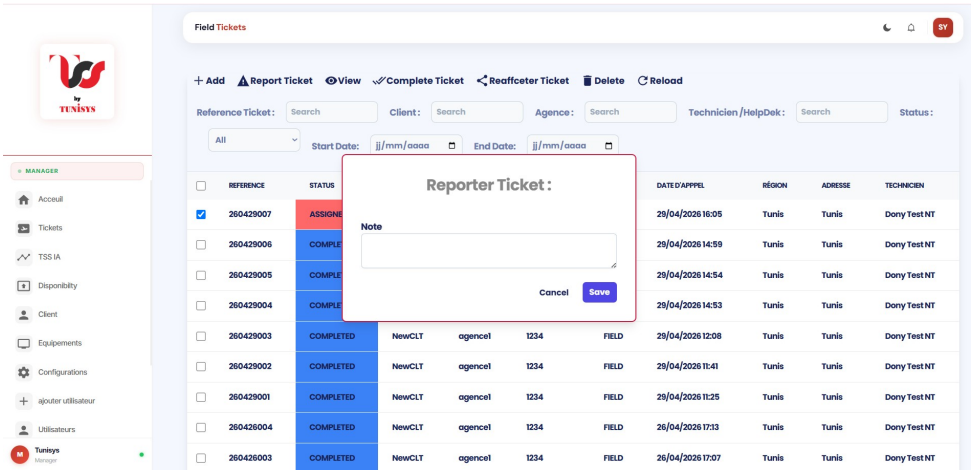


Figure III.15 : Tickets reportés

- **Tableau de bord :**

La figure III.16 illustre le tableau de bord de la plateforme TSS. Il offre au manager une vue d'ensemble des indicateurs clés : nombre total de tickets téléphoniques et terrain,

tickets reportés, nombre de techniciens et de clients actifs. Des graphiques interactifs permettent de visualiser l'évolution des interventions et la répartition des statuts sur une période donnée.

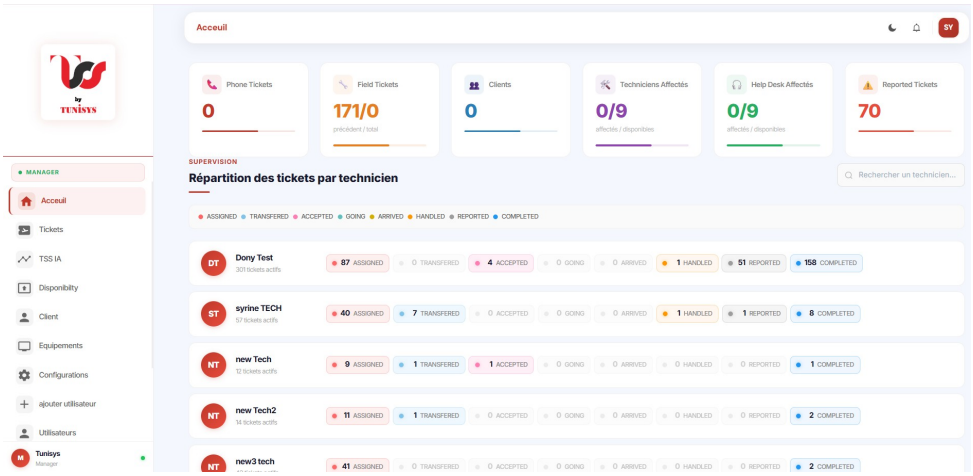


Figure III.16 : Tableau de bord

III.12 Conclusion

Ce chapitre a permis de présenter les fonctionnalités réalisées dans le cadre de la Release 1 de la plateforme TSS. Les deux premiers sprints ont permis la mise en place des modules d'authentification sécurisée par OTP, de gestion des utilisateurs, des clients, des agences et des équipements, ainsi que de la gestion complète des tickets d'intervention téléphoniques et terrain et du tableau de bord de supervision.

Ces fonctionnalités constituent le socle fonctionnel de la plateforme et assurent une gestion structurée, sécurisée et traçable des opérations de support et de maintenance.

Chapitre IV

Release 1:Gestion des espaces clients et pilotage des interventions

IV.1 Introduction

Ce chapitre est consacré au Sprint 3 de la plateforme TSS, constituant la deuxième release du projet. Il porte sur la fonctionnalité de **demande client**, permettant au client de soumettre directement ses demandes d'intervention depuis la plateforme, sans passer par la coordinatrice.

IV.2 Backlog du Sprint 3

Après la définition des objectifs du Sprint 3, les User Stories sélectionnées à partir du Product Backlog ont été décomposées en tâches techniques afin de faciliter leur implémentation et leur suivi.

Le tableau IV.1 présente le backlog détaillé du Sprint 3.

User Story	**Tâches**	**Estimation**
En tant que client, je veux Consulter consulter mes tickets ainsi que les statistiques relatives à mes équipements et agences.	Développer le Dashboard Client et Super-Client.	2
	Développer les API d'alimentation des statistiques.	1
	Afficher les statistiques des tickets par état.	1
	Afficher les statistiques des équipements.	1
	Afficher les statistiques des agences.	1
	Intégrer les graphiques et indicateurs de performance.	2
	Développer l'interface React du Dashboard.	1

User Story	Tâches	Estimation
	Tester les fonctionnalités.	1
En tant que client, je veux consulter la documentation des services et utiliser DocIA afin d'identifier automatiquement le type d'incident.	Construire la base de connaissances des incidents.	2
	Développer l'espace Documentation.	1
	Intégrer Ollama / Llama 3.	2
	Implémenter le mécanisme de fallback local.	1
	Développer l'interface React du chatbot DocIA.	2
	Pré-remplir automatiquement le type de service détecté.	1
	Tester la fonctionnalité.	1
En tant que superclient, je veux créer une demande d'intervention depuis la plateforme.	Concevoir le modèle `TicketClient`.	1
	Développer les contrôleurs de création des demandes.	1
	Exposer les API REST correspondantes.	1
	Développer les formulaires React.	2
	Valider les champs obligatoires.	1
	Autoriser l'ajout des images.	1
	Envoyer les notifications temps réel aux coordinatrices.	1
	Tester les fonctionnalités.	1
En tant que client, je veux suivre l'évolution de mes demandes et recevoir les notifications associées.	Développer les API de consultation des demandes.	1
	Afficher les demandes en attente, modifiées et rejetées.	1
	Afficher l'historique des statuts du ticket.	1
	Afficher les motifs de rejet.	1
	Afficher la progression du ticket.	1
	Envoyer des notifications e-mail lors des changements d'état.	1
	Tester les fonctionnalités.	1
En tant que coordinatrice, je veux gérer les demandes des clients	Développer l'interface de gestion des demandes.	2
	Afficher les informations des demandes.	1

User Story	Tâches	Estimation
	Rechercher et filtrer les demandes.	1
	Modifier les informations de la demande.	1
	Compléter les champs manquants.	1
	Affecter un technicien.	1
	Valider ou annuler une demande avec motif.	1
	Convertir `TicketClient` en ticket officiel.	2
	Tester les fonctionnalités.	1
En tant que système, je veux contrôler les délais de traitement afin de garantir le respect des SLA.	Développer le contrôleur des SLA.	2
	Configurer les calendriers des clients.	2
	Calculer automatiquement le début réel du SLA.	2
	Envoyer un rappel aux coordinatrices après 15 minutes.	1
	Déclencher une escalade vers le manager après 30 minutes.	1
	Enregistrer les événements d'escalade.	1
	Tester les mécanismes automatiques.	1
En tant que manager, je veux superviser les demandes clients et les dépassements des SLA.	Développer le Dashboard Manager.	2
	Afficher les demandes en attente.	1
	Afficher les demandes modifiées et annulées.	1
	Afficher les alertes de dépassement des SLA.	1
	Modifier ou rejeter une demande.	1
	Développer les indicateurs de supervision.	1
	Tester les fonctionnalités.	1

Tab. IV.1 : Backlog du Sprint 3

IV.3 Diagramme de cas d'utilisation du Sprint 3

La figure IV.1 présente le diagramme de cas d'utilisation regroupant l'ensemble des fonctionnalités du Sprint 3.

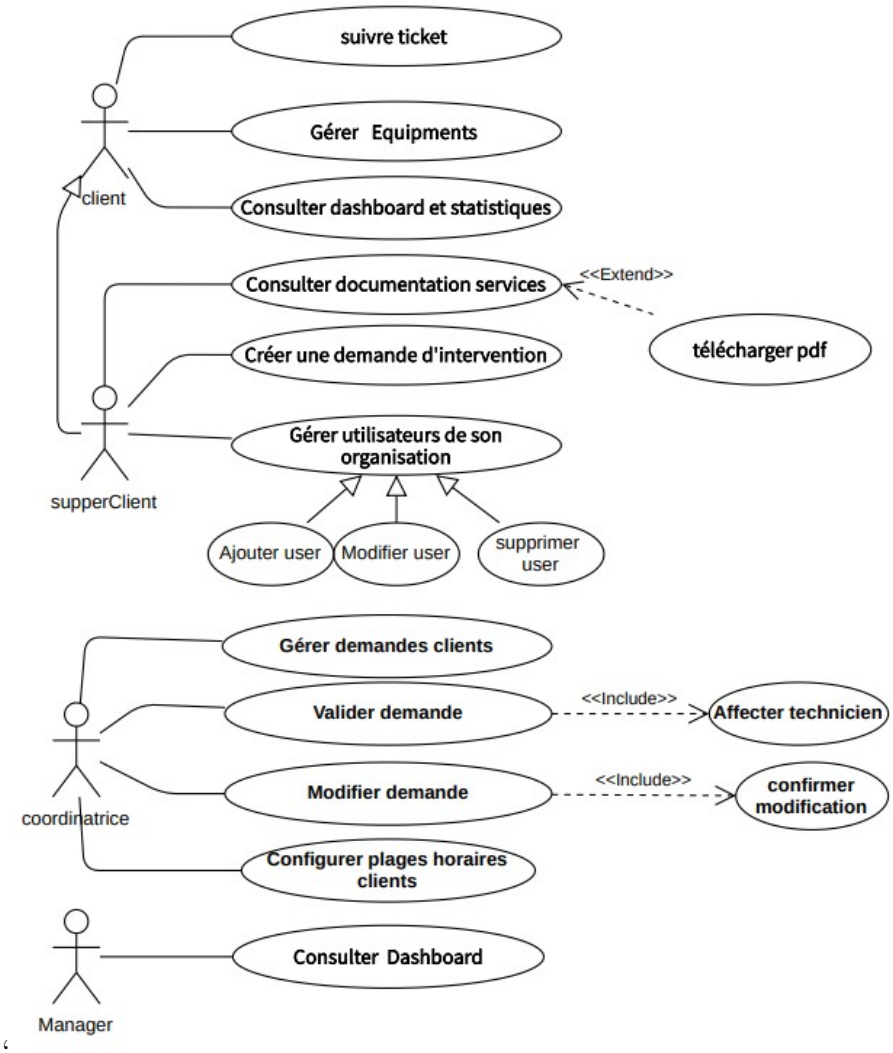


Figure IV.1 : Diagramme de cas d'utilisation du Sprint 3

IV.4 Services Web du Sprint 3

Les tableaux suivants présentent les services web exposés par l'API TSS pour les fonctionnalités du Sprint 3.

Méthode	URL	Description
Demande Client		
Post	/api/tickets/client-ticket/phone	Soumettre une demande d'intervention téléphonique.
Post	/api/tickets/client-ticket/field	Soumettre une demande d'intervention terrain.
Get	/api/tickets/pending/client	Consulter les demandes en attente de traitement.
Get	/api/tickets/rejected/client	Consulter les demandes rejetées avec leur motif.
Get	/api/tickets/modified/client	Consulter les demandes modifiées par la coordinatrice.
Get	/api/tickets/client/by-ref/ :ref	Récupérer une demande par sa référence.
Post	/api/tickets/ :ticketId/cancel	Annuler une demande en attente (avec motif).
Traitement des demandes — Coordinatrice / Manager		
Put	/api/tickets/complete/coor / :ticketId	Valider une demande et de la convertir en ticket d'intervention.
Post	/api/tickets/ :ticketId/cancel	Rejeter une demande avec enregistrement du motif.
Chatbot IA — DocBot		
Post	/api/ai/documentation	Analyser la description d'un incident et retourner le type de ticket le plus proche (classification IA).
Post	/api/ai/	Classifier un message en type de ticket ATM via Ollama.

Tab. IV.2 : Liste des services web du Sprint 3.

IV.4.1 Cas d'utilisation : « Traitement d'une demande et la convertir en ticket d'intervention »

Dans cette partie, nous proposons une description textuelle accompagnée des diagrammes de séquence système et objet relatifs au cas d'utilisation « Traitement d'une demande et la convertir en ticket d'intervention ».

IV.4.1.1 Description textuelle

Acteurs	Coordinatrice, Manager
Objectif	Permettre à la coordinatrice ou au manager de consulter les demandes clients en attente, de valider une demande en la convertissant en ticket d'intervention officiel, ou de la rejeter avec un motif transmis au client.
Pré-condition	L'utilisateur doit être authentifié. Au moins une demande client doit exister avec le statut `OPENED`.
Scénario principal	1- La coordinatrice reçoit une notification indiquant une nouvelle demande client. 2- Elle accède à la liste des demandes en attente. 3- Le système affiche les demandes avec leurs détails (client, agence, équipement, type d'incident). 4- La coordinatrice sélectionne une demande pour consultation. 5- Le système affiche un formulaire pré-rempli. 6- La coordinatrice valide la demande. 7- Le système convertit le *TicketClient* en ticket d'intervention officiel (*Ticket*) et génère une référence unique. 8- Le système marque la demande comme traitée . 9- Le client est notifié de l'acceptation de sa demande. 10- Le ticket devient disponible pour assignation à un technicien.
Scénarios alternatifs	**A1 : Rejet de la demande** 1- La coordinatrice rejette la demande. 2- Le système demande un motif de rejet. 3- Le motif est enregistré (`msgRejection`). 4- Le statut passe à `REJECTED`. 5- Le client est notifié avec le motif.

Tab. IV.3 : Cas d'utilisation : Compléter ou rejet d'une demande client

IV.4.1.2 Diagramme de séquence système

La figure IV.2 illustre le diagramme de séquence système associé à ce cas d'utilisation, mettant en évidence les interactions entre la coordinatrice et le système TSS lors du traitement d'une demande client.

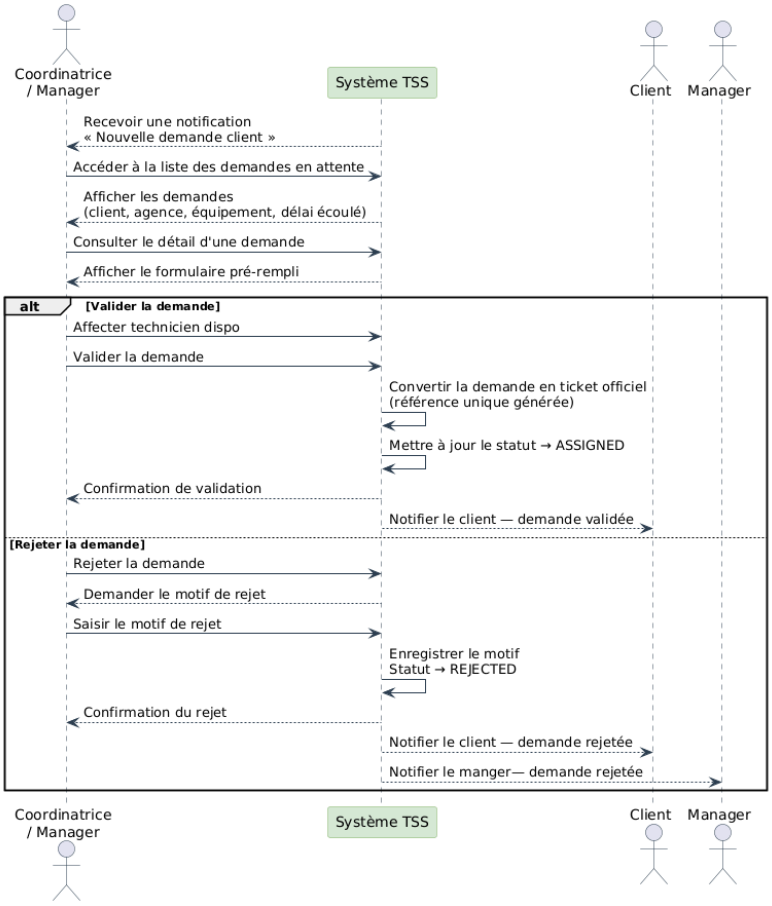


Figure IV.2 : Diagramme de séquence système du « Traitement d'une demande par la Coordinatrice »

IV.4.1.3 Diagramme de séquence objet

La figure IV.3 illustre le diagramme de séquence objet de ce cas d'utilisation, détaillant les interactions entre les composants internes du système lors de la conversion d'une demande client en ticket officiel.

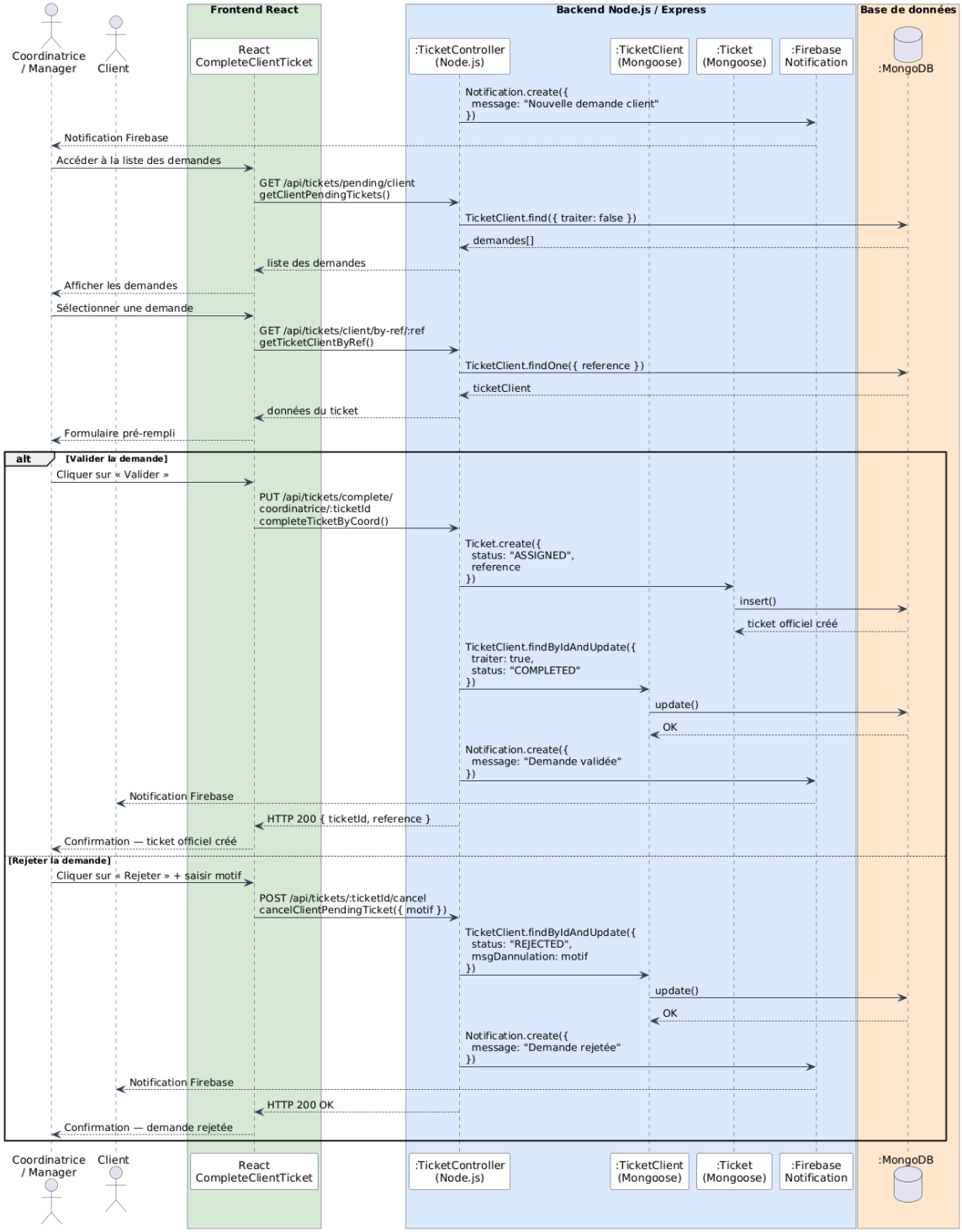


Figure IV.3 : Diagramme de séquence objet « Traitement d'une demande par la Coordinatricen »

IV.4.2 Cas d'utilisation : « Identifier un incident via DocBot »

Avant la mise en œuvre du module DocBot, il convient de préciser que TUNISYS s'appuie sur une documentation technique interne permettant de classifier les différents incidents pouvant affecter les distributeurs automatiques de billets (DAB).

Cette documentation repose sur une arborescence hiérarchique composée de catégories d'incidents, de composants et de sous-composants identifiés à l'aide d'abréviations normalisées.

IV.4.2.1 Classification des incidents

Les incidents sont d'abord répartis selon leur nature :

- **HW (Hardware Issue)** : incidents matériels ;
- **SW (Software Issue)** : incidents logiciels ;
- **Vandalism** : actes de vandalisme ;
- **Misuse** : mauvaise utilisation de l'équipement ;
- **Preventive Maintenance** : maintenance préventive ;
- **Site Conformity Check Visit** : contrôle de conformité ;
- **ATM Powerup Service** : remise en service du DAB.

IV.4.2.2 Présentation d'Ollama et de LLaMA 3

Ollama est une plateforme open source permettant d'exécuter des modèles de langage de grande taille (LLM) localement, sans dépendance à un service cloud.

Elle expose une API HTTP standardisée accessible via `http://localhost:11434`.

Dans le cadre du projet DocBot, Ollama est déployé sur le serveur backend Node.js et exécute le modèle **LLaMA 3** (Meta AI).

IV.4.2.3 Avantages dans le contexte de l'organisme :

- **Exécution locale et confidentialité** : aucune donnée ne quitte l'environnement interne.
- **Déterminisme** : le paramètre `temperature=0` garantit des résultats stables et reproductibles.
- **Sortie structurée** : le format `JSON` est imposé pour faciliter l'intégration backend.
- **Optimisation** : la limite `num_predict=220` réduit le temps de génération.

Ollama n'est sollicité qu'en dernier recours. En effet, si le moteur de scoring local obtient un score supérieur ou égal à 35, la réponse est générée sans appel au modèle IA. Cette approche hybride améliore les performances et garantit une continuité de service.

IV.4.2.4 Architecture logique de DocBot

La figure IV.4 présente l'architecture logique du module DocBot, depuis la saisie utilisateur jusqu'à la classification finale.

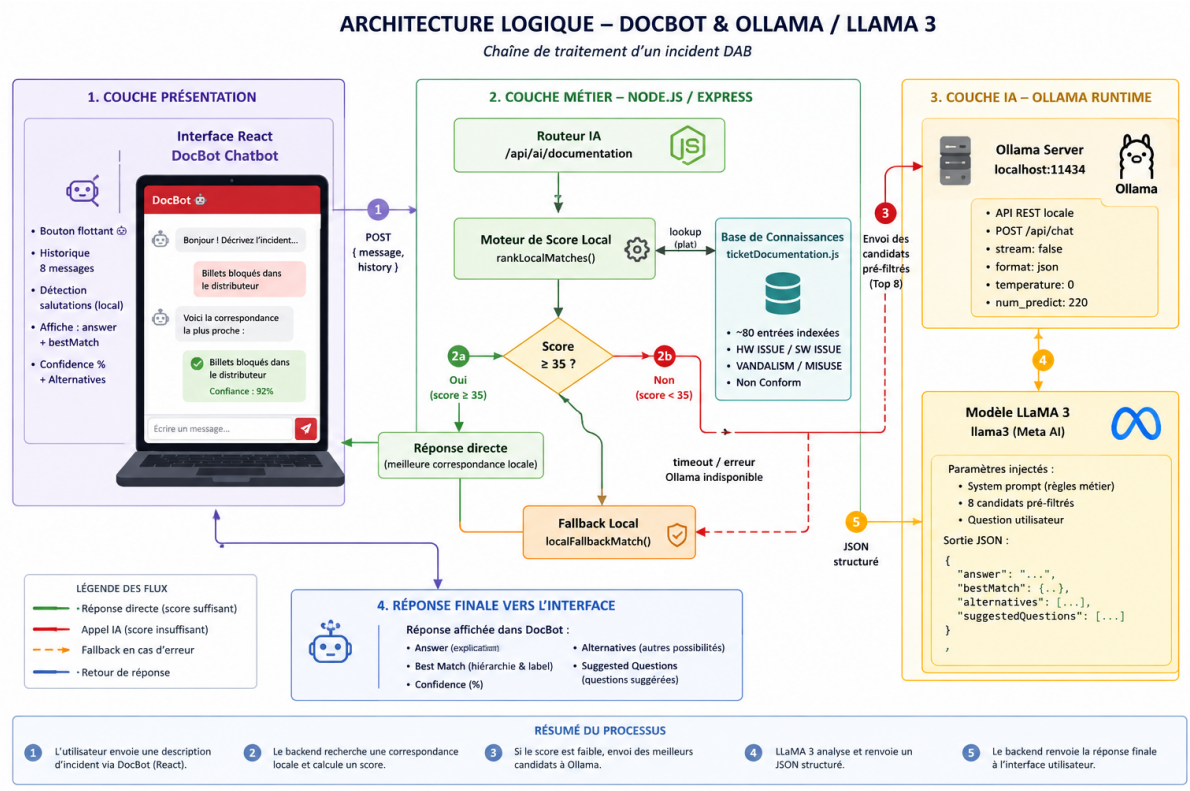


Figure IV.4 : Architecture logique du module DocBot basé sur Ollama

IV.4.2.5 Processus de traitement

Le traitement d'une requête suit les étapes suivantes :

- L'utilisateur saisit une description dans l'interface React.
- La requête est envoyée au backend via HTTP POST.
- Le backend calcule un score de similarité avec la base de connaissances.
- Si le score est ≥ 35, une classification locale est retournée.
- Sinon, la requête est envoyée à Ollama (LLaMA 3).
- Le modèle génère une réponse JSON structurée.
- En cas d'échec d'Ollama, un mécanisme de fallback local est activé.

IV.4.2.6 Description textuelle

Acteurs	Client, Système IA (DocBot)
Objectif	Permettre au client de décrire un incident DAB en langage naturel et d'obtenir une classification structurée (catégorie, composant, recommandation).
Préconditions	Le client est authentifié et le service IA est disponible (`/api/ai/documentation`).
Scénario nominal	1. Ouverture de DocBot depuis le formulaire de ticket. 2. Affichage d'un message d'accueil et d'exemples. 3. Saisie de l'incident par l'utilisateur. 4. Calcul du score de similarité. 5. Classification locale si score suffisant. 6. Sinon appel à Ollama (LLaMA 3). 7. Retour d'une classification hiérarchique. 8. Validation par le client. 9. Pré-remplissage du formulaire de ticket.
Scénarios alternatifs	**A1 : Incident non reconnu** - Reformulation demandée. **A2 : Service IA indisponible** - Basculement vers classification locale.

Tab. IV.4 : Cas d'utilisation : Identifier un incident via DocBot

IV.4.2.7 Diagramme de séquence système

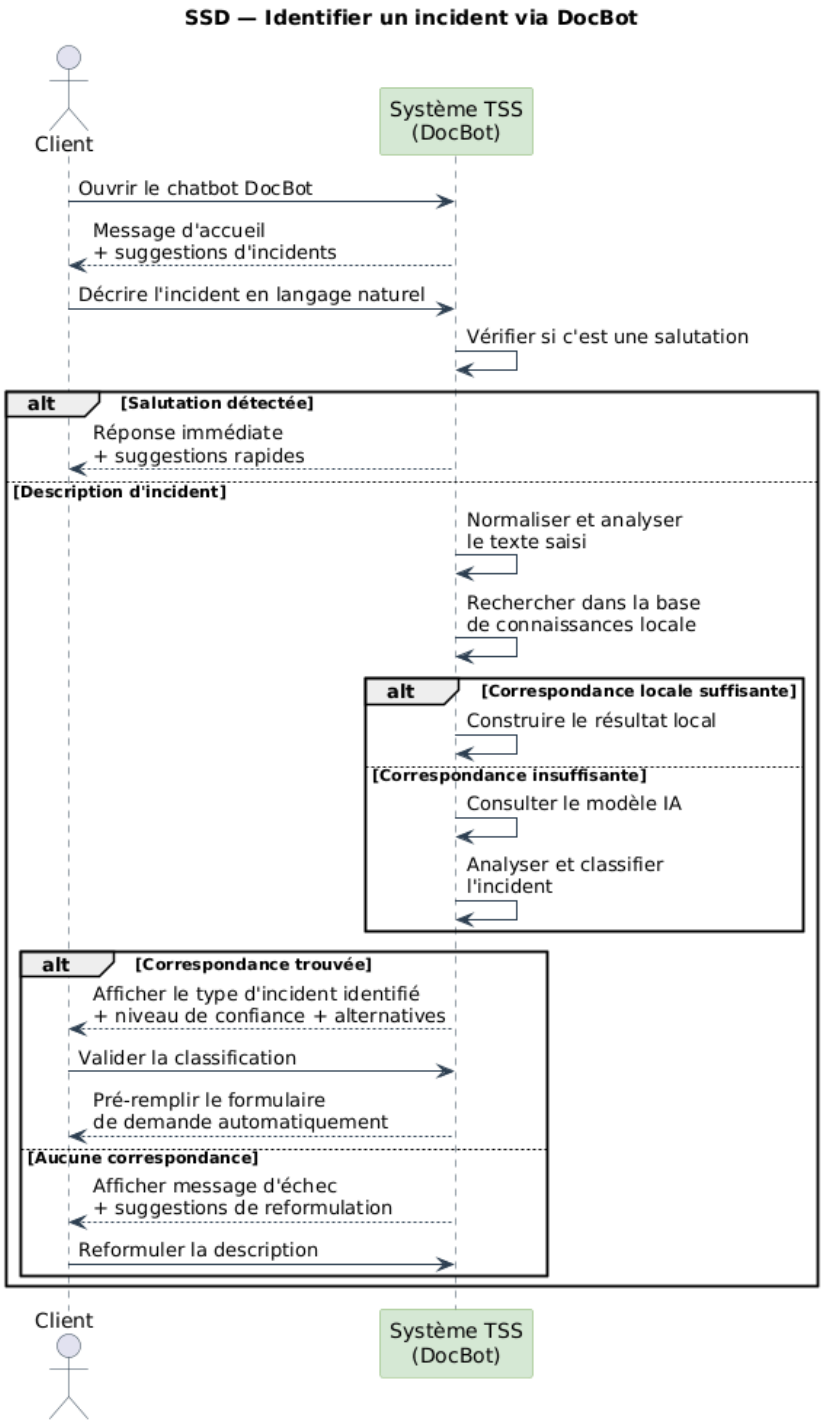


Figure IV.5 : Diagramme de séquence système - DocBot

IV.4.2.8 Diagramme de séquence objet

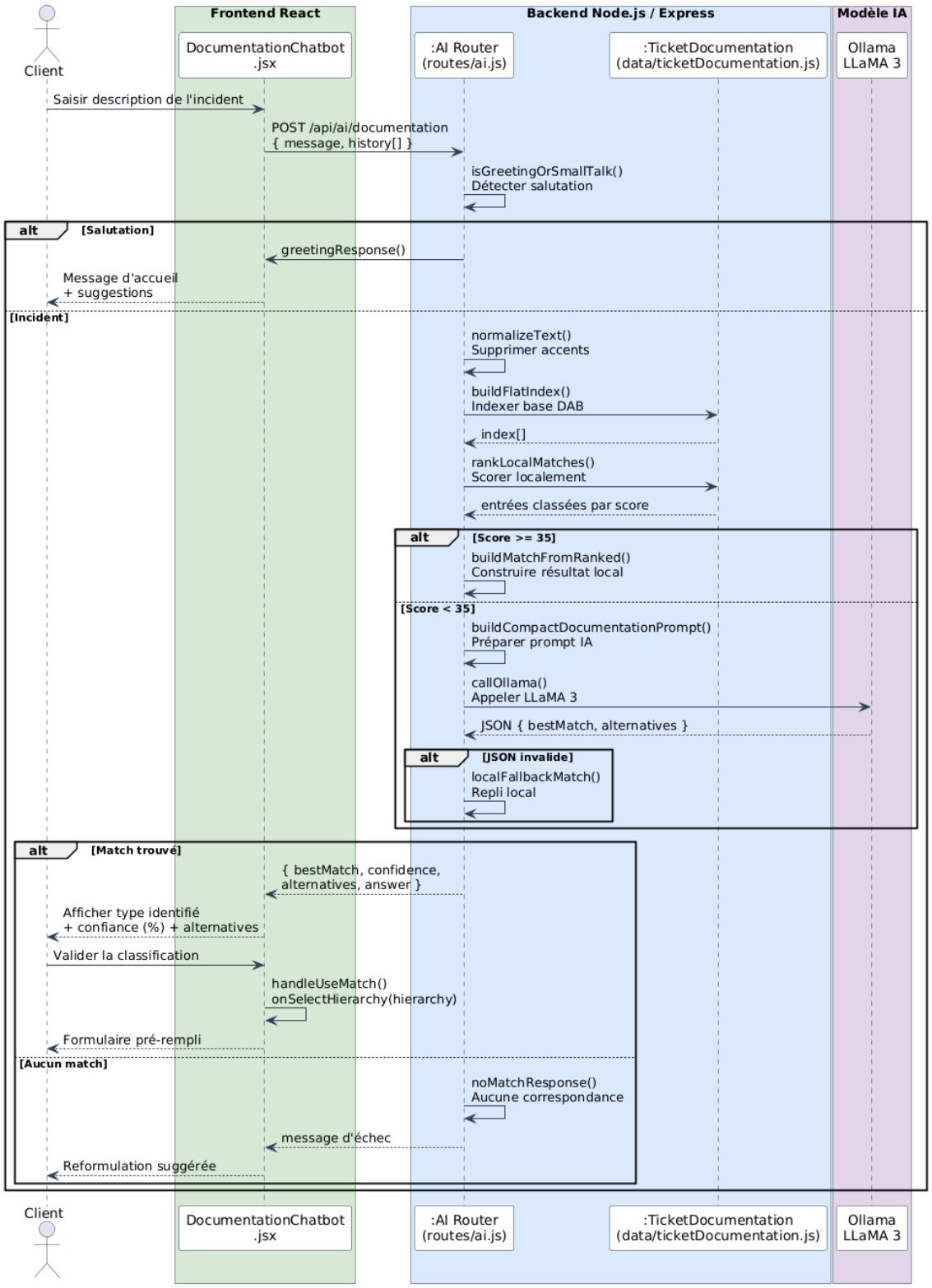


Figure IV.6 : Diagramme de séquence objet - DocBot

IV.4.2.9 Diagramme d'activités

Le diagramme suivant illustre le processus global de traitement d'un incident via DocBot.

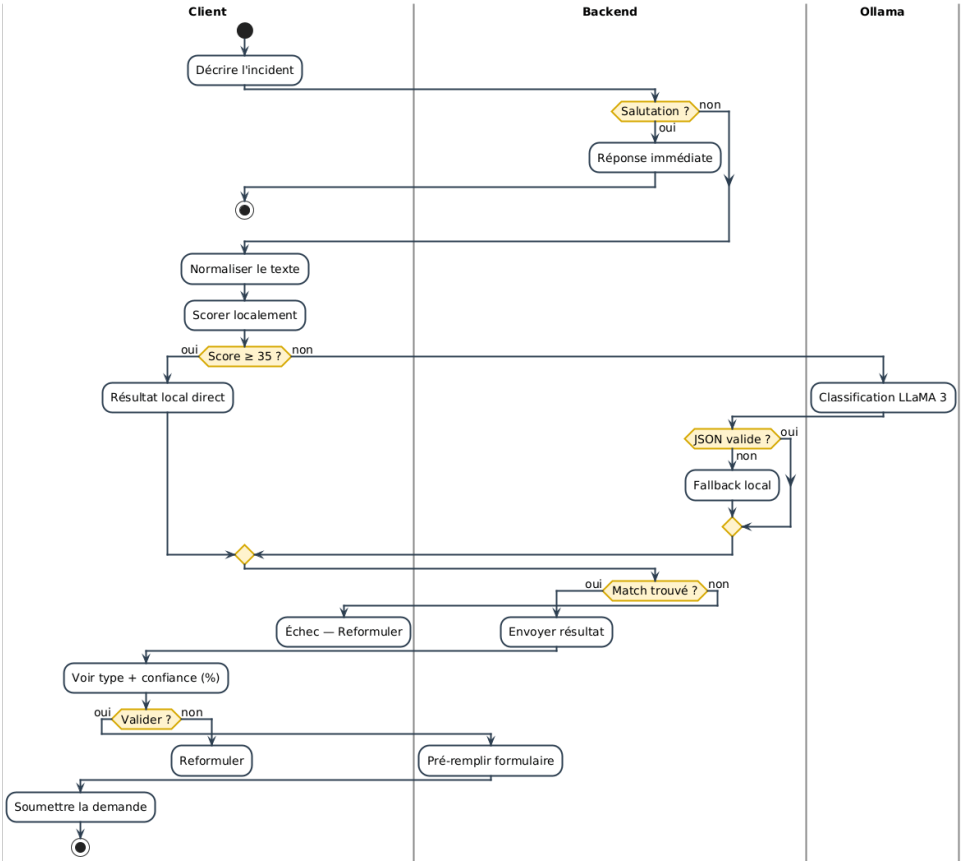


Figure IV.7 : Diagramme d'activités - Identification d'un incident via DocBot

IV.5 Réalisation du Sprint 3

- **Formulaire de soumission d'une demande client :**

La figure IV.8 illustre l'interface permettant au client de soumettre une demande d'intervention depuis la plateforme TSS. Il sélectionne l'agence, l'équipement , puis décrit l'incident. Une fois soumise, la coordinatrice est immédiatement notifiée et le chronomètre SLA démarre automatiquement.

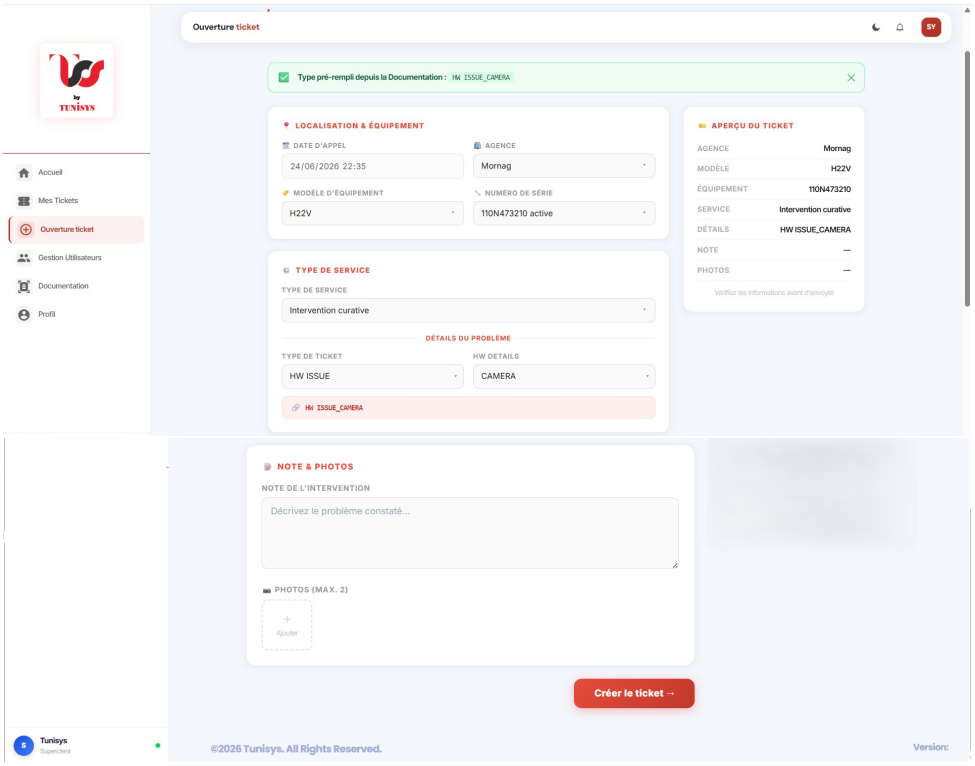


Figure IV.8 : Formulaire de soumission d'une demande client

- **Suivi des demandes — Espace client :**

La figure IV.11 présente l'interface de suivi des demandes accessible au client. Il peut y consulter l'ensemble de ses demandes avec leur statut en temps réel, la référence attribuée après validation, ainsi que le motif de rejet lorsque la demande n'a pas été retenue.

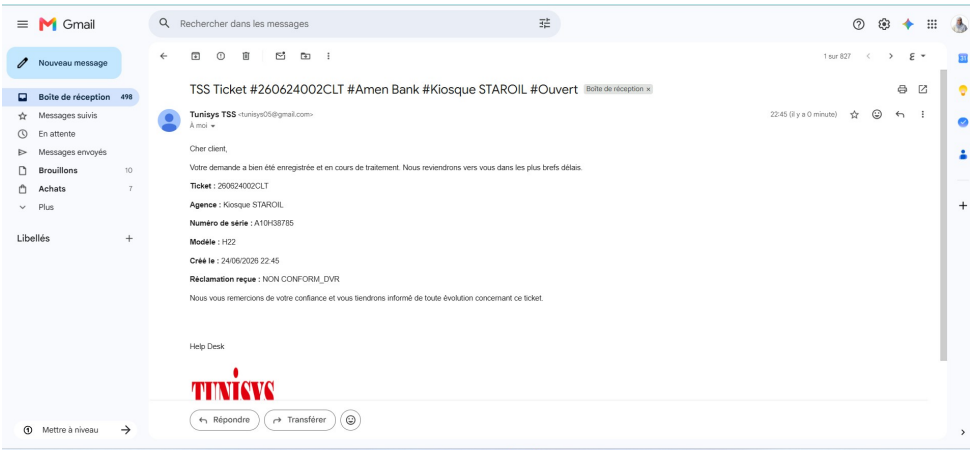


Figure IV.9 : email d'ouverture ticket client

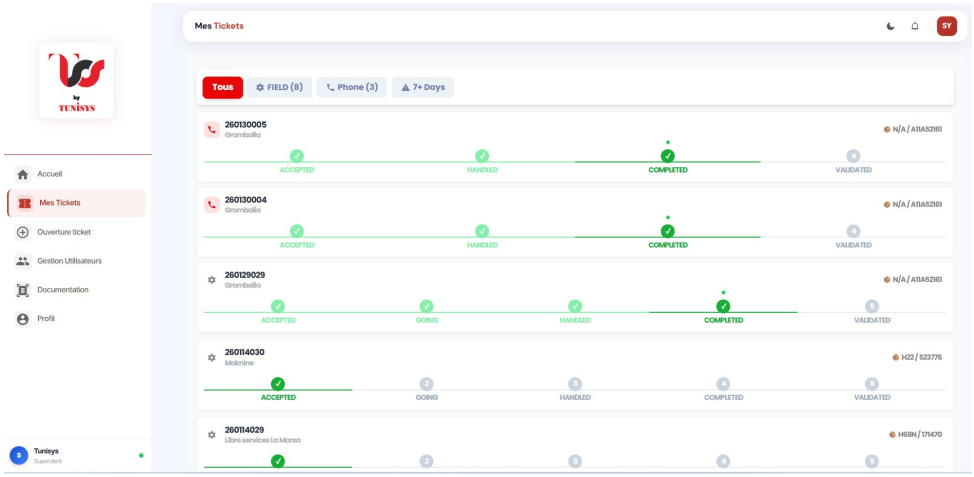


Figure IV.10 : Suivi des demandes — Espace client

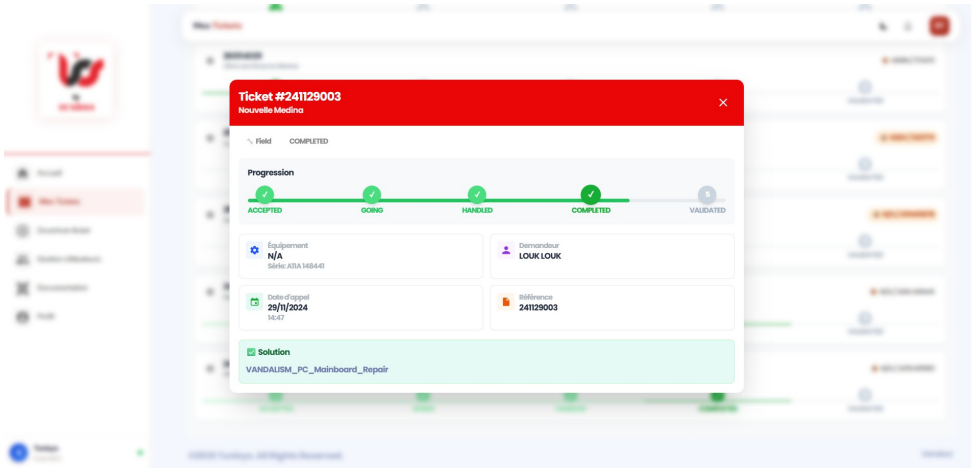


Figure IV.11 : détails des demandes — Espace client

- **Gestion des demandes — Coordinatrice :**

La figure IV.12 illustre l'interface de gestion des demandes clients côté coordinatrice. Après réception de la notification, elle consulte la liste des demandes en attente avec les informations clés (client, agence, équipement, délai écoulé) et peut valider ou rejeter chaque demande directement depuis cette vue.

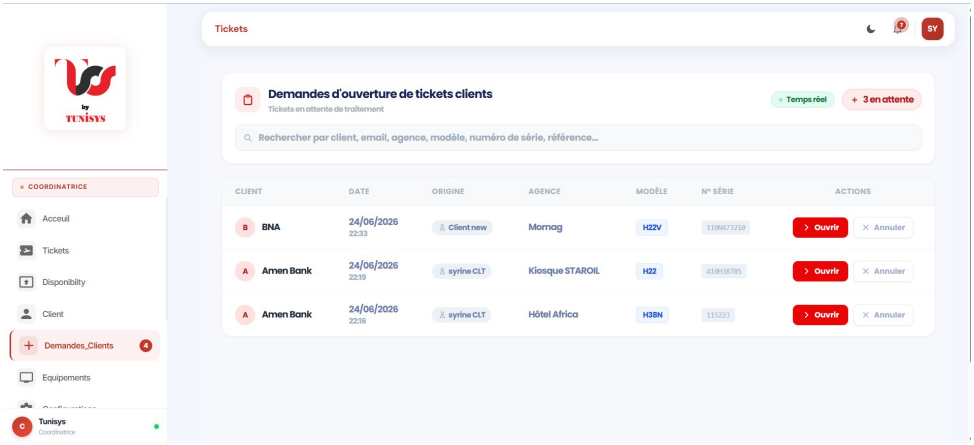


Figure IV.12 : Gestion des demandes — Coordinatrice

La figure IV.13 présente l'interface de validation d'une demande client. La coordinatrice complète les informations nécessaires à la création du ticket officiel (type d'intervention, HelpDesk assigné) et confirme la validation. Le système convertit automatiquement la demande en ticket d'intervention et notifie le client.

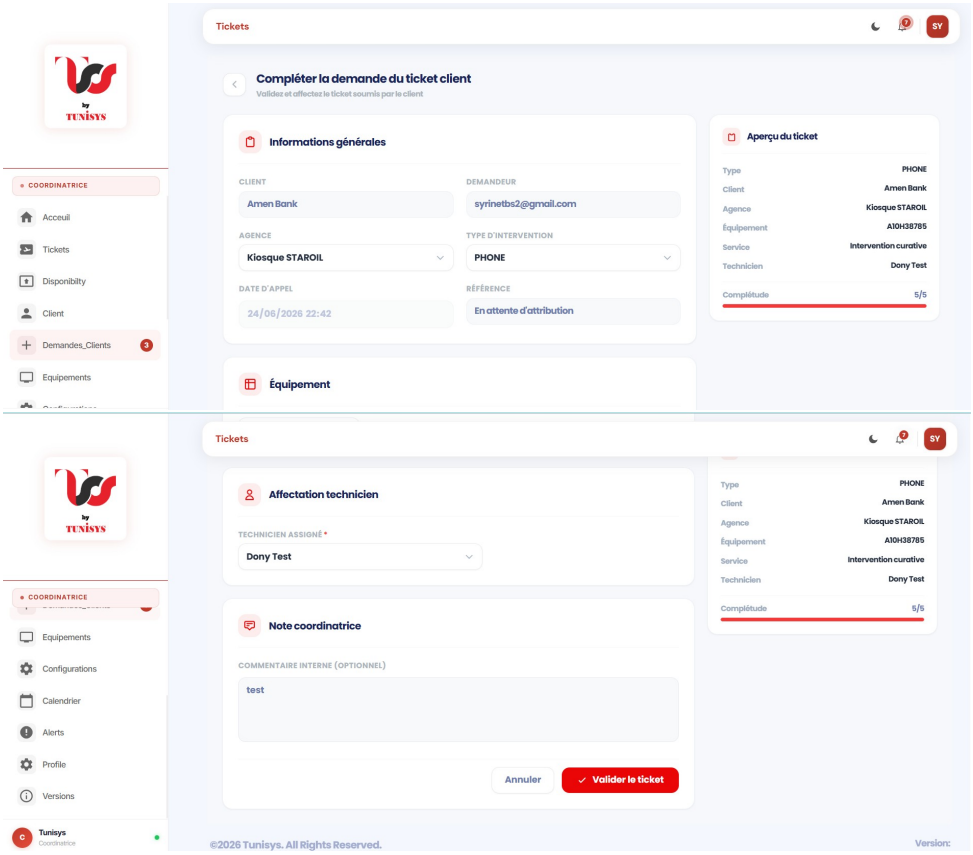


Figure IV.13 : Validation d'une demande et conversion en ticket d'intervention

La figure IV.17 illustre l'interface accessible au manager pour superviser les demandes non traitées. Lorsqu'une demande dépasse le délai SLA de 30 minutes sans être prise en charge, le manager reçoit une notification Firebase et peut consulter la liste des demandes en attente, les modifier ou les rejeter en précisant un motif.

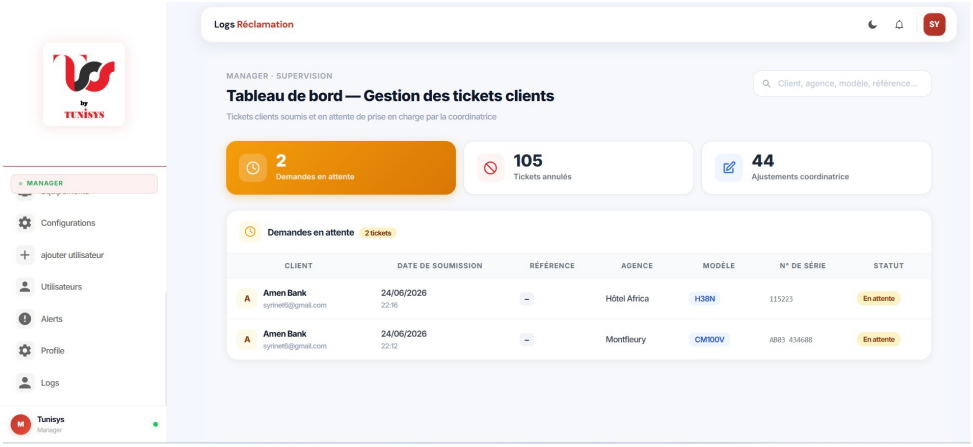


Figure IV.14 : Gestion des demandes non traitées — Manager

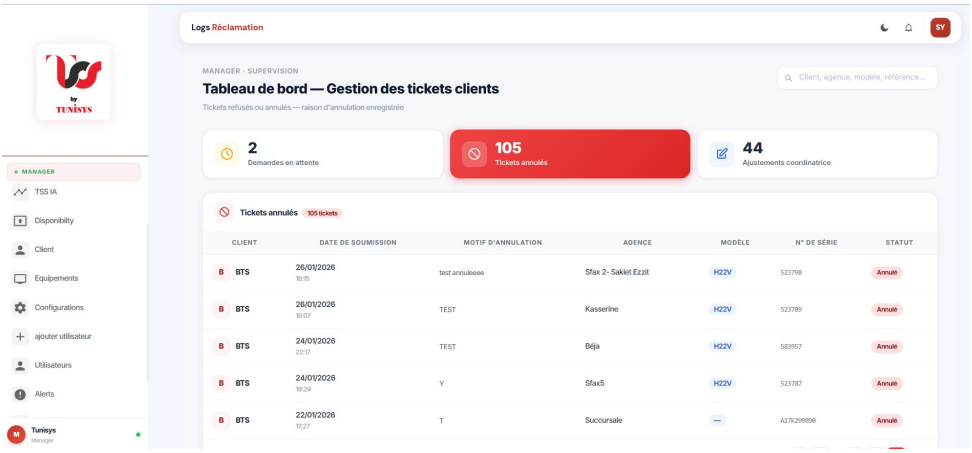


Figure IV.15 : Gestion des demandes non traitées — Manager

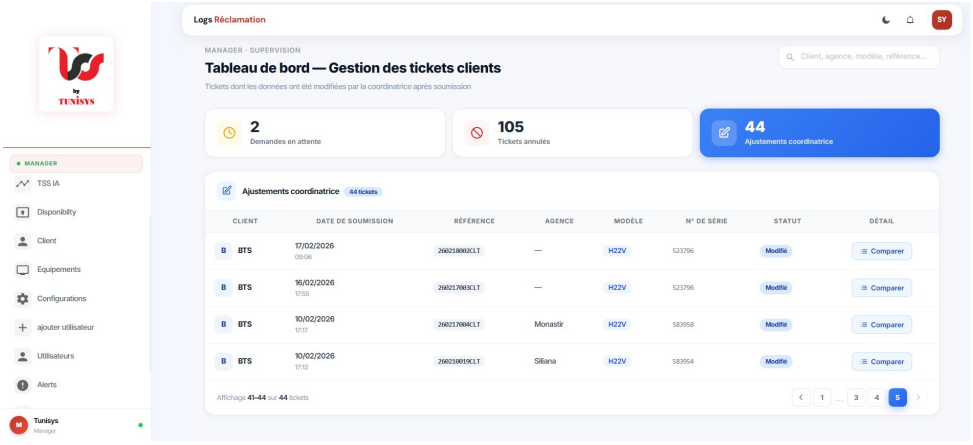


Figure IV.16 : Gestion des demandes non traitées — Manager

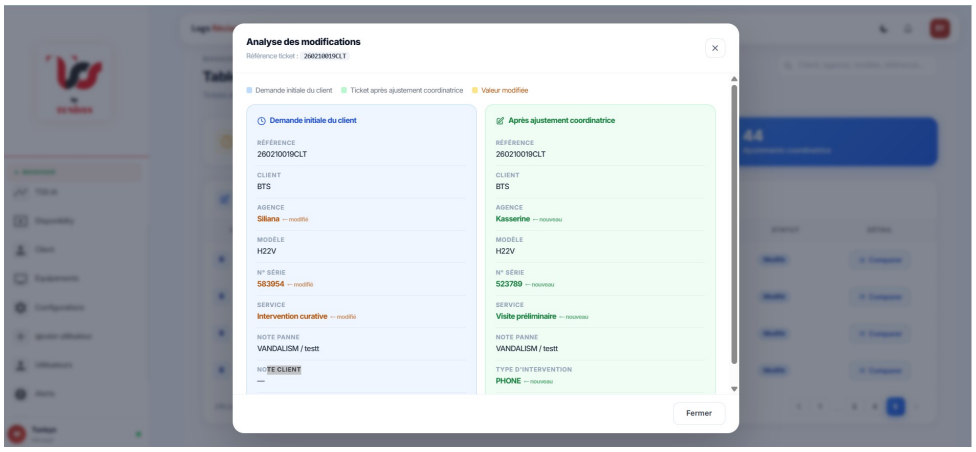


Figure IV.17 : Gestion des demandes Modifiées — Manager

- **Chatbot IA DocBot :**

La figure IV.18 illustre l'interface du chatbot IA DocBot. Le client décrit son incident en langage naturel et DocBot identifie automatiquement le type de ticket correspondant dans la hiérarchie Tunisys (exemple : *HW ISSUE › CDM › Presenter* pour un problème de billets bloqués). Le résultat est directement utilisé pour pré-remplir le formulaire de demande et garantir une classification précise dès la soumission.

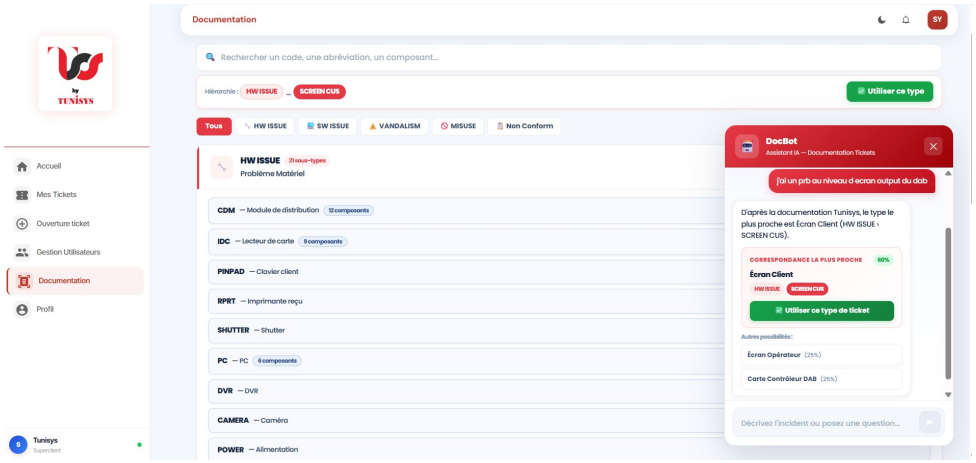


Figure IV.18 : Interface du chatbot IA DocBot

IV.6 Conclusion

Ce chapitre a présenté les fonctionnalités développées lors du Sprint 3 de la plateforme TSS. Ce sprint a permis la mise en place du module de demande client, permettant aux clients de soumettre leurs demandes d'intervention directement depuis la plateforme.

L'intégration de DocBot, basé sur Ollama/LLaMA 3 et une base de connaissances dédiée aux incidents DAB, facilite l'identification automatique des incidents et améliore la qualification des demandes.

Chapitre V

Release 2 — Sprint 4 : Gestion Automatique des Rapports

V.1 Introduction

Ce chapitre présente le quatrième sprint de la troisième release du projet. Il porte sur deux fonctionnalités complémentaires conçues pour renforcer le suivi qualité des interventions curatives.

Contexte et problématique :

Lorsqu'un équipement tombe en panne et qu'une intervention curative est réalisée, il est attendu que la résolution soit définitive. Si un nouveau ticket est ouvert sur le **même équipement** dans un intervalle inférieur à **7 jours**, cela révèle un manque de performance de la part du technicien ayant réalisé l'intervention précédente (ticket $n-1$) : la panne n'a pas été correctement résolue.

Fonctionnalités implémentées :

- **Détection automatique de récurrence** : au moment de la création d'un ticket, le système vérifie si le même équipement a fait l'objet d'un ticket curatif résolu dans les 7 derniers jours. Si c'est le cas, un *rapport de récurrence* est automatiquement généré.
- **Mise à jour du nombre de chances du technicien** : le technicien ayant traité le ticket $n-1$ reçoit une pénalité (`nbr_chance += 1`), reflétant le manque d'efficacité de son intervention.
- **Alertes et notifications manager** : à chaque étape (détection de récurrence, génération du rapport), le manager reçoit automatiquement une notification en temps réel, un e-mail et un rapport PDF en pièce jointe.

V.2 Backlog du Sprint 4

Après la définition des objectifs du Sprint 4, les User Stories sélectionnées ont été décomposées en tâches techniques afin de faciliter leur implémentation et leur suivi.

Le tableau V.1 présente le backlog détaillé du Sprint 4.

User Story	Tâches	Estimation
En tant que système, lors de la création d'un ticket curatif, je veux vérifier automatiquement si le même équipement a déjà eu un ticket résolu dans les 7 derniers jours et générer un rapport de récurrence.	Ajouter la vérification post-création dans le `TicketController`.	1
	Requêter les tickets résolus sur le même équipement dans les 7 derniers jours.	2
	Concevoir le modèle `Rapport` (équipement, tickets, technicien, date).	1
	Développer l'API de création automatique du rapport.	2
	Lier le rapport au ticket courant et au ticket $n-1$.	1
	Développer l'interface React de visualisation du rapport.	2
	Tester la détection et la génération automatique.	1
En tant que système, je veux incrémenter le compteur `nbr_chance` du technicien ayant traité le ticket $n-1$, afin de tracer les manques de performance sur intervention curative.	Ajouter le champ `nbr_chance` au modèle `Technicien`.	1
	Développer la logique d'incrémentation automatique (`nbr_chance += 1`).	1
	Historiser chaque pénalité (date, équipement, ticket concerné).	2
	Afficher le `nbr_chance` dans le profil et le tableau de bord technicien.	1
	Déclencher une alerte si `nbr_chance` dépasse un seuil configuré.	1
	Tester les mécanismes d'incrémentation et d'historique.	1
En tant que manager, je veux recevoir une notification en temps réel, un e-mail et un rapport PDF	Développer le service de notification en temps réel (WebSocket / SSE).	2
	Générer le rapport PDF automatiquement .	2
	Développer le service d'envoi d'e-mail avec PDF en pièce jointe.	1

User Story	Tâches	Estimation
	Intégrer les déclencheurs dans le `TicketController`.	1
	Développer la bannière d'alerte en temps réel dans l'interface manager.	1
	Tester le pipeline complet : création → PDF → e-mail → notification.	1
En tant que manager, je veux consulter le tableau de bord des rapports de récurrence et suivre l'évolution du `nbr_chance` de chaque technicien.	Développer l'API de liste des rapports de récurrence.	1
	Développer l'API de consultation du `nbr_chance` par technicien.	1
	Concevoir la vue tableau de bord (rapports + classement techniciens).	2
	Ajouter les filtres (période, équipement, client, technicien).	1
	Tester les fonctionnalités.	1

Tab. V.1 : Backlog du Sprint 4

V.3 Diagramme de cas d'utilisation du Sprint 4

La figure V.1 illustre les cas d'utilisation identifiés pour ce sprint. La coordinatrice déclenche le processus en créant un ticket ; le système prend en charge la détection de récurrence, la génération du rapport et la notification du manager.

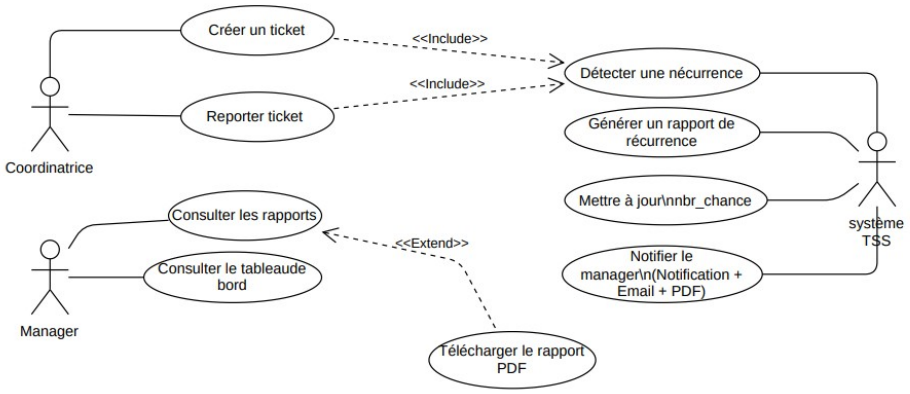


Figure V.1 : Diagramme de cas d'utilisation du Sprint 4

V.4 Services Web

Les services Web fournissent des services via Internet. Ils constituent une interface permettant à deux applications de communiquer. Les services ci-dessous sont résumés dans le tableau V.2.

Méthode	URL	Description
Tickets et détection de récurrence		
Post	/api/ticketclient/add	Créer un ticket client et déclencher la vérification de récurrence.
Get	/api/ticketclient/ :id	Consulter le détail d'un ticket client.
Get	/api/ticketclient/recurrence/ :equipementId	Vérifier si l'équipement a eu un ticket résolu dans les 7 derniers jours.
Rapports de récurrence		
Get	/api/rapport/list	Lister tous les rapports de récurrence.
Get	/api/rapport/ :id	Consulter le détail d'un rapport.
Get	/api/rapport/export/ :id	Télécharger le rapport en PDF.
Performance technicien		
Get	/api/technicien/chances/ :id	Consulter le `nbr_chance` d'un technicien.
Get	/api/technicien/chances/all	Classement de tous les techniciens par `nbr_chance`.

Tab. V.2 : Liste des services web du Sprint 4.

V.5 Les cas d'utilisation du Sprint 4

V.5.1 Cas d'utilisation : « Détecter une récurrence et générer un rapport automatique »

V.5.1.1 Description textuelle

Acteurs	Coordinatrice (déclencheur), Système (traitement automatique), Manager (destinataire)
Objectif	Détecter automatiquement, lors de la création d'un ticket curatif, si le même équipement a déjà subi une intervention dans les 7 derniers jours, puis générer un rapport, pénaliser le technicien et alerter le manager.
Pré-condition	La coordinatrice est authentifiée. L'équipement concerné existe dans la base de données. Le ticket est de type **intervention curative +Field**.
Scénario principal	1- La coordinatrice crée un nouveau ticket pour un équipement client. 2- Le système enregistre le ticket en base de données. 3- Le système vérifie : existe-t-il un ticket curatif **résolu** sur le même équipement dans les **7 derniers jours** ? 4- Une récurrence est détectée (ticket $n-1$ trouvé). 5- Le système génère automatiquement un **rapport de récurrence** (équipement, client, ticket n, ticket $n-1$, technicien, intervalle). 6- Le système incrémente `nbr_chance` du technicien ayant traité le ticket $n-1$ (`nbr_chance += 1`). 7- Le système envoie au manager : une **notification en temps réel**, un **e-mail** et un **PDF** du rapport en pièce jointe.
Scénario alternatif	**Aucune récurrence :** Aucun ticket résolu trouvé sur cet équipement dans les 7 derniers jours → le ticket est créé normalement, aucun rapport n'est généré. **Plusieurs récurrences :** Plusieurs tickets précédents trouvés → le système prend le plus récent comme ticket $n-1$ de référence.

Tab. V.3 : Cas d'utilisation : Détecter une récurrence et générer un rapport

V.5.1.2 Diagramme de séquence système

La figure V.2 illustre les interactions entre la coordinatrice, le système et le manager lors de la détection d'une récurrence.

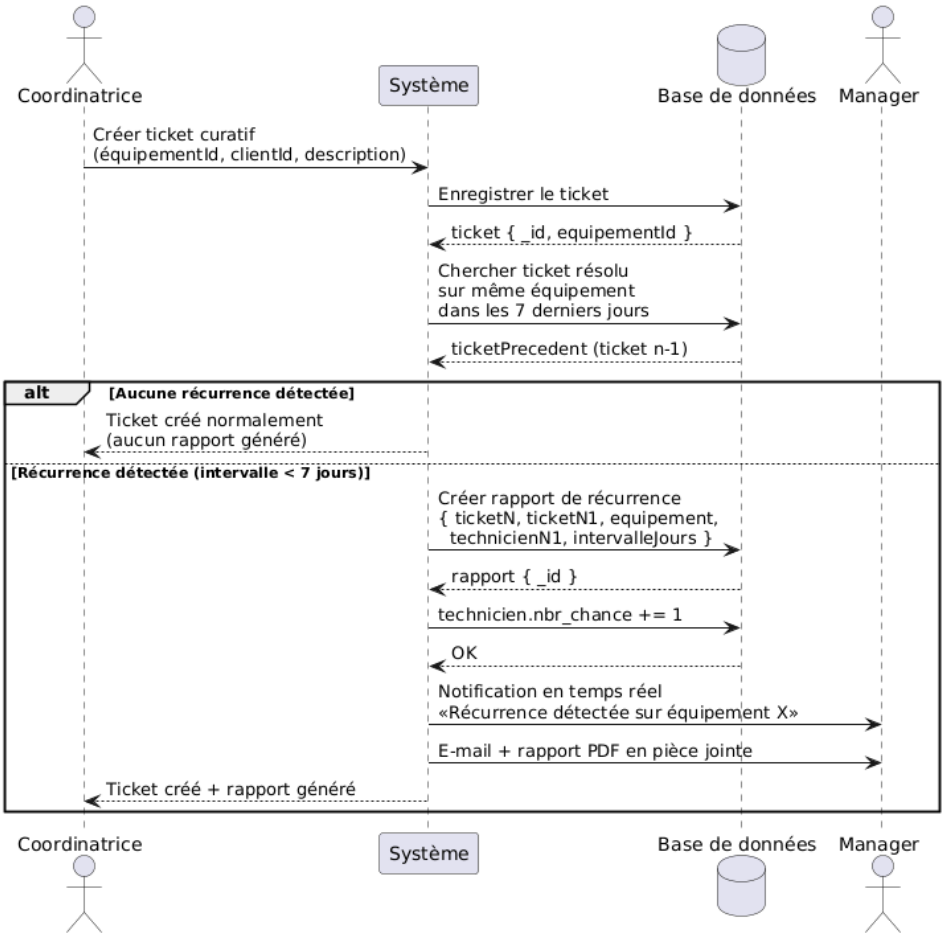


Figure V.2 : Diagramme de séquence système — Détecter une récurrence et générer un rapport

V.5.1.3 Diagramme de séquence objet

La figure V.3 illustre les interactions entre les composants internes : le contrôleur de tickets, le service de rapport, le service technicien et le service de notification.

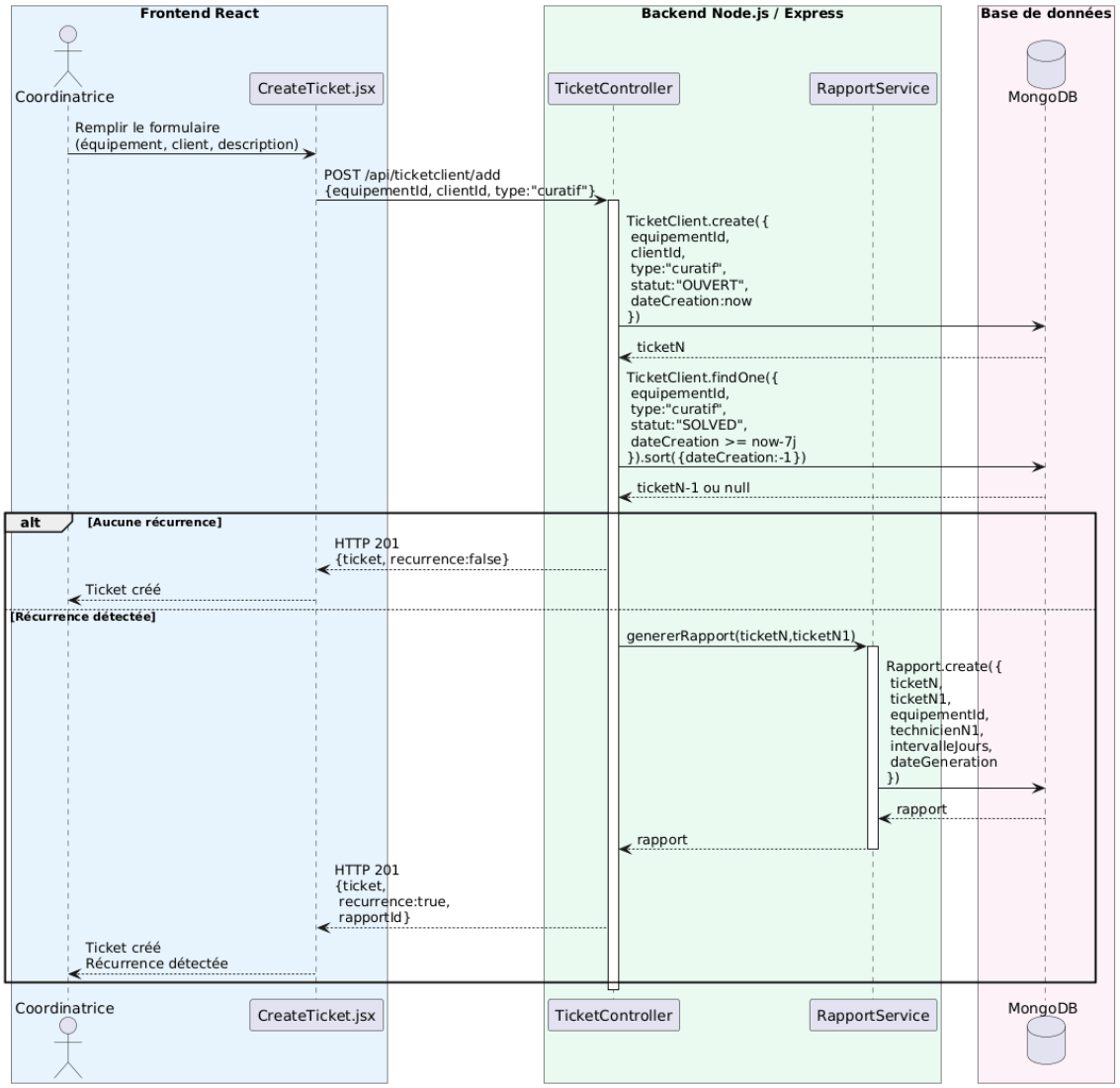


Figure V.3 : Diagramme de séquence objet — Détecter une récurrence et générer un rapport

V.5.2 Cas d'utilisation : « Consulter le tableau de bord des rapports et du `nbr_chance` »

V.5.2.1 Description textuelle

Acteurs	Manager (principal)
Objectif	Permettre au manager de consulter la liste des rapports de récurrence générés, de visualiser les données de chaque équipement concerné et de suivre l'évolution du `nbr_chance` de chaque technicien.
Pré-condition	Le manager est authentifié. Au moins un rapport de récurrence a été généré.

Scénario principal	1- Le manager accède au tableau de bord des rapports de récurrence. 2- Il visualise la liste des rapports : équipement, client, date, technicien $n-1$, intervalle entre les deux tickets. 3- Il clique sur un rapport pour consulter le détail : ticket n, ticket $n-1$, fiche équipement, historique des interventions. 4- Il consulte le classement des techniciens par `nbr_chance` (du plus pénalisé au moins pénalisé). 5- Il peut filtrer par période, client, équipement ou technicien. 6- Il peut télécharger un rapport individuel en PDF.
Scénario alternatif	**Aucun rapport disponible :** Aucune récurrence n'a été détectée sur la période → le tableau affiche « Aucun rapport pour cette période ».

Tab. V.4 : Cas d'utilisation : Consulter le tableau de bord des rapports

V.5.2.2 Diagramme de séquence système

La figure V.4 illustre les interactions entre le manager et le système lors de la consultation du tableau de bord.

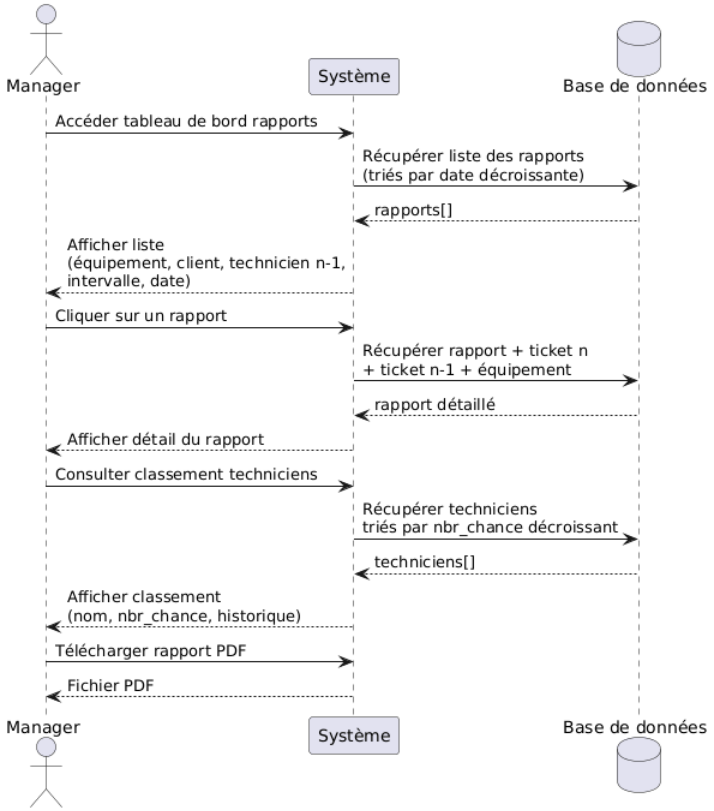


Figure V.4 : Diagramme de séquence système — Consulter le tableau de bord des rapports

V.5.2.3 Diagramme de séquence objet

La figure V.5 illustre les interactions entre les composants internes lors de la consultation du tableau de bord manager.

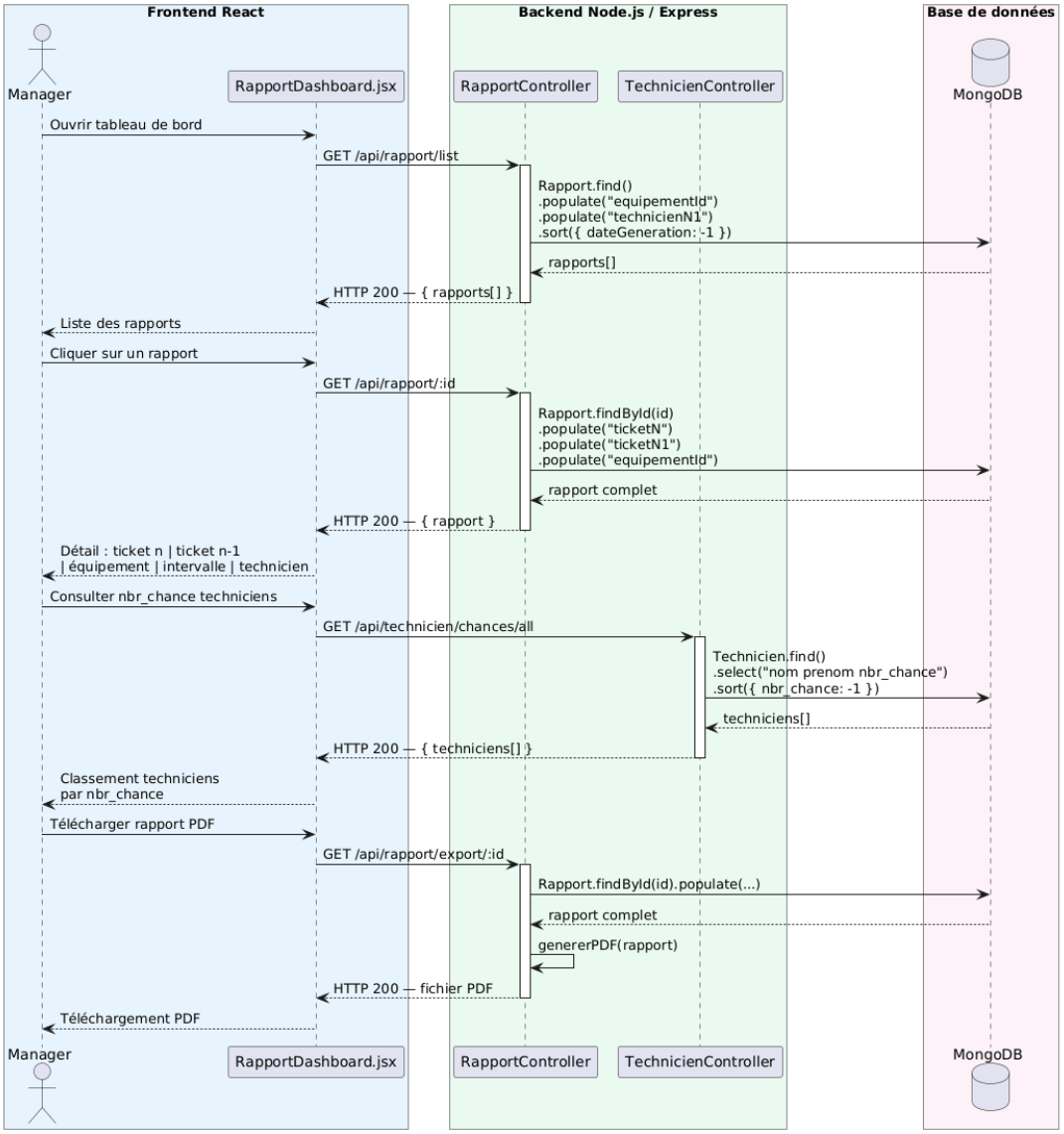


Figure V.5 : Diagramme de séquence objet — Consulter le tableau de bord des rapports

V.6 Diagramme d'activité

La figure V.6 modélise le flux d'activité complet depuis la création du ticket jusqu'à la réception de l'alerte par le manager.

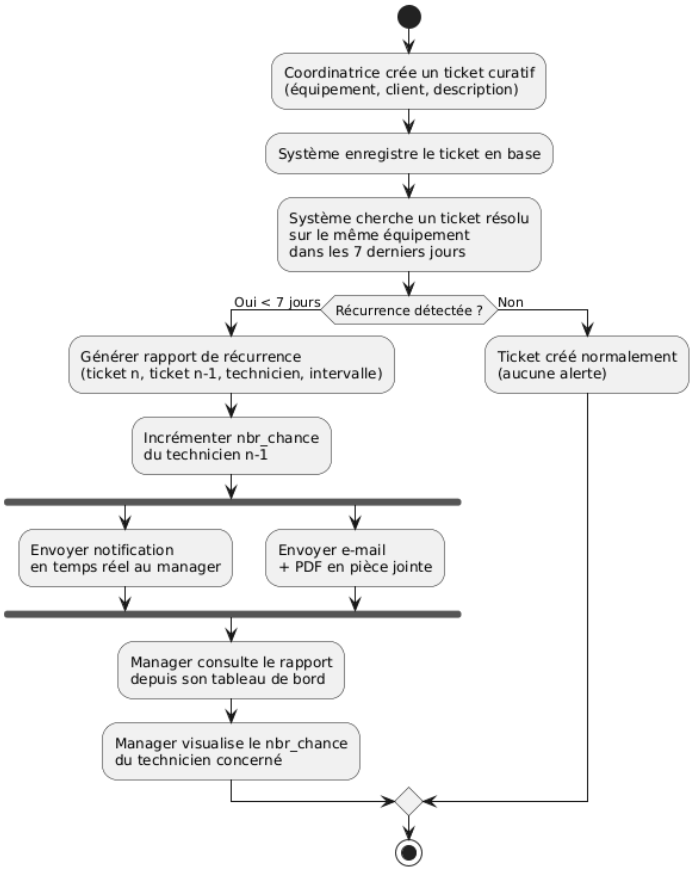


Figure V.6 : Diagramme d'activité — Sprint 4 : Détection de récurrence et alertes

V.7 Réalisation du Sprint 4

- **Création d'un ticket — détection automatique de récurrence :**

La figure V.7 illustre l'interface de création d'un ticket curatif par la coordinatrice. Dès que le formulaire est soumis, le système vérifie en arrière-plan si le même équipement a déjà fait l'objet d'un ticket résolu dans les 7 derniers jours. En cas de récurrence détectée, un message d'avertissement s'affiche immédiatement et le rapport est généré automatiquement.

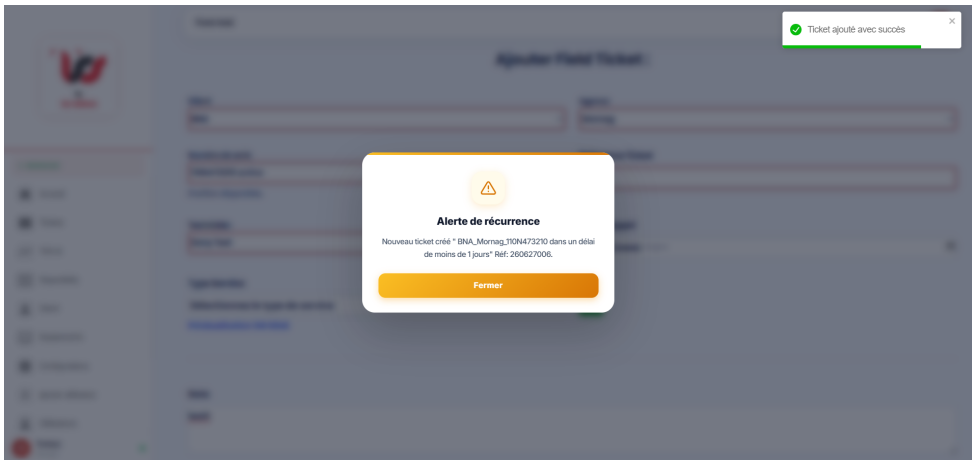


Figure V.7 : Création d'un ticket curatif et détection automatique de récurrence

- **Rapport de récurrence généré automatiquement :**

La figure V.8 présente le rapport de récurrence généré automatiquement lors de la détection. Il regroupe les informations des deux tickets (ticket n et ticket $n-1$), la fiche de l'équipement concerné, l'identité du technicien ayant réalisé l'intervention précédente et l'intervalle exact entre les deux pannes.

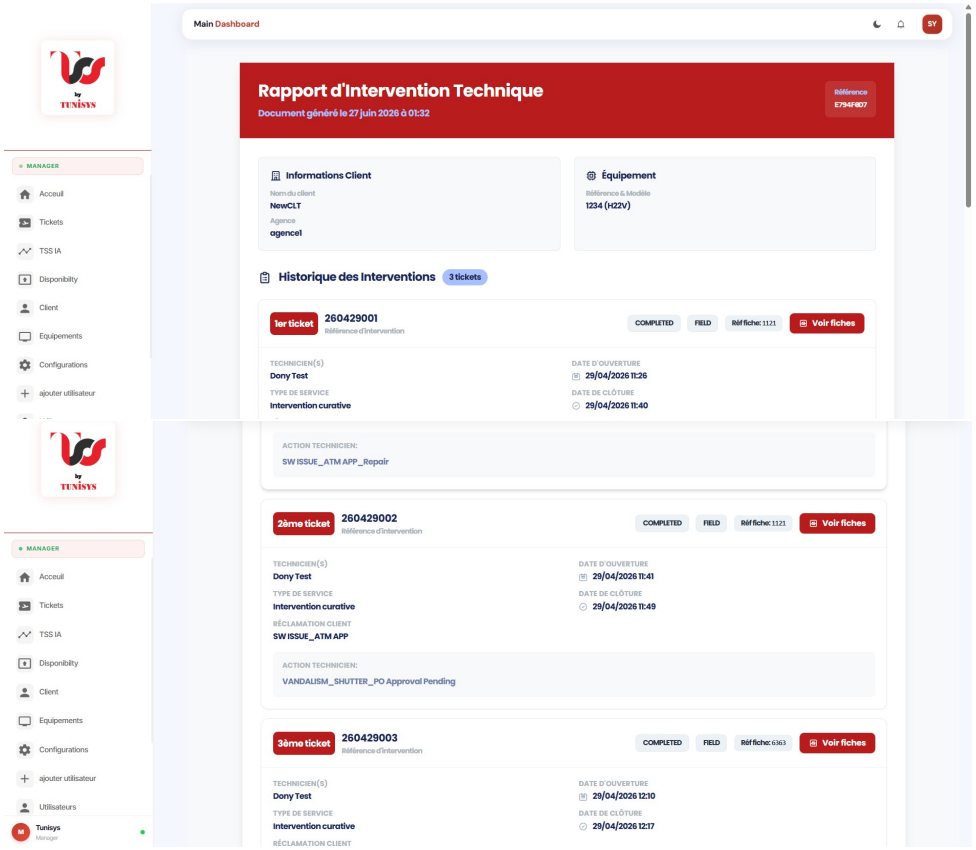


Figure V.8 : Rapport de récurrence — Détail automatique

- **Notification e-mail manager avec rapport PDF :**

La figure V.9 illustre l'e-mail reçu automatiquement par le manager dès qu'une récurrence est détectée. Il contient le résumé de la situation (équipement, client, intervalle) et le rapport complet en pièce jointe au format PDF, permettant au manager d'être informé immédiatement sans avoir à se connecter à la plateforme.

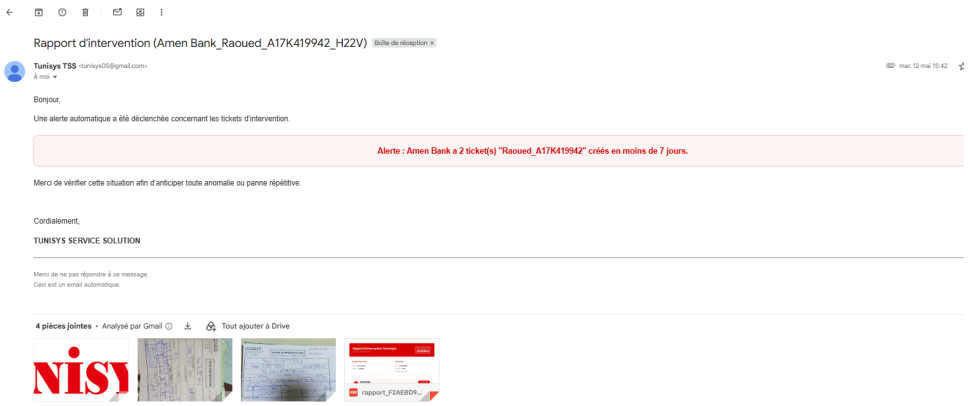


Figure V.9 : E-mail de notification envoyé au manager avec rapport PDF

- **Tableau de bord des rapports de récurrence — Manager :**

La figure V.10 présente le tableau de bord accessible au manager pour consulter l'ensemble des rapports de récurrence générés. Chaque ligne affiche l'équipement concerné, le client, l'intervalle entre les deux pannes et le technicien pénalisé. Le manager peut cliquer sur un rapport pour en consulter le détail ou télécharger le PDF.

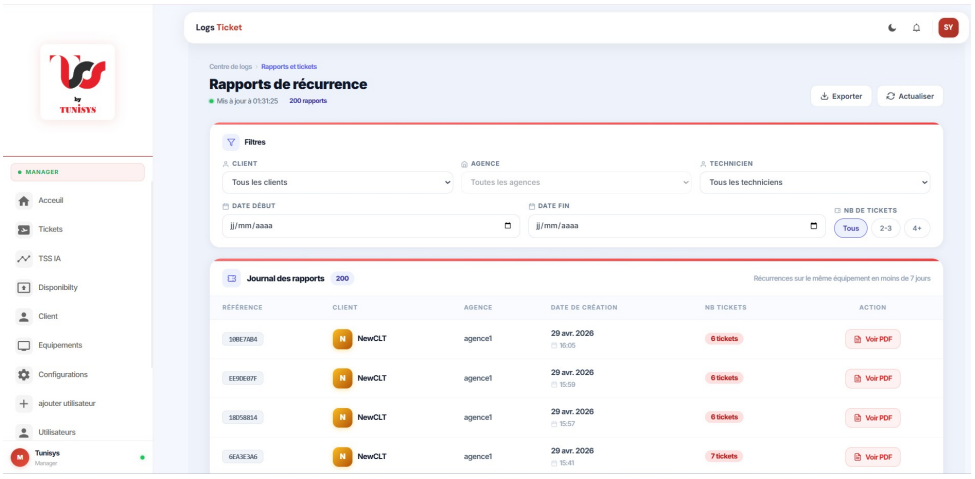


Figure V.10 : Tableau de bord des rapports de récurrence — Manager

- **Suivi du `nbr_chance` des techniciens :**

La figure V.11 illustre l'interface de suivi du compteur de chances des techniciens. Le manager visualise en un coup d'œil le classement des techniciens par nombre de

pénalités reçues, avec pour chacun l'historique des récurrences qui ont déclenché l'incrémentation. Cette vue permet d'identifier rapidement les techniciens nécessitant un accompagnement ou une formation complémentaire.

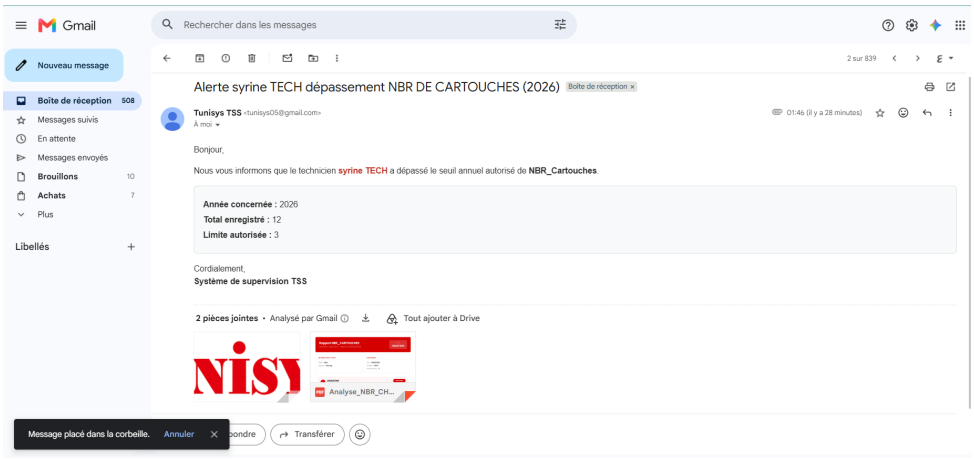


Figure V.11 : Suivi du `nbr_chance` des techniciens — Manager

- **Notification en temps réel — Interface manager (mobile) :**

La figure V.13 présente la bannière de notification reçue par le manager sur l'interface mobile de la plateforme TSS au moment exact de la détection de la récurrence. La notification indique l'équipement concerné et redirige directement vers le rapport correspondant.

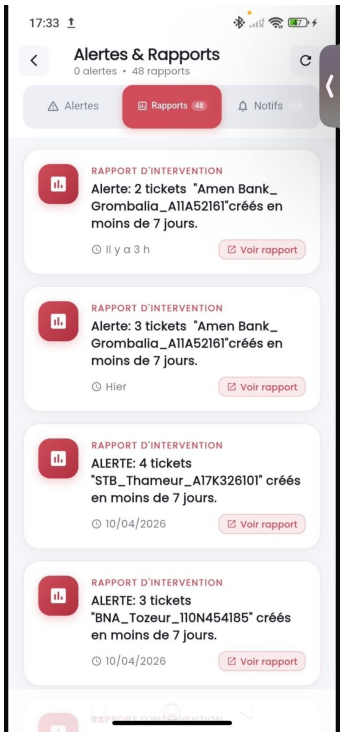


Figure V.12 : Notification en temps réel

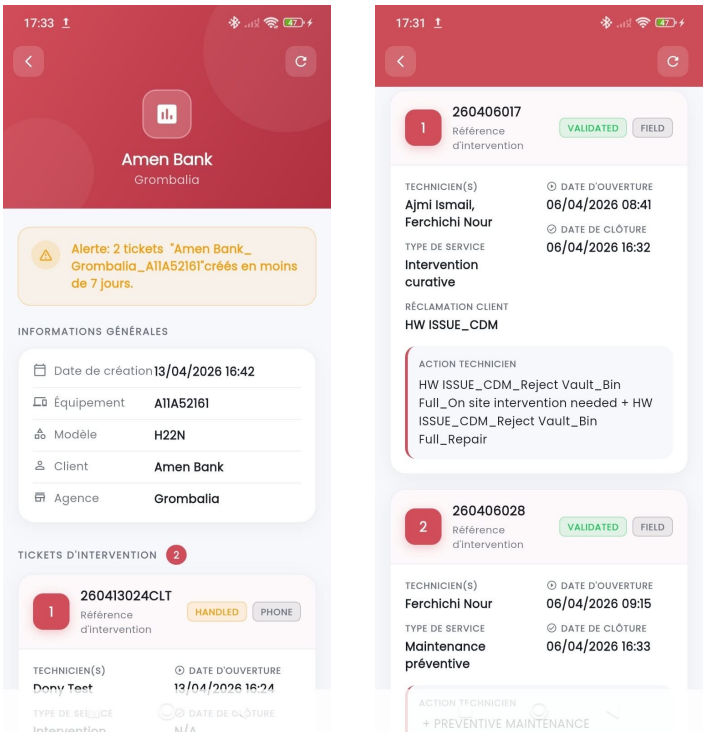


Figure V.13 : Notification en temps réel

V.8 Conclusion

Ce sprint a introduit un mécanisme de contrôle qualité automatique basé

Ces fonctionnalités positionnent la plateforme TSS comme un outil de supervision qualité proactive, permettant au manager d'identifier et de corriger rapidement les défaillances répétées avant qu'elles n'impactent la satisfaction client.

Chapitre VI

Release 3 — Sprint 5 : Gestion des Logs

VI.1 Introduction

Ce chapitre présente le cinquième sprint de la troisième release du projet TSS. Il porte sur la traçabilité complète des actions effectuées au sein de la plateforme et sur la mise en place d'un mécanisme de validation manager pour les opérations sensibles.

Contexte et problématique.

Dans un système multi-utilisateurs gérant des équipements critiques, il est essentiel de savoir *qui a fait quoi et quand*. De plus, certaines opérations telles que la modification de données sensibles (clients, agences, équipements, utilisateurs) ou la suppression définitive d'enregistrements ne doivent pas être exécutées sans supervision du manager. Sans ce contrôle, des erreurs ou des suppressions accidentelles peuvent compromettre l'intégrité des données.

Fonctionnalités implémentées :

- **Logs de connexion** : chaque tentative de connexion (réussie ou échouée) est automatiquement enregistrée avec l'identité de l'utilisateur, la date, l'heure et le résultat.
- **Logs de configuration** : toute suppresion ou modification de données (équipement, utilisateur, client, agence) génère un log en état **« en attente »** et déclenche une notification au manager, qui doit l'approuver ou la rejeter via la plateforme ou par e-mail avant qu'elle soit appliquée.

VI.2 Backlog du Sprint 5

Après la définition des objectifs du Sprint 5, les User Stories sélectionnées ont été décomposées en tâches techniques afin de faciliter leur implémentation et leur suivi.

Le tableau VI.1 présente le backlog détaillé du Sprint 5.

User Story	Tâches	Estimation
En tant que système, je veux enregistrer automatiquement chaque tentative de connexion (réussie ou échouée) avec les informations complètes de l'utilisateur.	Concevoir le modèle `LogConnexion` (userId, date, IP, statut).	1
	Injecter l'enregistrement du log dans le `AuthController`.	1
	Enregistrer les connexions échouées (email inconnu, mot de passe erroné, OTP invalide).	2
	Développer l'API de consultation des logs de connexion.	1
	Développer l'interface React d'affichage des logs (tableau + filtres).	2
	Tester les enregistrements en cas de succès et d'échec.	1
En tant que système, je veux créer un log pour toute modification de données (équipement, client, agence, utilisateur) et notifier le manager pour approbation.	Concevoir le modèle `LogModification` (entité, action, avant, après, statut).	1
	Intercepter les appels de modification dans les contrôleurs concernés.	2
	Sauvegarder l'état avant/après la modification dans le log.	2
	Développer l'API de création automatique du log .	1
	Envoyer notification en temps réel + e-mail au manager.	1
	Développer l'API d'approbation et de rejet du manager.	2
	Tester le cycle complet modification → attente → validation.	1
En tant que système, je veux bloquer toute suppression jusqu'à validation du manager	Concevoir le modèle `LogSuppression` (entité, statut, demandeur, manager).	1
	Intercepter les appels de suppression et créer un log `EN_ATTENTE`.	2
	Envoyer notification + e-mail au manager avec lien d'action rapide.	1
	Développer l'API de validation/refus de suppression par le manager.	2
	Exécuter la suppression uniquement après validation (`VALIDE`).	1

User Story	Tâches	Estimation
	Tester les scénarios de validation et de refus.	1
En tant que manager, je veux consulter le tableau de bord complet des logs (connexions, modifications en attente, suppressions en attente) avec filtres et actions.	Développer l'API de liste des logs avec filtres (type, statut, période, utilisateur).	2
	Développer la vue tableau de bord des logs avec onglets.	2
	Intégrer les boutons Approuver / Rejeter directement dans la liste.	1
	Afficher la comparaison avant/après pour les modifications.	1
	Tester les fonctionnalités.	1

Tab. VI.1 : Backlog du Sprint 5

VI.3 Diagramme de cas d'utilisation du Sprint 5

La figure VI.1 illustre les cas d'utilisation identifiés pour ce sprint.

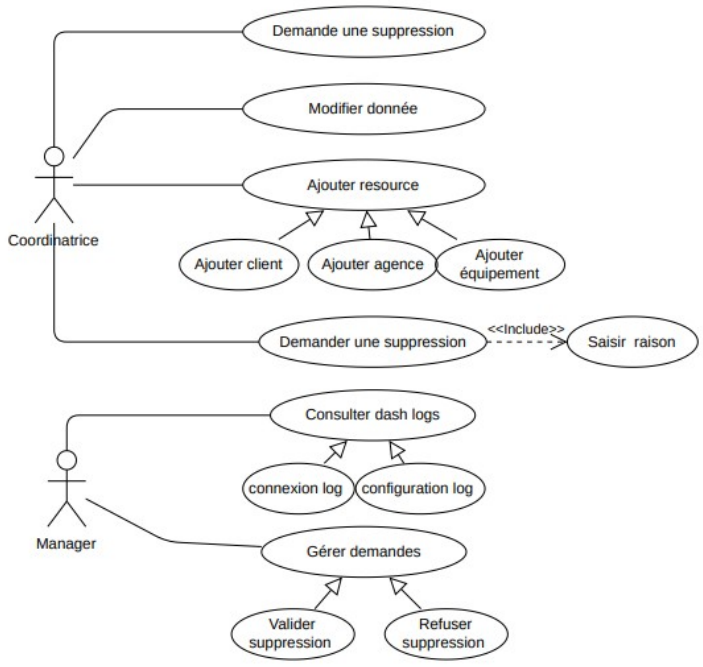


Figure VI.1 : Diagramme de cas d'utilisation du Sprint 5

VI.4 Services Web

Les services Web fournissent des services via Internet. Ils constituent une interface permettant à deux applications de communiquer. Les services ci-dessous sont résumés dans le tableau VI.2.

Méthode	URL	Description
Logs de connexion		
Get	/api/logs/connexion	Lister tous les logs de connexion.
Get	/api/logs/connexion/ :userId	Logs de connexion d'un utilisateur.
Logs de modification (validation manager)		
Get	/api/logs/modification	Lister les logs de modification.
Get	/api/logs/modification/pending	Lister les modifications en attente.
Put	/api/logs/modification/ :id/approuver	Manager approuve la modification.
Put	/api/logs/modification/ :id/rejeter	Manager rejette la modification.
Logs de suppression (validation manager)		
Get	/api/logs/suppression	Lister les logs de suppression.
Get	/api/logs/suppression/pending	Lister les suppressions en attente.
Put	/api/logs/suppression/ :id/valider	Manager valide la suppression.
Put	/api/logs/suppression/ :id/refuser	Manager refuse la suppression.

Tab. VI.2 : Liste des services web du Sprint 5.

VI.5 Les cas d'utilisation du Sprint 5

VI.5.1 Cas d'utilisation : « Enregistrer les logs de connexion »

VI.5.1.1 Description textuelle

Acteurs	Utilisateur (coordinatrice, technicien, manager), Système
Objectif	Enregistrer automatiquement chaque tentative de connexion avec l'identité, la date, l'heure et le résultat, afin de garantir la traçabilité complète des accès à la plateforme.
Pré-condition	Le module d'authentification est actif. Le modèle `LogConnexion` est défini en base de données.
Scénario principal	1- L'utilisateur soumet ses identifiants (email, mot de passe). 2- Le système vérifie les identifiants. 3- Si les identifiants sont valides, le système envoie le code OTP. 4- L'utilisateur saisit le code OTP. 5- Le système valide l'OTP et génère le token JWT. 6- Le système enregistre un log de connexion : { userId, email, date, heure, statut : `SUCCES` }. 7- L'utilisateur est redirigé vers son tableau de bord.
Scénario alternatif	**Identifiants incorrects :** Le système enregistre un log { email, date, statut : `ECHEC`, raison : « Identifiants incorrects » }. **OTP invalide ou expiré :** Le système enregistre un log { userId, date, statut : `ECHEC`, raison : « OTP invalide » }.

Tab. VI.3 : Cas d'utilisation : Enregistrer les logs de connexion

VI.5.1.2 Diagramme de séquence système

La figure VI.2 illustre les interactions entre l'utilisateur et le système lors de l'enregistrement d'un log de connexion.

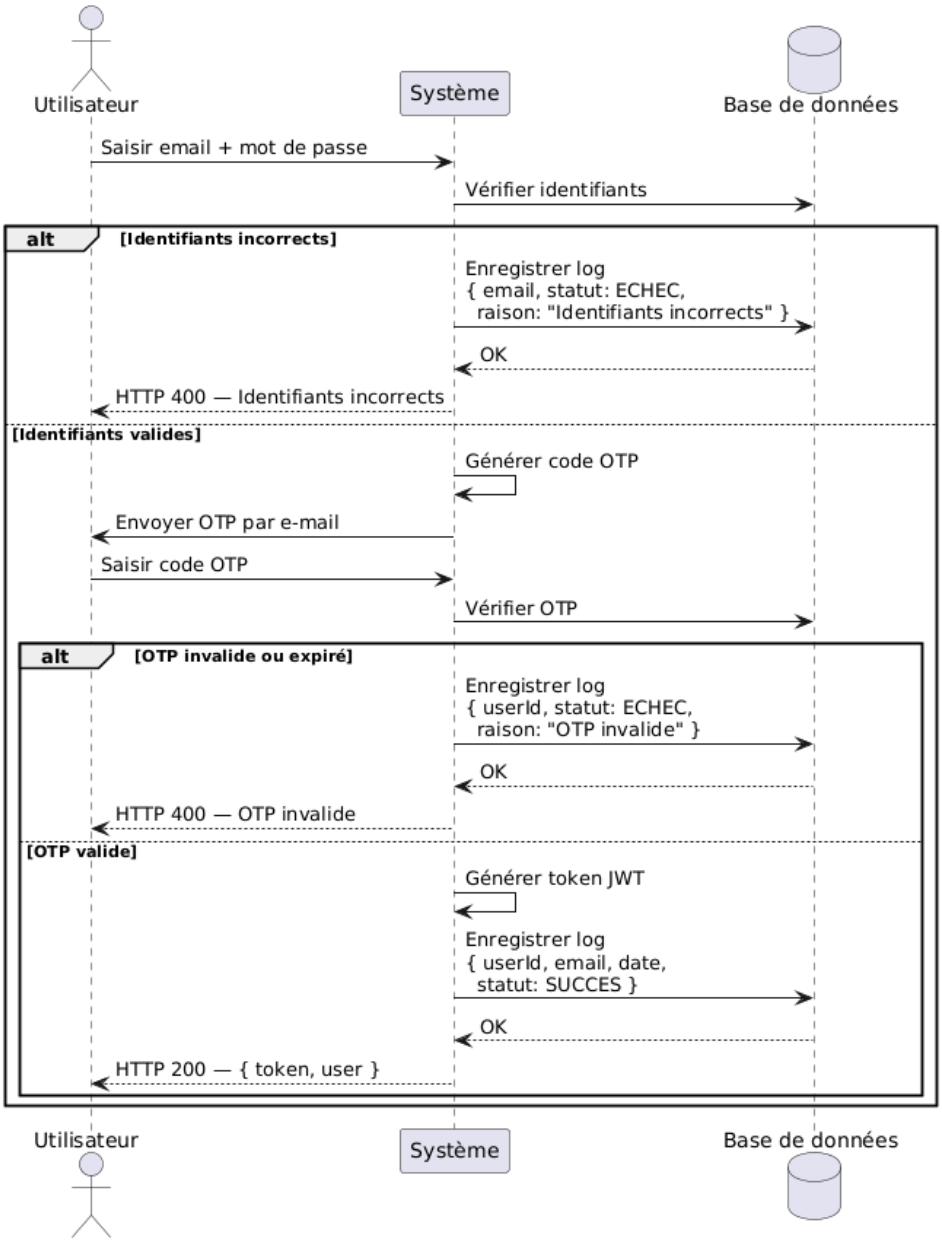


Figure VI.2 : Diagramme de séquence système — Enregistrer un log de connexion

VI.5.1.3 Diagramme de séquence objet

La figure VI.3 illustre les interactions entre les composants internes lors de l'enregistrement d'un log de connexion.

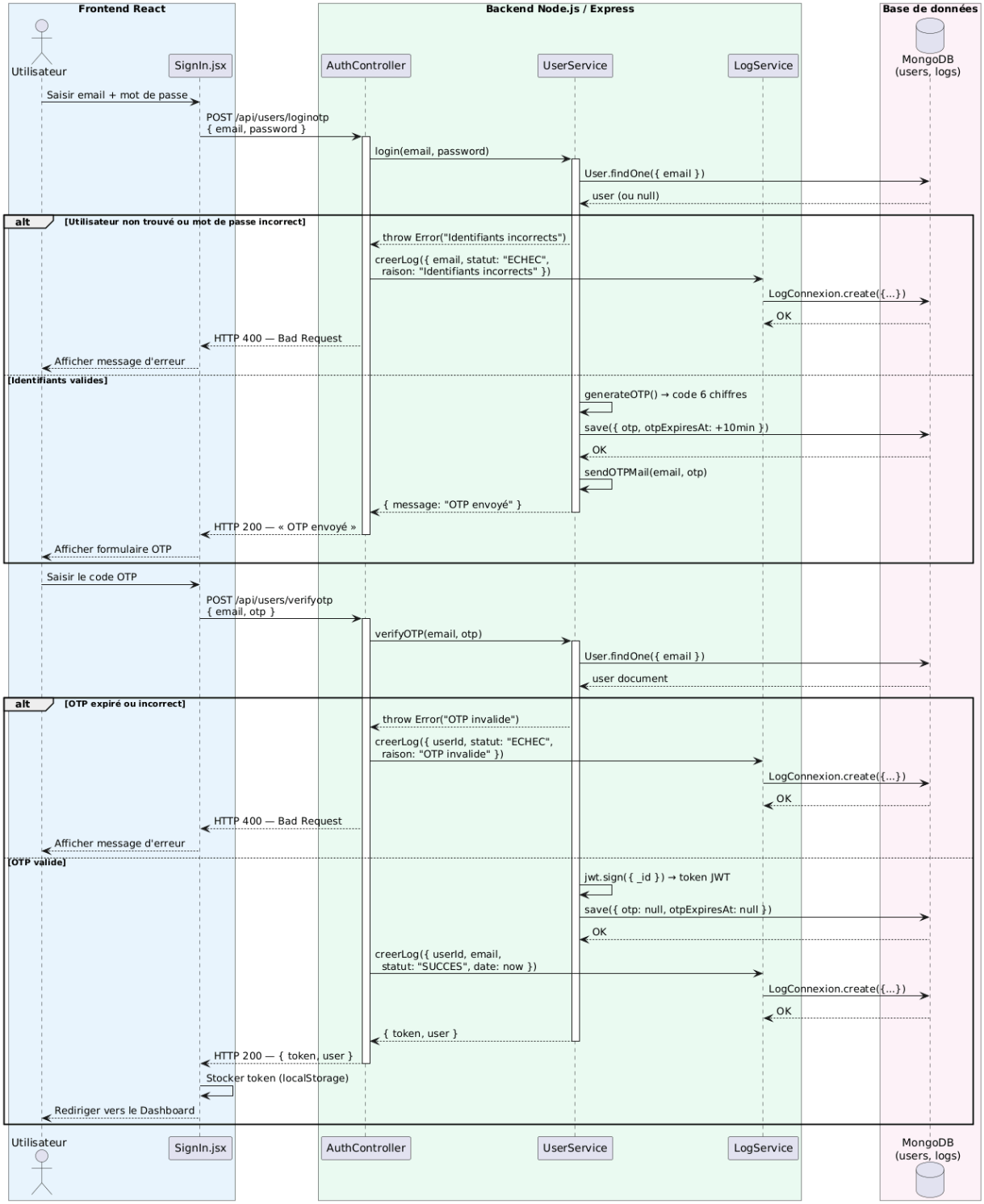


Figure VI.3 : Diagramme de séquence objet — Enregistrer un log de connexion

VI.5.2 Cas d'utilisation : « Valider une modification par le manager »

VI.5.2.1 Description textuelle

Acteurs	Coordinatrice (demandeur), Système, Manager (validateur)
Objectif	Garantir que toute modification de données sensibles (équipement, client, agence, utilisateur) est enregistrée en état **« en attente »** et ne sera appliquée qu'après approbation explicite du manager.
Pré-condition	La coordinatrice est authentifiée. L'entité à modifier existe en base de données.
Scénario principal	1- La coordinatrice modifie les données d'un équipement (ou client, agence, utilisateur). 2- Le système intercepte la demande de modification **avant** de l'appliquer. 3- Le système crée un log de modification contenant l'état avant et l'état après la modification. 4- Le système envoie au manager une **notification en temps réel** et un **e-mail** contenant le détail de la modification. 5- Le manager consulte la modification dans le tableau de bord des logs. 6- Il visualise la comparaison avant / après. 7a- **Approbation** : le manager clique sur **Approuver** → le système applique la modification en base et marque le log `APPROUVE`. 7b- **Rejet** : le manager clique sur **Rejeter** → la modification est annulée, le log est marqué `REJETE` et la coordinatrice est notifiée.
Scénario alternatif	**Manager non disponible :** La modification reste en état `EN_ATTENTE` jusqu'à action du manager. La coordinatrice voit le statut « En attente de validation » dans l'interface.

Tab. VI.4 : Cas d'utilisation : Valider une modification par le manager

VI.5.2.2 Diagramme de séquence système

La figure VI.4 illustre les interactions entre la coordinatrice, le système et le manager lors de la validation d'une modification.

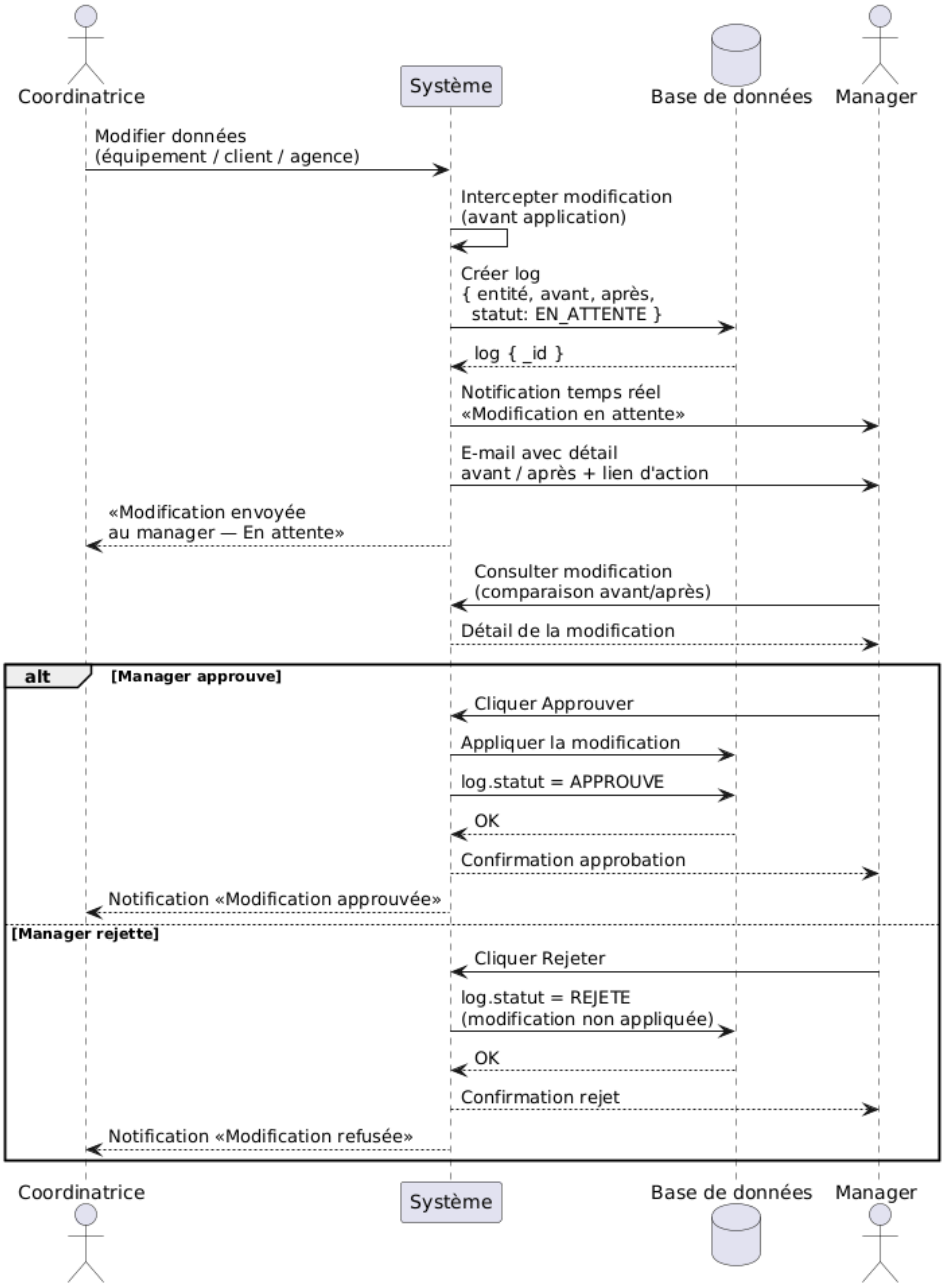


Figure VI.4 : Diagramme de séquence système — Valider une modification

VI.5.2.3 Diagramme de séquence objet

La figure VI.5 illustre les interactions entre les composants internes lors du cycle de validation d'une modification.

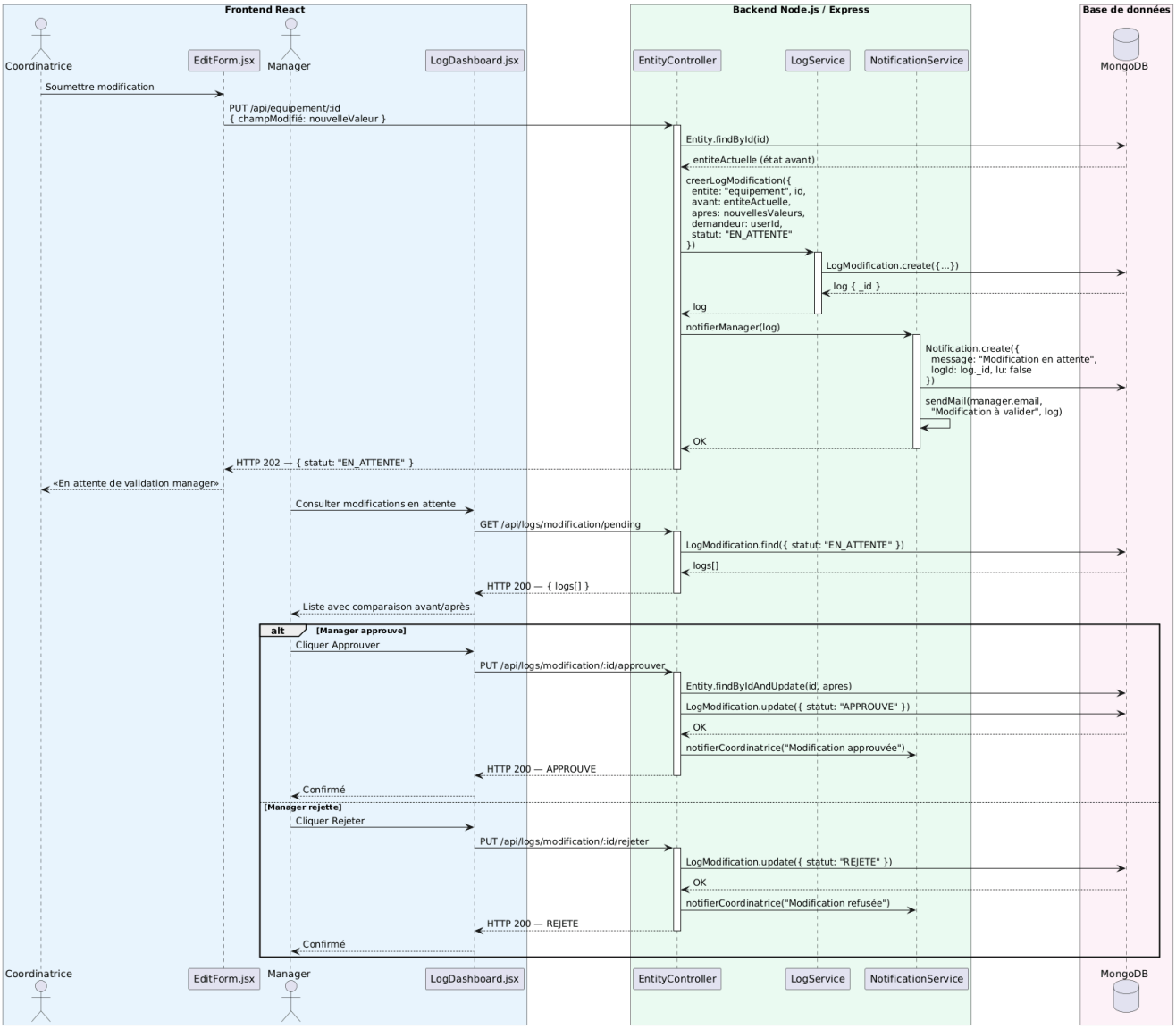


Figure VI.5 : Diagramme de séquence objet — Valider une modification

VI.5.3 Cas d'utilisation : « Valider une suppression par le manager »

VI.5.3.1 Description textuelle

Acteurs	Coordinatrice (demandeur), Système, Manager (validateur)
Objectif	Garantir qu'aucune suppression de données n'est exécutée sans validation explicite du manager, afin de prévenir toute perte de données accidentelle ou non autorisée.
Pré-condition	La coordinatrice est authentifiée. L'entité à supprimer existe en base de données.
Scénario principal	1- La coordinatrice clique sur **Supprimer** pour un équipement (ou client, agence, utilisateur). 2- Le système bloque la suppression immédiate et crée un log en état `EN_ATTENTE`. 3- Le système envoie au manager une **notification en temps réel** et un **e-mail** contenant les détails de l'entité à supprimer et un lien d'action rapide. 4- La coordinatrice voit le message : « Demande envoyée au manager, en attente de validation ». 5- Le manager consulte la demande de suppression dans le tableau de bord. 6- **Validation** : le manager clique sur **Valider** → le système exécute la suppression et marque le log `VALIDE`. 7- Le système notifie la coordinatrice du résultat.
Scénario alternatif	**Refus du manager :** Le manager clique sur **Refuser** → la suppression est annulée, le log est marqué `REFUSE` et la coordinatrice reçoit une notification « Suppression refusée par le manager ». **Lien e-mail :** Le manager peut approuver ou refuser directement depuis l'e-mail reçu, sans passer par la plateforme.

Tab. VI.5 : Cas d'utilisation : Valider une suppression par le manager

VI.5.3.2 Diagramme de séquence système

La figure VI.6 illustre les interactions entre la coordinatrice, le système et le manager lors de la validation d'une suppression.

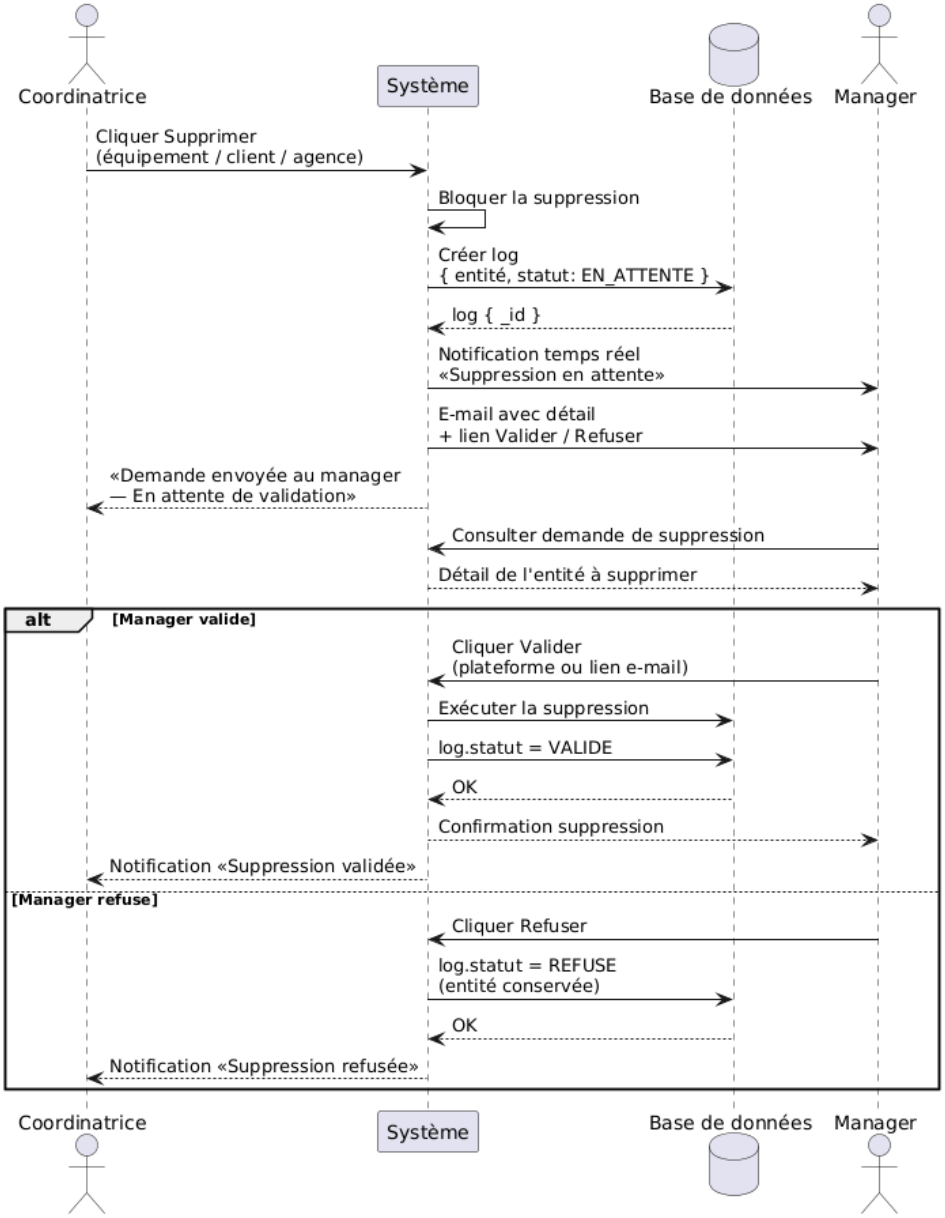


Figure VI.6 : Diagramme de séquence système — Valider une suppression

VI.5.3.3 Diagramme de séquence objet

La figure VI.7 illustre les interactions entre les composants internes lors du cycle de validation d'une suppression.

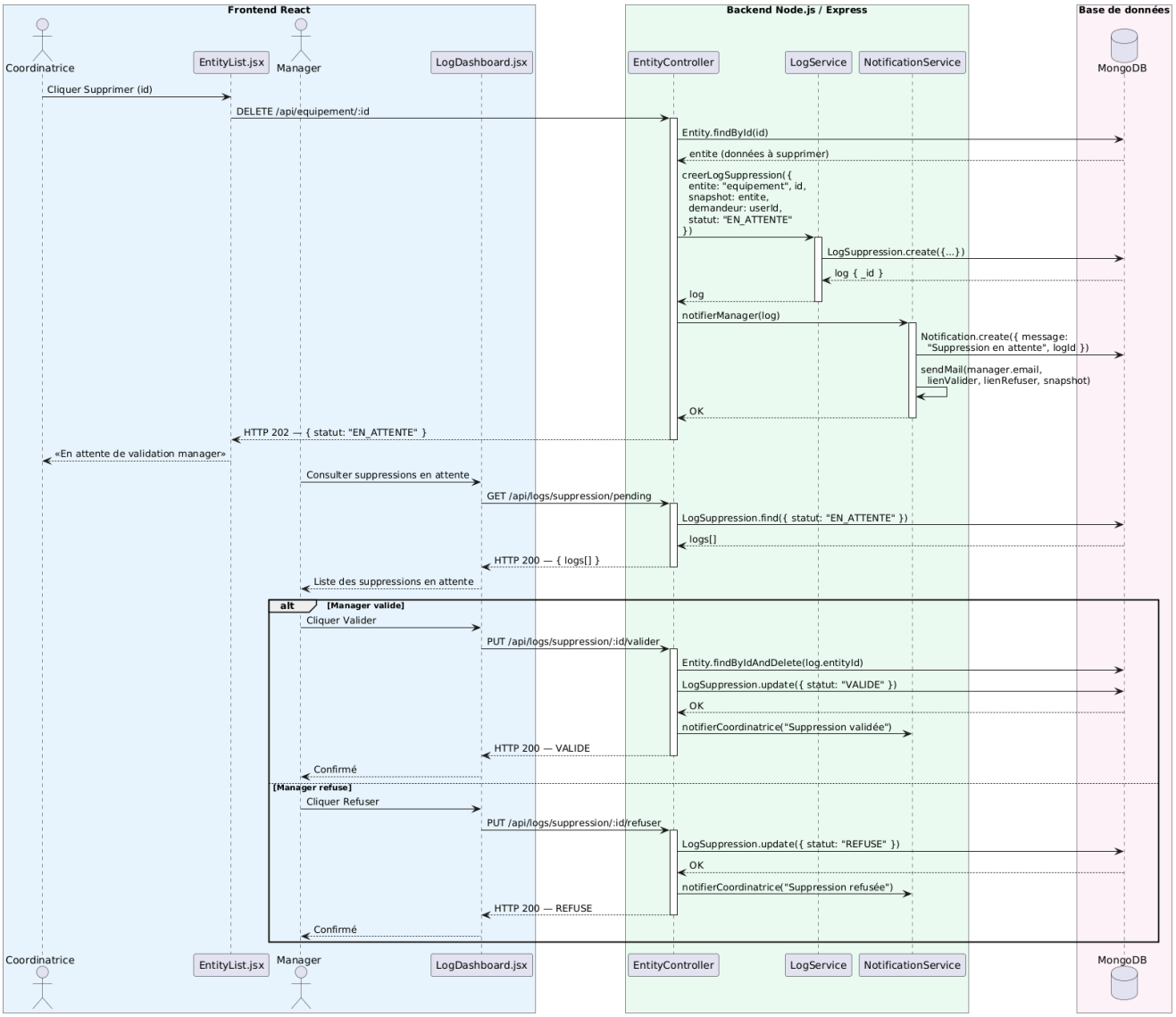


Figure VI.7 : Diagramme de séquence objet — Valider une suppression

VI.6 Diagramme d'activité

La figure VI.8 modélise le flux d'activité complet depuis l'action d'un utilisateur jusqu'à la validation (ou le refus) par le manager.

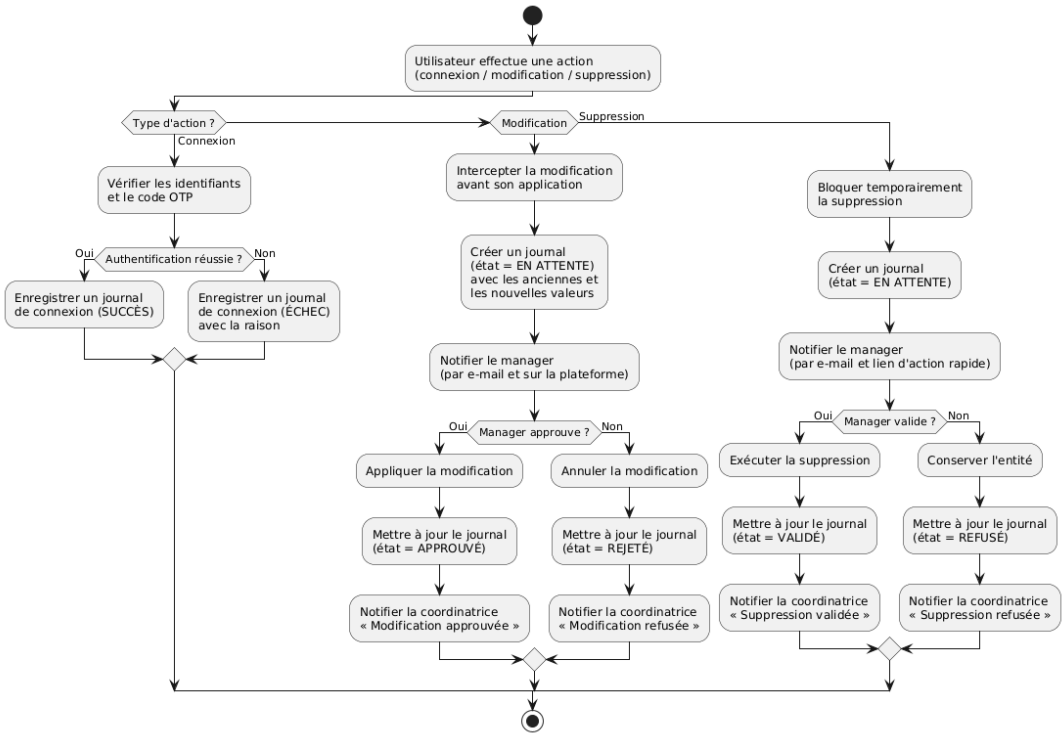


Figure VI.8 : Diagramme d'activité — Sprint 5 : Logs et validation manager

VI.7 Réalisation du Sprint 5

- **Consultation des logs de connexion :**

La figure VI.9 illustre l'interface de consultation des logs de connexion accessible au manager. Chaque ligne du tableau affiche l'identité de l'utilisateur, son adresse e-mail, la date et l'heure de la tentative, l'adresse IP ainsi que le statut (`SUCCES` ou `ECHEC`) avec la raison en cas d'échec. Des filtres permettent de rechercher par utilisateur, par statut ou par période.

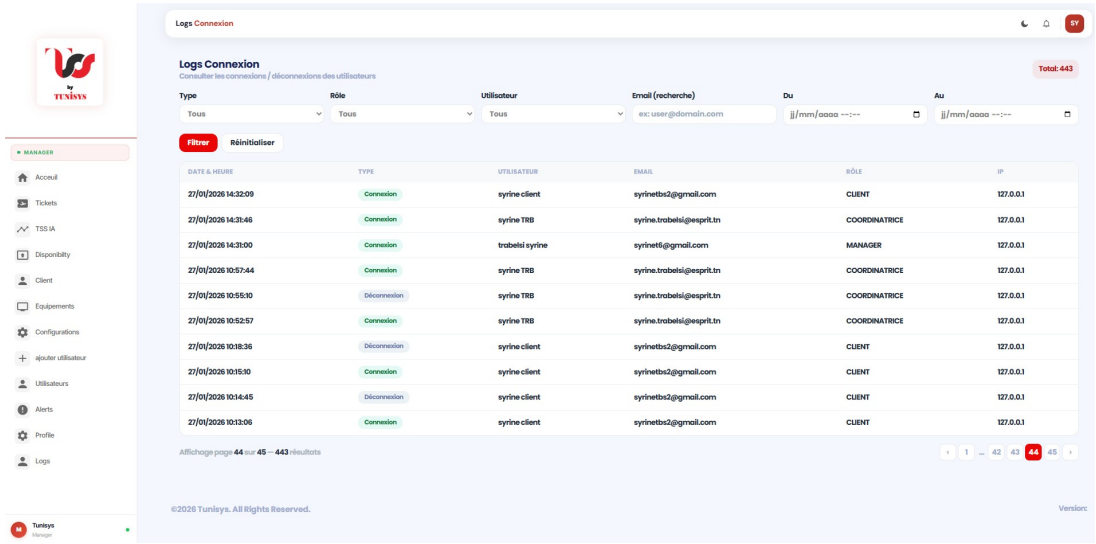


Figure VI.9 : Interface de consultation des logs de connexion — Manager

- **Logs de modification en attente de validation :**

La figure VI.10 présente le tableau de bord des logs de modification. Chaque entrée affiche l'entité modifiée (équipement, client, agence ou utilisateur), l'auteur de la demande, la date et le statut courant (`EN_ATTENTE`, `APPROUVE` ou `REJETE`). Le manager peut visualiser la comparaison avant / après la modification avant de prendre sa décision.

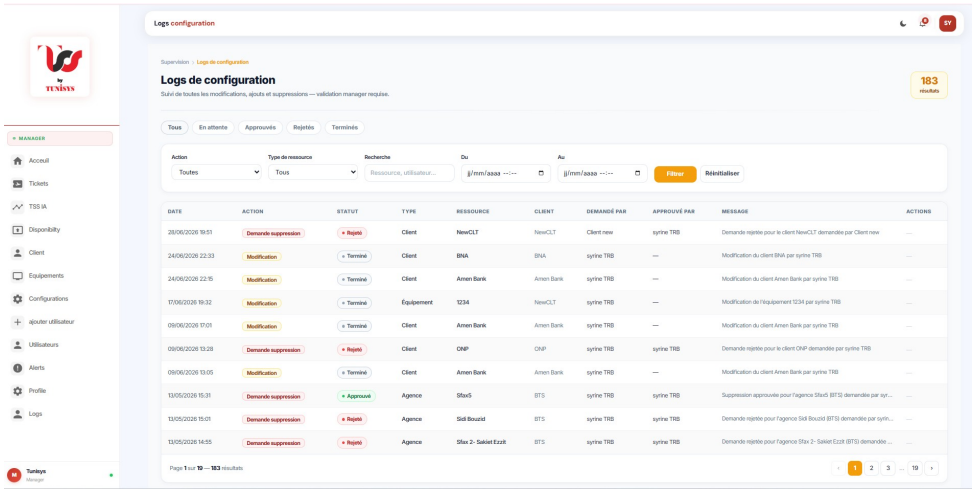


Figure VI.10 : Interface de consultation des logs de modification

- **Approbation ou rejet d'une demande par le manager :**

La figure VI.11 présente l'interface dédiée aux demandes de suppression. Chaque demande est bloquée dès son déclenchement et reste en état `EN_ATTENTE` jusqu'à action explicite du manager. Le tableau affiche le type de l'entité à supprimer, l'auteur de la demande et la date.

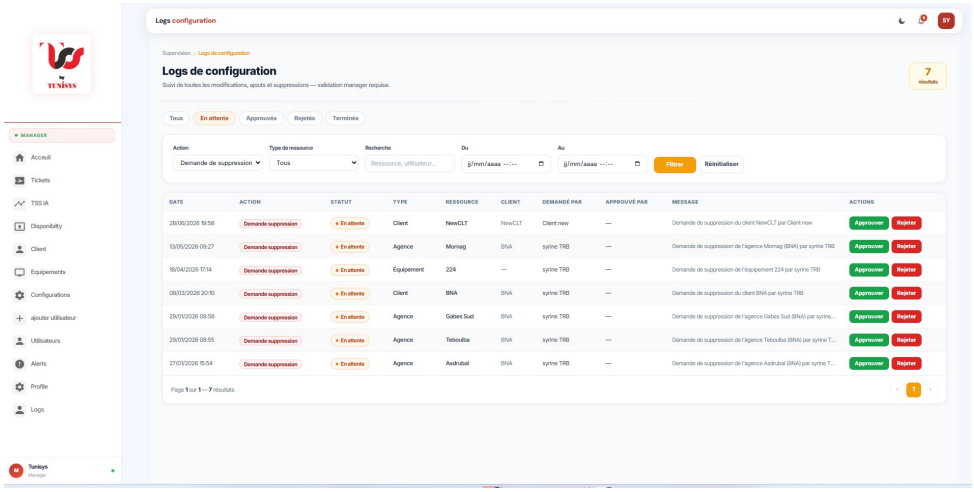


Figure VI.11 : Tableau de bord des logs de suppression — Manager

- **Notification e-mail au manager — action rapide :**

La figure VI.12 illustre l'e-mail reçu automatiquement par le manager lorsqu'une demande de modification ou de suppression est enregistrée. L'e-mail contient le détail de l'opération et deux liens d'action rapide (**Approuver** / **Refuser**) permettant au manager de valider directement depuis sa boîte de réception sans se connecter à la plateforme.

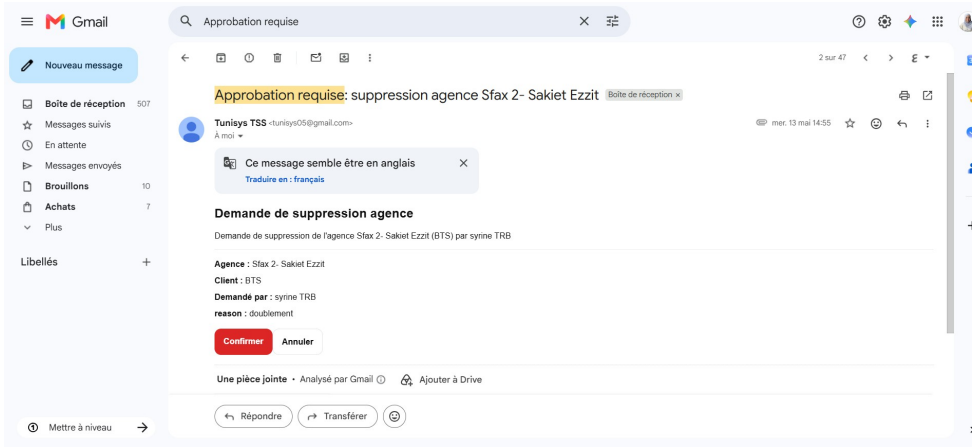


Figure VI.12 : E-mail de notification envoyé au manager avec liens d'action rapide

VI.8 Conclusion

Ce sprint a permis de renforcer la sécurité et la traçabilité de la plateforme TSS grâce à la mise en place d'un système complet de journalisation, de validation des modifications et des suppressions, ainsi que de notifications en temps réel. Ces fonctionnalités garantissent un meilleur contrôle des opérations sensibles et assurent l'intégrité des données. Le manager joue désormais un rôle central dans la supervision et la validation des actions critiques.

Chapitre VII

Déploiement de la Plateforme TSS

VII.1 Introduction

Ce chapitre décrit la stratégie de déploiement adoptée pour la plateforme TSS. Dans un premier temps, nous présentons l'approche initiale utilisée avant notre intervention : un déploiement traditionnel sur une machine virtuelle Windows, avec un serveur IIS pour le frontend et un service Windows pour le backend. Nous en identifions les limites principales, notamment l'absence de pipeline d'intégration continue et la nécessité d'une interruption de service à chaque mise à jour.

Dans un second temps, nous proposons et mettons en œuvre une pipeline CI/CD basée sur **Jenkins** et **Docker**, testée localement avec Docker Desktop. Cette solution élimine les coupures de service, automatise entièrement le cycle build → test → déploiement et garantit la reproductibilité de l'environnement d'exécution.

VII.2 Déploiement initial — Approche traditionnelle

VII.2.1 Architecture initiale

Avant notre intervention, la plateforme TSS était hébergée sur une **machine virtuelle Windows**. Les trois composants applicatifs étaient installés et gérés **manuellement** sur ce serveur unique :

- **Backend Node.js** : exécuté en arrière-plan sous la forme d'un service Windows, démarré automatiquement à chaque redémarrage du serveur.
- **Frontend React** : compilé localement par un développeur (`npm run build`), puis le dossier `build/` était copié manuellement vers le répertoire de publication du serveur IIS (Internet Information Services)
- **Base de données MongoDB** : installée directement sur la machine virtuelle et lancée en tant que service Windows.

La figure VII.1 illustre cette architecture initiale.

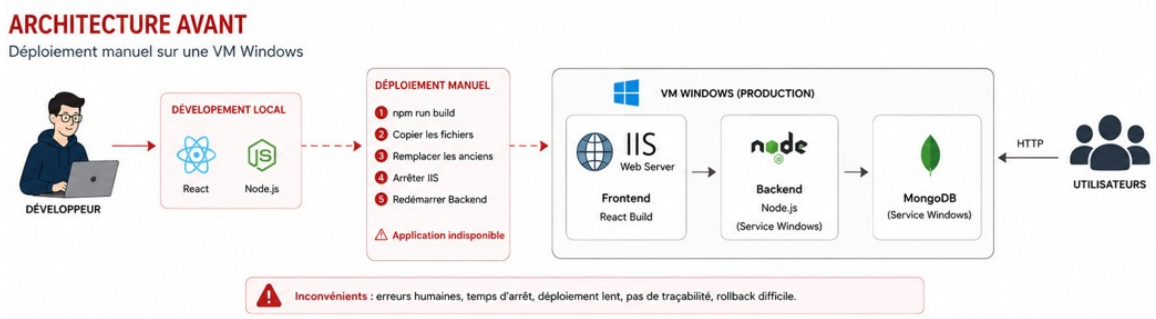


Figure VII.1 : Architecture de déploiement initiale — Machine virtuelle Windows

VII.2.2 Le serveur IIS (Internet Information Services)

VII.2.2.1 Présentation d'IIS

IIS (Internet Information Services)[6] est un serveur web développé par Microsoft et intégré aux systèmes d'exploitation Windows Server. Il permet l'hébergement et la publication d'applications web, qu'elles soient statiques ou dynamiques, directement sur une infrastructure Windows, sans recourir à des outils tiers.

Rôle dans la plateforme TSS : Dans le cadre de notre projet, IIS était principalement utilisé pour assurer le **déploiement du frontend**. Il servait les fichiers statiques générés par le build de l'application React, notamment le fichier `index.html`, ainsi que les ressources JavaScript, CSS et les fichiers multimédias associés. Le serveur était configuré pour écouter sur le port HTTP standard (port 80) et rendait l'application accessible aux utilisateurs au sein du réseau local.

Avantages et limites : IIS présente l'avantage d'être nativement intégré à l'environnement Windows, avec une interface de configuration graphique simple et accessible. Toutefois, dans un contexte de déploiement moderne, cette approche montre certaines limites. En effet, chaque mise à jour du frontend nécessite généralement l'arrêt du site, la copie manuelle des nouveaux fichiers issus du build, puis le redémarrage du service. Cette procédure engendre une interruption temporaire de service, ce qui constitue une contrainte importante en termes de disponibilité et de continuité applicative.

VII.2.3 Problèmes et limites identifiés

L'analyse du fonctionnement en production a mis en évidence plusieurs problèmes critiques résumés dans le tableau VII.1.

Problème	Impact
Absence de pipeline CI/CD	Chaque mise en production d'une nouvelle version nécessitait une intervention manuelle complète (build local, copie FTP, redémarrage du service).
Interruption de service obligatoire	Le déploiement impliquait l'arrêt du backend et du serveur IIS. Pour minimiser l'impact, les mises à jour devaient être effectuées en dehors des heures de travail (nuit ou week-end).
Environnement non reproductible	L'application dépendait des versions de Node.js, Python et MongoDB installées sur la VM. Une migration vers une autre machine imposait une reconfiguration manuelle complète.
Absence de tests automatisés	Aucun mécanisme ne vérifiait la qualité du code avant le déploiement ; un bug pouvait être mis en production sans détection préalable.
Pas de traçabilité des déploiements	Il n'existait aucun historique des versions déployées ni de possibilité de retour arrière (*rollback*) rapide en cas d'incident.
Pas de scalabilité	L'architecture mono-processus sur une VM unique ne permettait aucune montée en charge ni haute disponibilité.

Tab. VII.1 : Problèmes identifiés dans le déploiement initial

VII.3 Solution proposée : Pipeline CI/CD avec Jenkins et Docker

VII.3.1 Choix technologiques

Pour remédier aux limitations identifiées, nous avons proposé une architecture de déploiement basée sur la conteneurisation et l'intégration continue. Les technologies retenues sont présentées ci-dessous.

- **Docker** : plateforme de conteneurisation permettant d'empaqueter chaque composant applicatif (backend, frontend, OCR) avec toutes ses dépendances dans une image isolée et portable. Cela garantit que l'application se comporte de manière identique en local, en test et en production.
- **Docker Compose** : outil de définition et d'orchestration multi-conteneurs. Un fichier `docker-compose.yml` unique décrit les quatre services (backend, frontend,

OCR, MongoDB), leurs ports, leurs variables d'environnement et le réseau partagé qui les relie.

- **Docker Hub** : registre d'images Docker public utilisé comme dépôt centralisé. Après chaque build Jenkins, les images versionées sont poussées sur Docker Hub sous le compte `syrinetrabelsi8`, puis tirées par la cible de déploiement.
- **Jenkins** : serveur d'automatisation open-source qui orchestre la pipeline CI/CD. Il détecte chaque nouveau commit Git, exécute les étapes de build, de test et de déploiement de façon totalement automatique et sans intervention humaine.
- **Docker Desktop** : utilisé comme environnement de test local sous Windows, permettant de faire tourner Jenkins lui-même dans un conteneur et d'orchestrer les autres conteneurs de l'application sur la même machine.

VII.3.2 Architecture cible avec Docker

La figure VII.2 présente l'architecture de déploiement cible après migration vers la pipeline CI/CD conteneurisée.

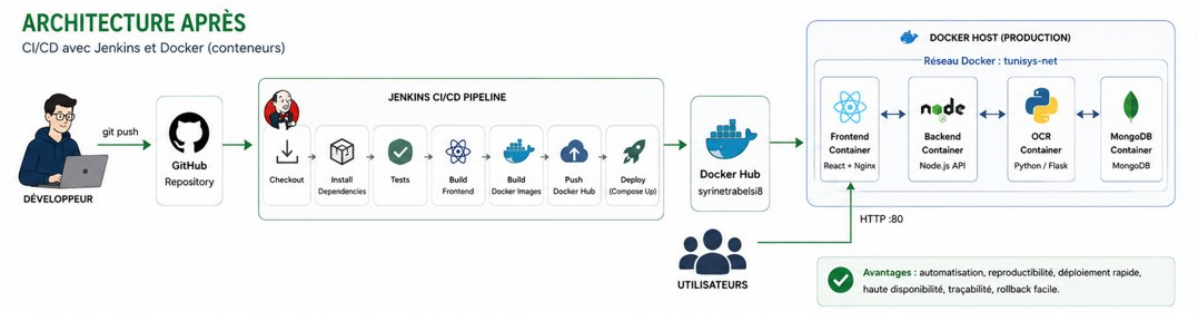


Figure VII.2 : Architecture de déploiement cible — Conteneurs Docker

La plateforme est désormais composée de **quatre conteneurs Docker** interconnectés via un réseau bridge dédié (`tunisys-net`), décrits dans le tableau VII.2.

Conteneur	Port	Rôle
`tunisys-backend`	4000	API REST Node.js/Express. Communique avec MongoDB et le service OCR via le réseau interne.
`tunisys-frontend`	80	Application React compilée, servie par Nginx. Expose l'interface utilisateur sur le port 80 (HTTP).
`tunisys-ocr`	5001	Micro-service Python/Flask dédié à la reconnaissance optique de caractères (Gemini Vision).
`tunisys-mongo`	27017	Base de données MongoDB 7. Les données sont persistées dans un volume Docker nommé `tunisys-mongo-data`, partagé entre les déploiements successifs.

Tab. VII.2 : Description des quatre conteneurs de la plateforme TSS

VII.4 Mise en œuvre de la pipeline Jenkins

VII.4.1 Vue d'ensemble de la pipeline

La pipeline Jenkins est définie dans un fichier `Jenkinsfile` placé à la racine du dépôt Git. Elle comporte **sept étapes** qui s'enchaînent automatiquement à chaque push sur la branche principale, comme illustré dans la figure VII.3.

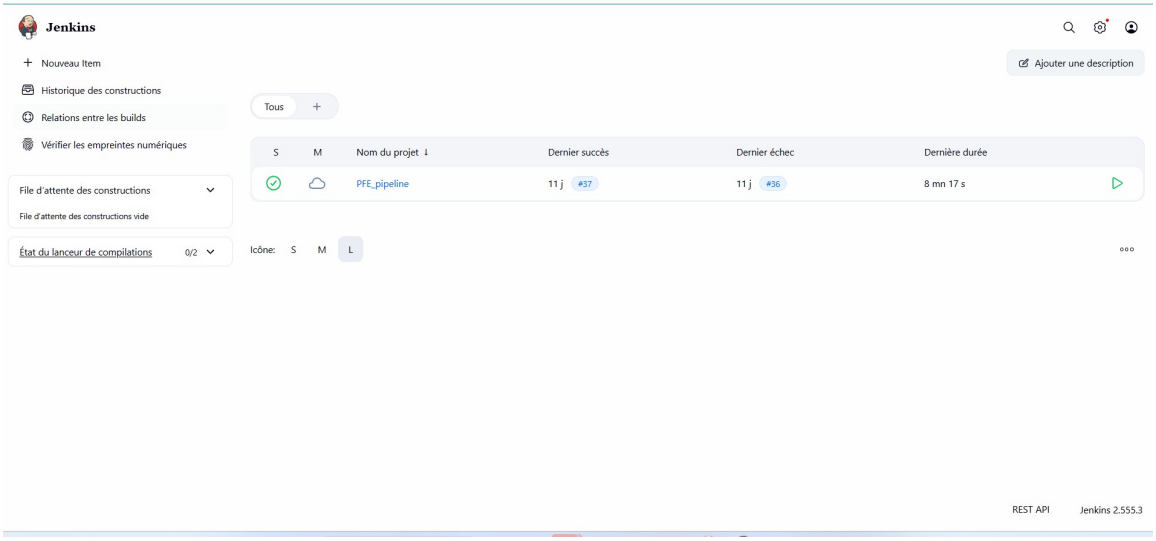


Figure VII.3 : Vue d'ensemble de la pipeline Jenkins — Étapes du build

VII.4.2 Description des étapes

#	Étape	Description
1	**Checkout**	Jenkins clone le dépôt Git et extrait le dernier commit de la branche active. Le numéro de commit et la branche sont affichés dans les logs pour la traçabilité.
2	**Install Dependencies**	Installation parallèle des dépendances des trois composants : `npm ci` pour le backend Node.js, `npm ci --legacy-peer-deps` pour le frontend React, et `pip3 install` pour le service OCR Python.
3	**Tests**	Exécution parallèle des tests unitaires backend et frontend. La pipeline continue même si aucun test n'est encore défini (`--passWithNoTests`).
4	**Build Frontend**	Compilation de l'application React avec `npm run build`. Le dossier `front/build/` produit est ensuite copié dans l'image Docker du frontend lors de l'étape suivante.
5	**Build Docker Images**	Construction en parallèle des trois images Docker : `tunisys-backend`, `tunisys-frontend` et `tunisys-ocr`, chacune taguée avec le numéro de build Jenkins et l'étiquette `latest`.
6	**Push Docker Images**	Authentification sur Docker Hub via les credentials Jenkins (`DOCKER_HUB_TOKEN`), puis push des trois images versionées et `latest` vers le compte `syrinetrabelsi8`.
7	**Deploy**	Déploiement automatique via `docker compose` : pull des nouvelles images, arrêt et suppression des conteneurs précédents, puis relance avec `docker compose up -d`. Le volume `tunisys-mongo-data` est déclaré externe afin de conserver les données entre deux déploiements successifs.

Tab. VII.3 : Description des étapes de la pipeline Jenkins

VII.5 Comparaison avant / après

Le tableau VII.4 synthétise les gains apportés par la migration vers la pipeline CI/CD conteneurisée.

Critère	Avant (IIS / VM)	Après (Jenkins / Docker)
Déclenchement du déploiement	Manuel — intervention développeur	Automatique — push Git
Interruption de service	Oui — arrêt IIS + service requis	Non — remplacement à chaud des conteneurs
Heure de déploiement	En dehors des heures de travail	À tout moment, sans contrainte
Reproductibilité	Dépendante de la configuration de la VM	Garantie par les images Docker
Tests automatisés	Absents	Intégrés à la pipeline (étape Tests)
Traçabilité des versions	Aucune	Tags Docker + numéro de build Jenkins
Retour arrière (*rollback*)	Impossible sans sauvegarde manuelle	Possible en 1 commande (image taguée)
Scalabilité	Limitée à la VM unique	Extensible via `docker compose scale`

Tab. VII.4 : Comparaison du déploiement avant et après la mise en place de la pipeline CI/CD

VII.6 Réalisation

- **Jenkins — Interface de la pipeline :**

La figure VII.4 présente l'interface Jenkins affichant le résultat d'un build complet. Chaque étape est représentée par une colonne colorée indiquant son statut (vert = succès). Le numéro de build et la durée totale d'exécution sont visibles en haut de la page.

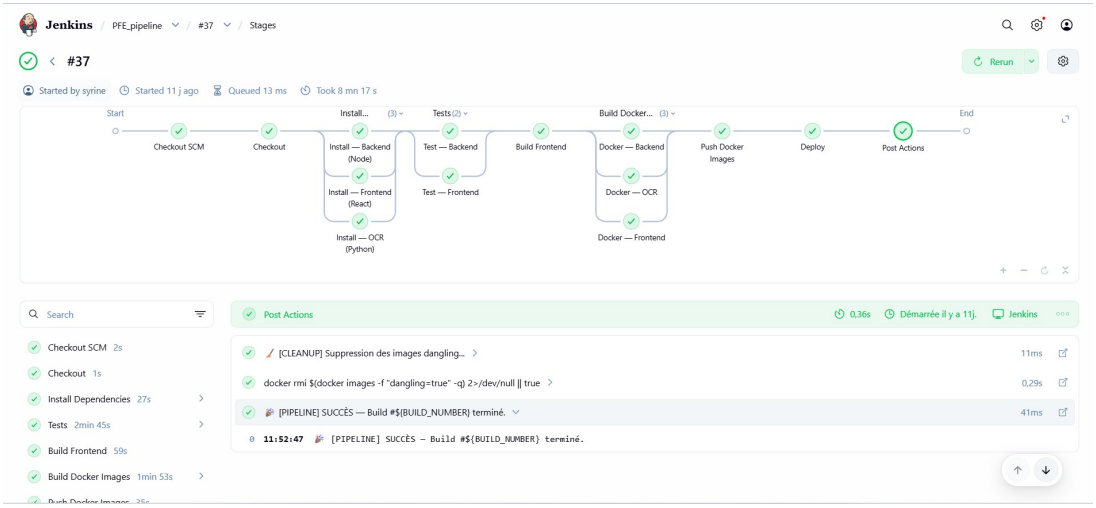


Figure VII.4 : Interface Jenkins — Pipeline exécutée avec succès

- **Jenkins — Logs d'exécution des étapes :**

La figure VII.5 illustre les logs détaillés d'une exécution de pipeline. Chaque étape affiche ses messages d'information, permettant de diagnostiquer rapidement tout problème survenu lors du build, du push ou du déploiement.

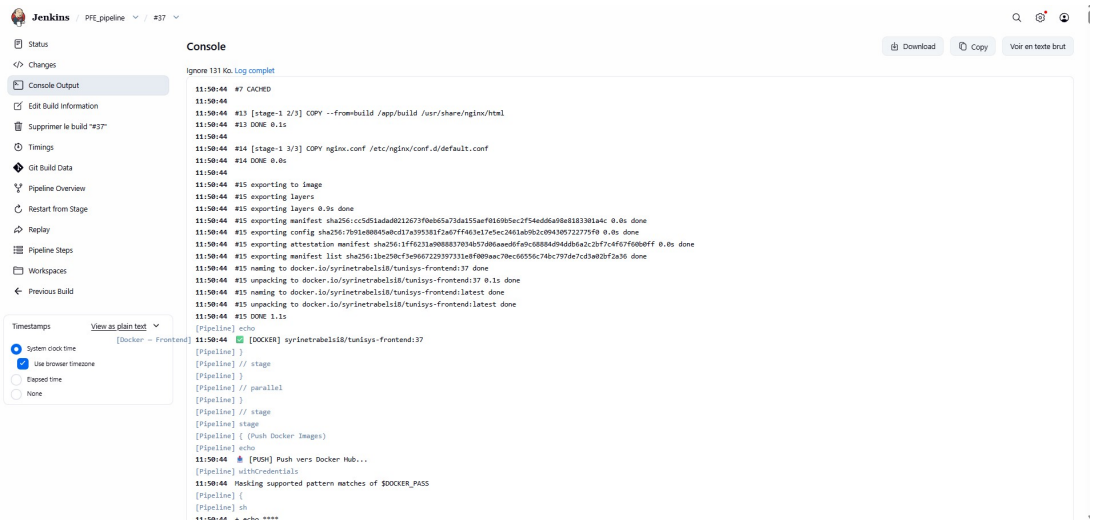


Figure VII.5 : Logs d'exécution détaillés d'un build Jenkins

- **Docker Hub — Images publiées :**

La figure VII.6 présente le dépôt Docker Hub du projet. Les trois images (`tunisys-backend`, `tunisys-frontend`, `tunisys-ocr`) y sont publiées après chaque build réussi, avec le tag correspondant au numéro de build Jenkins et l'étiquette `latest`.

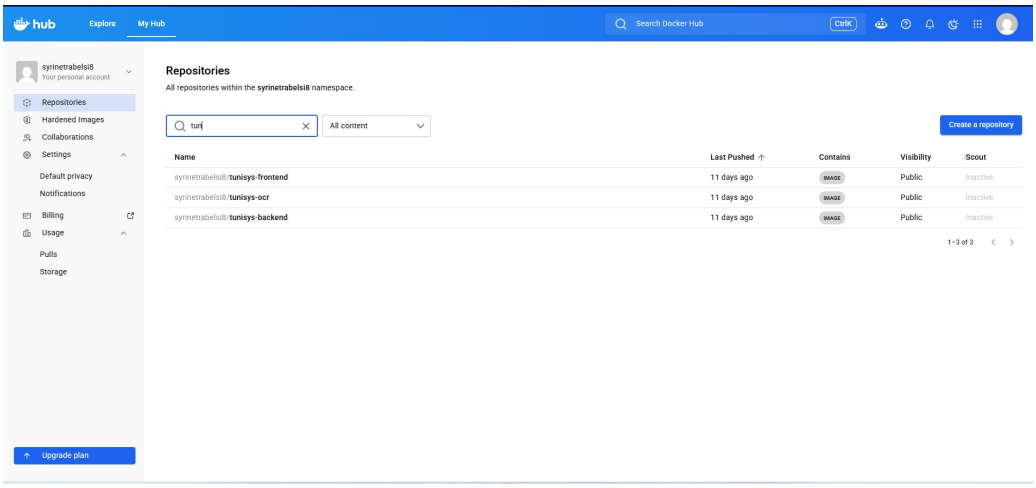


Figure VII.6 : Dépôt Docker Hub — Images versionées de la plateforme TSS

- **Docker Desktop — Conteneurs en cours d'exécution :**

La figure VII.7 illustre Docker Desktop après le déploiement. Les quatre conteneurs (`tunisys-backend`, `tunisys-frontend`, `tunisys-ocr`, `tunisys-mongo`) sont actifs et en bonne santé, tous rattachés au réseau `tunisys-net`.

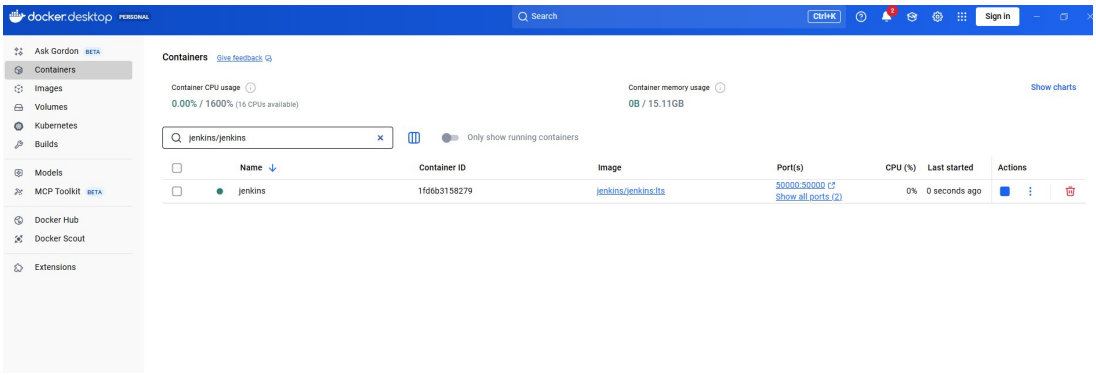


Figure VII.7 : Docker Desktop — Les quatre conteneurs TSS en cours d'exécution

- **Plateforme TSS accessible après déploiement :**

La figure VII.8 confirme que l'application TSS est pleinement fonctionnelle après déploiement via la pipeline. Le frontend React est accessible sur le port 80, tandis que le backend répond sur le port 4000, le tout sans la moindre intervention manuelle ni interruption de service.

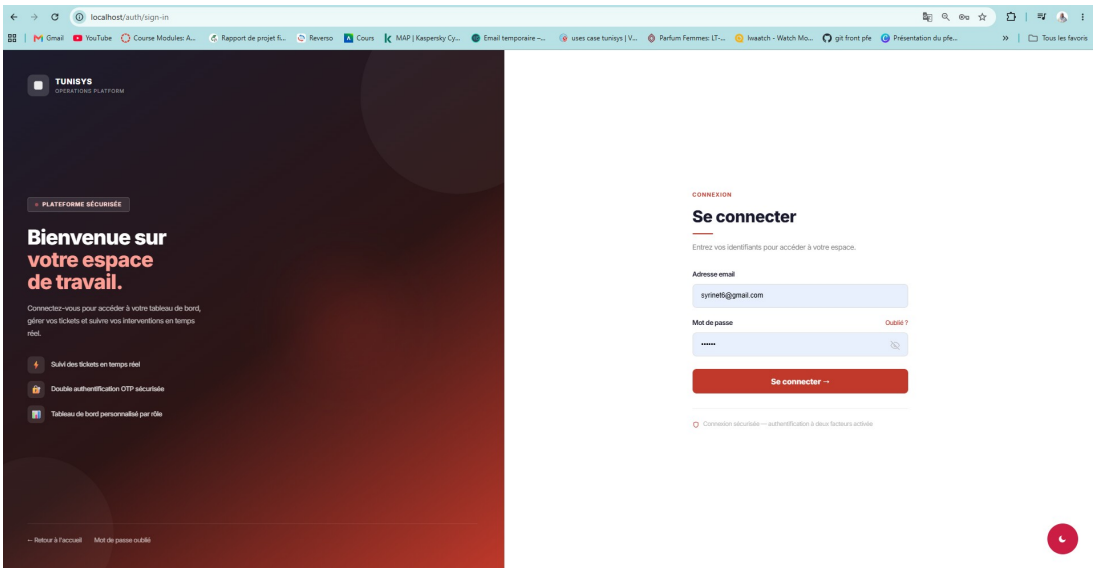


Figure VII.8 : Plateforme TSS accessible après déploiement automatique

VII.7 Conclusion

Ce chapitre a présenté la migration du déploiement de la plateforme TSS d'une approche traditionnelle manuelle vers une pipeline CI/CD moderne et automatisée. Les apports essentiels de cette migration sont :

- **Zéro interruption de service** : le remplacement à chaud des conteneurs Docker supprime les coupures de production qui obligeaient auparavant les équipes à intervenir en dehors des heures de travail.
- **Déploiement en un clic** : un simple push Git déclenche automatiquement l'ensemble de la pipeline (build, tests, image Docker, push Hub, déploiement), sans aucune action manuelle.
- **Traçabilité complète** : chaque version déployée est identifiée par un tag Docker correspondant au numéro de build Jenkins, ce qui permet un retour arrière immédiat en cas d'incident.
- **Environnement reproductible** : les images Docker garantissent que l'application se comporte de façon identique sur tout environnement cible, éliminant les problèmes de configuration manuelle propres à la VM initiale.
- **Fondation scalable** : l'architecture multi-conteneurs ouvre la voie à une montée en charge future (réplication de services, orchestration Kubernetes) sans remise en cause de l'architecture applicative.

Cette modernisation du déploiement constitue une étape structurante pour la plateforme TSS, en lui apportant les standards de l'industrie en matière de livraison logicielle continue.

Chapitre VIII

Conclusion et perspectives

Ce projet de fin d'études, réalisé au sein de **TUNISYS**, avait pour objectif de concevoir, développer et déployer **TSS (Tunisys Service Solution)**, une plateforme intelligente et complète de gestion des tickets techniques et de suivi des interventions, couvrant à la fois une **application web** (React.js / Node.js) destinée aux coordinatrices, managers et clients, et une **application mobile** (Flutter) dédiée aux techniciens de terrain.

Tout au long de ce travail, nous avons suivi une démarche structurée : étude du contexte et de l'existant, spécification des besoins fonctionnels et non fonctionnels, choix d'une architecture N-Tiers robuste, puis mise en œuvre progressive à travers cinq sprints organisés selon la méthodologie Scrum. Cette approche nous a permis de livrer, release après release, un socle applicatif solide (authentification sécurisée par OTP et JWT, gestion des utilisateurs, des clients, des équipements et des tickets), avant d'y greffer des fonctionnalités à forte valeur ajoutée : l'espace client en libre-service avec soumission de demandes et suivi de SLA, l'assistant intelligent **DocBot** basé sur Ollama/LLaMA 3 pour la classification automatique des incidents, la **détection automatique de récurrence** avec génération de rapports PDF ainsi qu'un système complet de **journalisation et de validation manager** garantissant l'intégrité des données sensibles.

Le fait de disposer, dès la fin du projet, d'une solution disponible **simultanément sur web et sur mobile**, alimentée par une seule et même API REST et une base de données MongoDB partagée, démontre la cohérence de l'architecture retenue et sa capacité à répondre aux usages variés des différents acteurs métier (bureau pour la coordination et le pilotage, terrain pour l'intervention). La modernisation du déploiement, à travers une pipeline CI/CD Jenkins/Docker éliminant les interruptions de service, achève de positionner TSS comme une plateforme industrialisable et prête à évoluer vers un usage en production à grande échelle.

Sur le plan personnel, ce stage nous a permis de consolider nos compétences en développement Full Stack (React, Node.js/Express, Flutter, MongoDB), de nous initier à l'intégration de modèles d'intelligence artificielle dans un contexte métier réel, et de nous familiariser avec les pratiques DevOps modernes (conteneurisation, intégration et déploiement continus). Cette expérience nous a également confrontés aux réalités du monde professionnel : contraintes de temps, travail en équipe, prise en compte des retours utilisateurs et arbitrages techniques.

Si la plateforme TSS répond pleinement aux besoins exprimés lors du cadrage du projet, plusieurs axes d'évolution plus ambitieux peuvent être envisagés afin de la faire progresser vers une solution encore plus intelligente, résiliente et industrielle :

- **Supervision temps réel des équipements (IoT)** : connecter directement les distributeurs automatiques de billets (DAB) via des capteurs , afin de générer automatiquement des tickets à partir d'anomalies détectées en temps réel, sans attendre la déclaration manuelle d'un incident.
- **Enrichissement multimodal de DocBot** : faire évoluer l'assistant IA vers une interaction vocale et une analyse d'images (photo de l'écran d'erreur du DAB, code d'erreur scanné) en s'appuyant sur un modèle de vision (*vision-language model*) afin de fiabiliser et d'accélérer davantage le diagnostic initial.
- **Observabilité et supervision d'infrastructure** : mettre en place une stack de monitoring (Prometheus/Grafana) et de centralisation des logs applicatifs (stack ELK) afin de suivre en temps réel la santé des services conteneurisés, détecter proactivement les incidents d'infrastructure et alimenter des tableaux de bord SRE.
- **Passage à Kubernetes et déploiement multi-environnements** : faire évoluer l'orchestration Docker Compose vers Kubernetes afin de garantir l'auto-scaling, la haute disponibilité et des déploiements *blue-green* ou *canary* sans aucune interruption, y compris en environnement de production réel chez TUNISYS.
- **Renforcement de la sécurité** : compléter le dispositif actuel (JWT, OTP, contrôle d'accès par rôle) par des tests d'intrusion réguliers, une politique de gestion des secrets (Vault) et une mise en conformité avec les référentiels de sécurité applicables au secteur bancaire et financier.

En conclusion, ce projet a constitué une expérience à la fois techniquement exigeante et humainement enrichissante. La plateforme TSS, disponible dès à présent sur web et sur mobile, répond de façon concrète aux besoins actuels de TUNISYS, tout en posant les fondations d'une solution évolutive, capable d'accompagner la transformation digitale de l'entreprise sur le long terme.

Bibliographie

[1] Société. *Tunisys*. https://www.tunisys.com.tn/. consulté le 5 Mai, 2026.

[2] application. *GLPPI*. https://www.glpi-project.org/fr/. consulté le 5 Mai, 2026.

[3] application. *IBM*. https://www.ibm.com/fr-fr/products/maximo. consulté le 5 Mai, 2026.

[4] platforme. *ServiceNow*. https://www.servicenow.com/. consulté le 2 janvier, 2021.

[5] site. *Mongodb*. https://www.mongodb.com/fr-fr/resources/basics/databases/nosql-explained. consulté le 5 Mai, 2026.

[6] IIS. *iis*. https://learn.microsoft.com/fr-fr/iis/get-started/introduction-to-iis/iis-web-server-overview. consulté le 5 Mai, 2026.

[7] GitHub. https://github.com/?locale=fr-fr/. consulté le 20 mars, 2021.

[8] Meta. *React – The library for web and native user interfaces*. https://react.dev/. consulté le 2 Juillet, 2026.

[9] OpenJS Foundation. *Node.js Documentation*. https://nodejs.org/en/docs. consulté le 8 janvier , 2026.

[10] OpenJS Foundation. *Express – Node.js web application framework*. https://expressjs.com/. consulté le 8 janvier, 2026.

[11] Mongoose. *Mongoose – Elegant MongoDB object modeling for Node.js*. https://mongoosejs.com/. consulté le 2 Juillet, 2026.

[12] Google. *Flutter Documentation*. https://docs.flutter.dev/. consulté 19 Mai , 2026.

[13] Docker Inc.. *Docker Documentation*. https://docs.docker.com/. consulté le 11 Juin , 2026.

[14] Docker Inc.. *Docker Compose Overview*. https://docs.docker.com/compose/. consulté le 11 Juin, 2026.

[15] Jenkins. *Jenkins User Documentation*. https://www.jenkins.io/doc/. consulté le 11 JUIN Juillet, 2026.

[16] Ollama. *Ollama – Get up and running with large language models locally*. https://ollama.com/. consulté le 20 Mai, 2026.

[17] Meta AI. *Meta Llama 3*. https://ai.meta.com/llama/. consulté le 20 Mai, 2026.

[18] Google. *Firebase Documentation*. https://firebase.google.com/docs. consulté le 20 Janvier, 2026.

[19] Socket.IO. *Socket.IO Documentation*. https://socket.io/docs/v4/. consulté le 12 Janvier , 2026.



ECOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES

www.esprit.tn - E-mail : contact@esprit.tn

Siége Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454